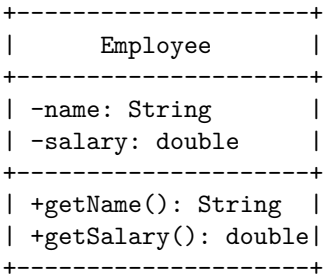
Software engineering is a set of engineering methods used in the software development of system applications. It defines principles for specification, design, development, testing, evaluation, and maintenance. There are different types of diagrams that can be used to represent the structure, behavior, and interactions of a software system, such as class diagrams, sequence diagrams, use case diagrams, activity diagrams, and component diagrams.

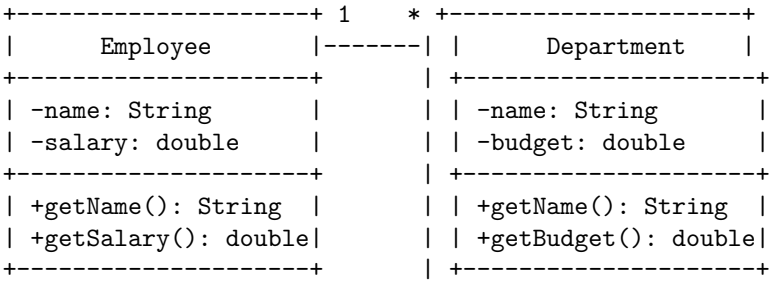
A class diagram is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among objects . A class diagram can be used to model the domain concepts, the design of the system, and the implementation details.

A class diagram consists of the following elements:

- Classes: A class is a template that defines the properties and behaviors of a set of objects. A class is represented by a rectangle with three compartments: the top compartment shows the class name, the middle compartment shows the class attributes, and the bottom compartment shows the class operations. For example:

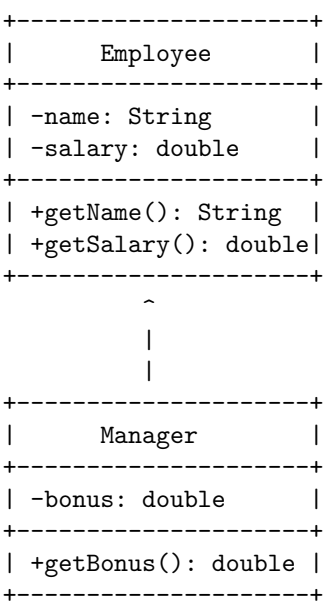


- Associations: An association is a relationship between two or more classes that indicates how the objects of those classes are connected. An association is represented by a solid line connecting the classes, with optional multiplicity and role labels at the ends. For example:



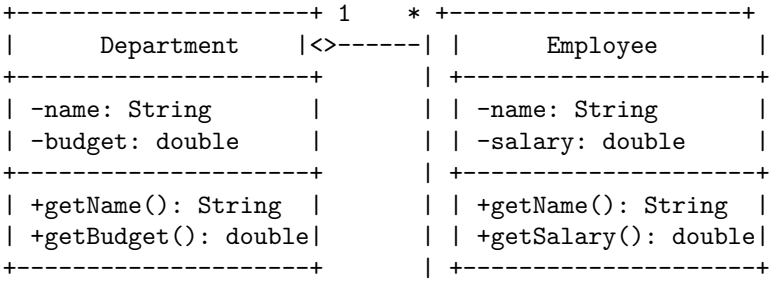
This association means that one employee can belong to many departments, and one department can have many employees. The role labels indicate the name of the association from the perspective of each class.

- Generalizations: A generalization is a relationship between a more general class (the superclass) and a more specific class (the subclass) that indicates that the subclass inherits the properties and behaviors of the superclass. A generalization is represented by a solid line with a hollow triangle at the end pointing to the superclass. For example:



This generalization means that a manager is a special kind of employee, and inherits the name, salary, getName, and getSalary attributes and operations from the employee class. The manager class also has its own bonus and getBonus attributes and operations.

- Aggregations: An aggregation is a relationship between a whole class and its parts that indicates that the parts can exist independently of the whole. An aggregation is represented by a solid line with a hollow diamond at the end pointing to the whole. For example:



This aggregation means that a department is composed of many employees, but the employees can exist without the department.

- Compositions: A composition is a relationship between a whole class and its parts that indicates that the parts cannot exist independently of the whole. A composition is represented by a solid line with a filled diamond at the end pointing to the whole. For example:

```
+---------------------+ 1    * +---------------------
```

Unit 1 - Introduction to Software Engineering

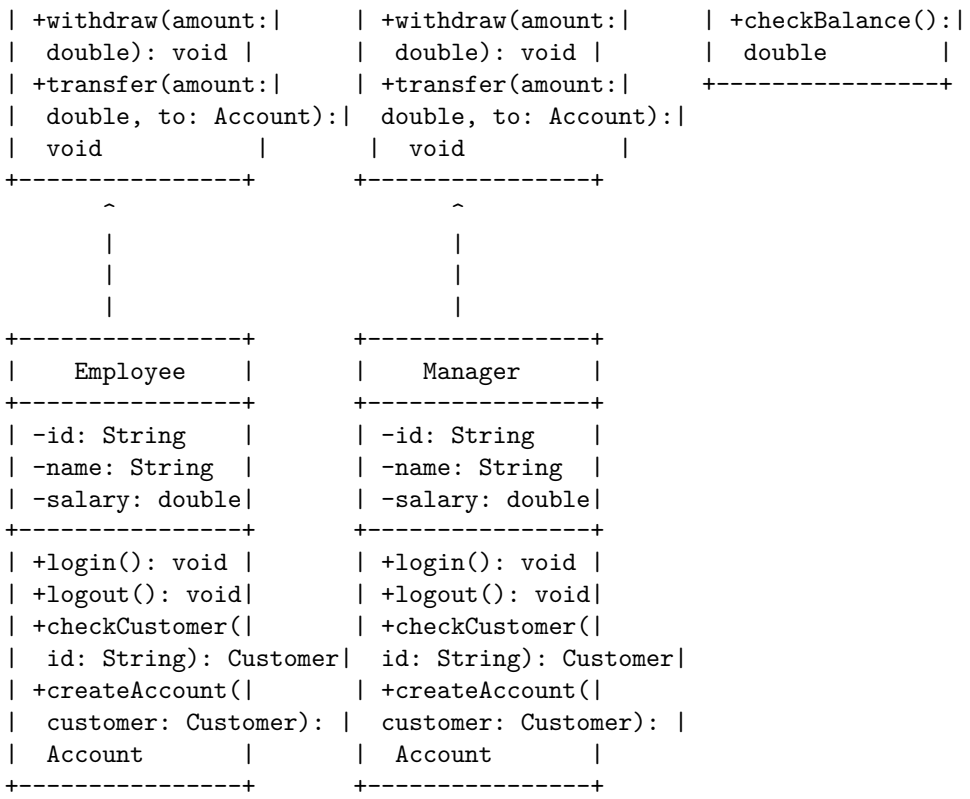
One of the diagrams that can be used to introduce software engineering is the class diagram. A class diagram is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among objects. A class diagram can be used to model the logical design of a software system, as well as the physical design of a database or a component.

A class diagram consists of the following elements:

- Classes: A class is a blueprint for an object. It defines the properties and behaviors of a group of objects that share the same characteristics. A class is represented by a rectangle with the class name at the top, followed by the attributes and operations in separate compartments.
- Attributes: An attribute is a property or characteristic of a class. It defines the state or data of an object. An attribute is represented by a name and a type, optionally followed by a visibility indicator and an initial value.
- Operations: An operation is a function or method that defines the behavior or action of a class. It specifies what an object can do or how it can interact with other objects. An operation is represented by a name and a parameter list, optionally followed by a visibility indicator and a return type.
- Relationships: A relationship is a connection or association between two or more classes. It defines how the classes interact or depend on each other. There are different types of relationships, such as inheritance, association, aggregation, composition, and dependency. A relationship is represented by a line or an arrow between the classes, optionally labeled with a name, a multiplicity, and a role.

The following diagram illustrates the basic structure of a class diagram using an example of a bank system:

```
+----------------+       +----------------+       +----------------+
|    Customer    |       |    Account     |       |    BankCard    |
+----------------+       +----------------+       +----------------+
| -name: String  |       | -number: String|       | -number: String|
| -address: String|      | -balance: double|      | -pin: int      |
+----------------+       +----------------+       +----------------+
| +deposit(amount:|      | +deposit(amount:|      | +withdraw(amount:|
|  double): void |<>-----|  double): void |<>-----|  double): void |
```



The diagram shows that:

- A Customer class has attributes name and address, and operations deposit, withdraw, and transfer. A Customer class has an aggregation relationship with an Account class, meaning that a customer can have one or more accounts, but the accounts can exist independently of the customer.
- An Account class has attributes number and balance, and operations deposit, withdraw, and transfer. An Account class has a composition relationship with a BankCard class, meaning that an account has one or more bank cards, and the bank cards cannot exist without the account.
- A BankCard class has attributes number and pin, and operations withdraw and checkBalance.
- An Employee class has attributes id, name, and salary, and operations login, logout, and checkCustomer. An Employee class has an inheritance relationship with a Manager class, meaning that a manager is a special type of employee

Introduction to Software Engineering

Software engineering is the application of engineering principles and practices to the development and maintenance of software systems. Software engineering

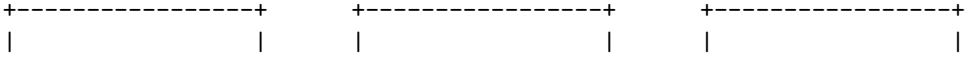
covers a wide range of activities, such as:

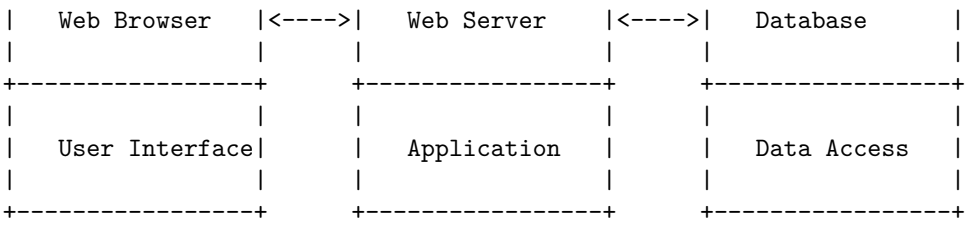
- Requirements analysis: The process of eliciting, analyzing, and documenting the needs and expectations of the stakeholders of a software system.
- Design: The process of defining the structure, behavior, and interfaces of the software components and subsystems.
- Implementation: The process of writing, testing, and debugging the source code of the software system.
- Testing: The process of verifying and validating that the software system meets the specified requirements and quality standards.
- Deployment: The process of installing, configuring, and running the software system in the target environment.
- Maintenance: The process of correcting, improving, and adapting the software system to changing requirements, technologies, and user feedback.

One of the common ways to represent and communicate the software engineering process is by using diagrams. Diagrams are graphical models that show the elements, relationships, and properties of a software system or a software engineering activity. There are different types of diagrams for different purposes, such as:

- Class diagram: A type of static structure diagram that shows the classes, attributes, operations, and associations of a software system. A class diagram can be used to model the domain concepts, the design of the software components, or the database schema of the software system.
- Sequence diagram: A type of interaction diagram that shows the messages exchanged between the objects or actors of a software system over time. A sequence diagram can be used to model the dynamic behavior, the use cases, or the test scenarios of the software system.
- Activity diagram: A type of behavior diagram that shows the actions, decisions, and flows of a software system or a software engineering activity. An activity diagram can be used to model the business processes, the workflows, or the algorithms of the software system or the software engineering activity.
- Component diagram: A type of static structure diagram that shows the components, interfaces, and dependencies of a software system. A component diagram can be used to model the architecture, the deployment, or the integration of the software system.
- State diagram: A type of behavior diagram that shows the states, transitions, and events of an object or a subsystem of a software system. A state diagram can be used to model the lifecycle, the state machine, or the protocol of an object or a subsystem of the software system.

The following diagram illustrates the basic architecture of a software system using a component diagram:





The diagram shows that the software system consists of three components: a web browser, a web server, and a database. The web browser provides the user interface for the software system, the web server provides the application logic for the software system, and the database provides the data access for the software system. The components communicate with each other using interfaces and dependencies. The web browser depends on the web server, and the web server depends on the database. The web browser and the web server use the HTTP protocol to exchange messages, and the web server and the database use the SQL protocol to exchange queries and results.

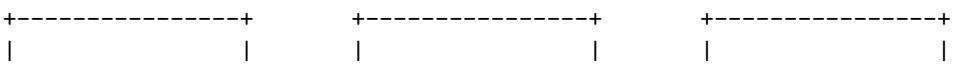
A software component diagram is a type of UML diagram that shows the structure and dependencies of the components of a software system. A component is a modular unit that provides a specific functionality or a set of functionalities. A component can be a software module, a library, a framework, a hardware device, or a business unit.

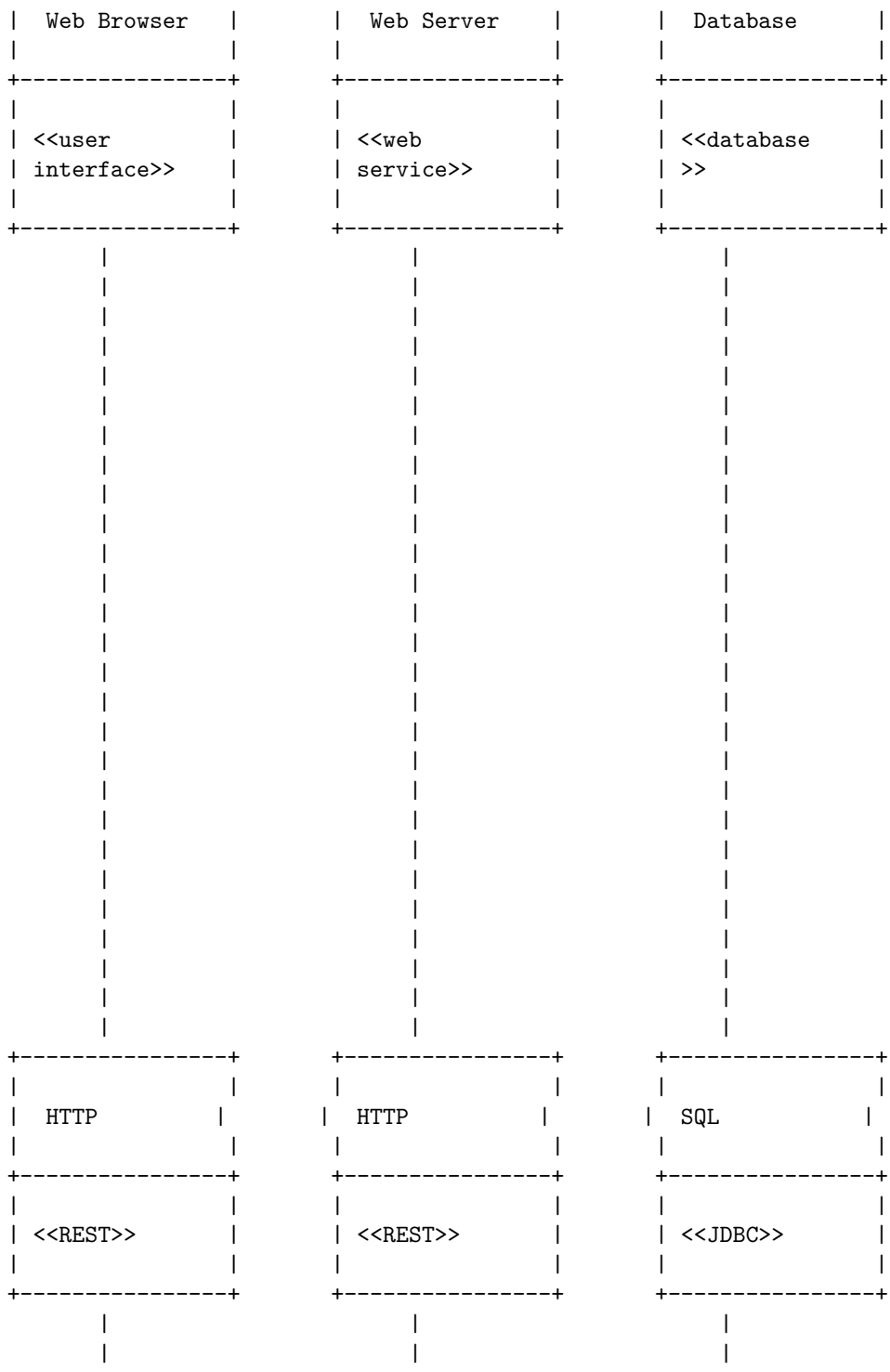
A software component diagram consists of the following elements:

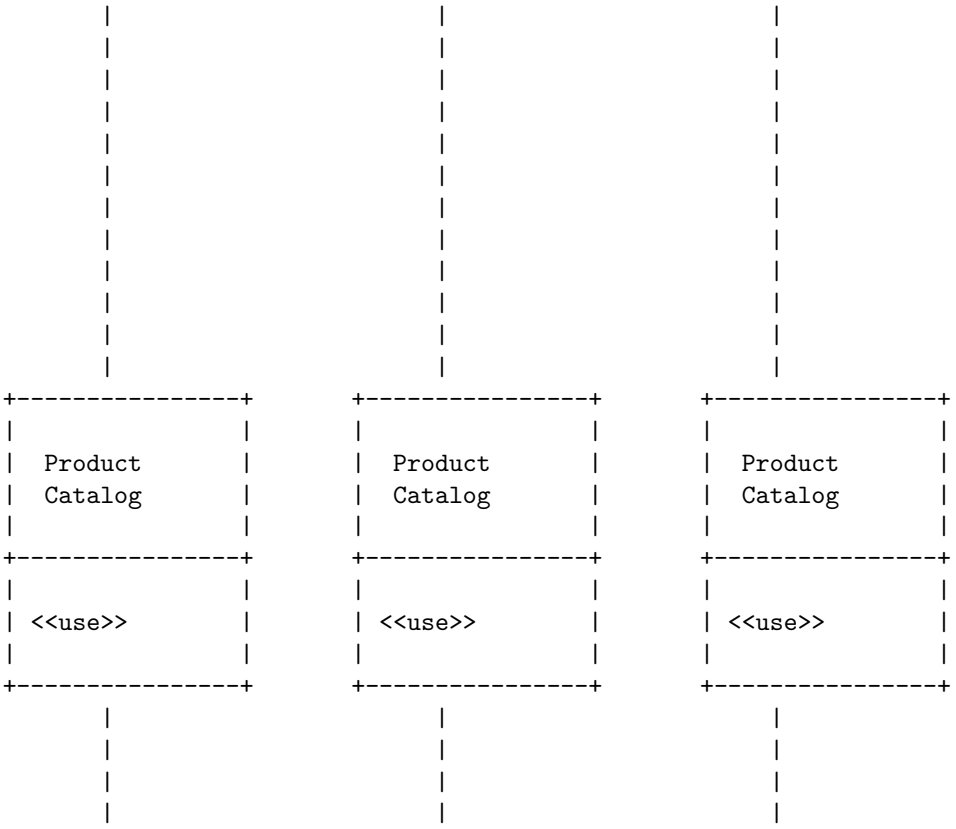
- Components: Represented by rectangles with two small rectangles on the left side. The name of the component is written inside the rectangle. Optionally, the component can have a stereotype, such as <>, <>, <>, etc. to indicate its type or role.
- Interfaces: Represented by circles or lollipops. They show the services that a component provides or requires. The name of the interface is written next to the circle. Optionally, the interface can have a stereotype, such as <>, <>, <>, etc. to indicate its protocol or technology.
- Dependencies: Represented by dashed arrows with an open arrowhead. They show the relationships between components or interfaces. The arrow points from the dependent element to the independent element. Optionally, the dependency can have a stereotype, such as <>, <>, <>, etc. to indicate its nature or purpose.

Software Components

The following is an example of a software component diagram for an online shopping system. It shows the components and interfaces of the system, as well as their dependencies.




Web Browser
<<user interface>>
Web Server
<<web service>>
Database
<<database >>
HTTP
<<REST>>
HTTP
<<REST>>
SQL
<<JDBC>>



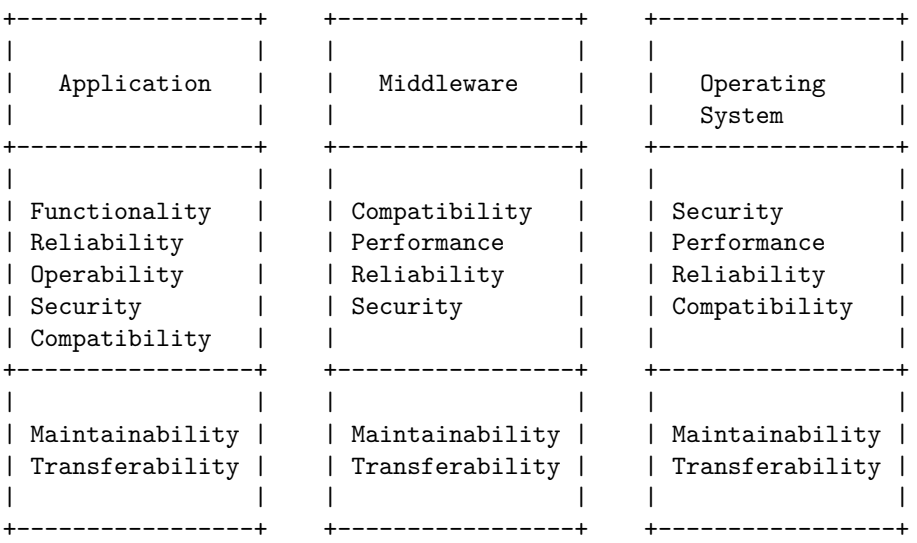
Software is a set of instructions, data or programs used to operate computers and execute specific tasks. Software characteristics are the qualities or features that describe the software and affect its performance, usability, reliability, security, etc. Software characteristics are classified into six major components :

- Functionality: Refers to the degree of performance of the software against its intended purpose.
- Reliability: Refers to the ability of the software to provide desired functionality under the given conditions.
- Operability: Refers to the ease of use and learnability of the software by the users.
- Performance efficiency: Refers to the responsiveness, resource utilization, and scalability of the software.
- Security: Refers to the protection of the software from unauthorized access, modification, or damage.
- Compatibility: Refers to the ability of the software to coexist and interact with other software or systems.
- Maintainability: Refers to the ease of modifying, testing, and correcting the software.
- Transferability: Refers to the ease of adapting, installing, and deploying

the software in different environments.

Software Characteristics

The following diagram illustrates the basic architecture of a software system and how the software characteristics are related to it:

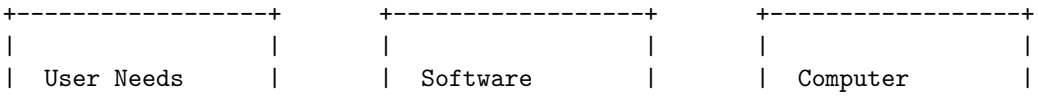


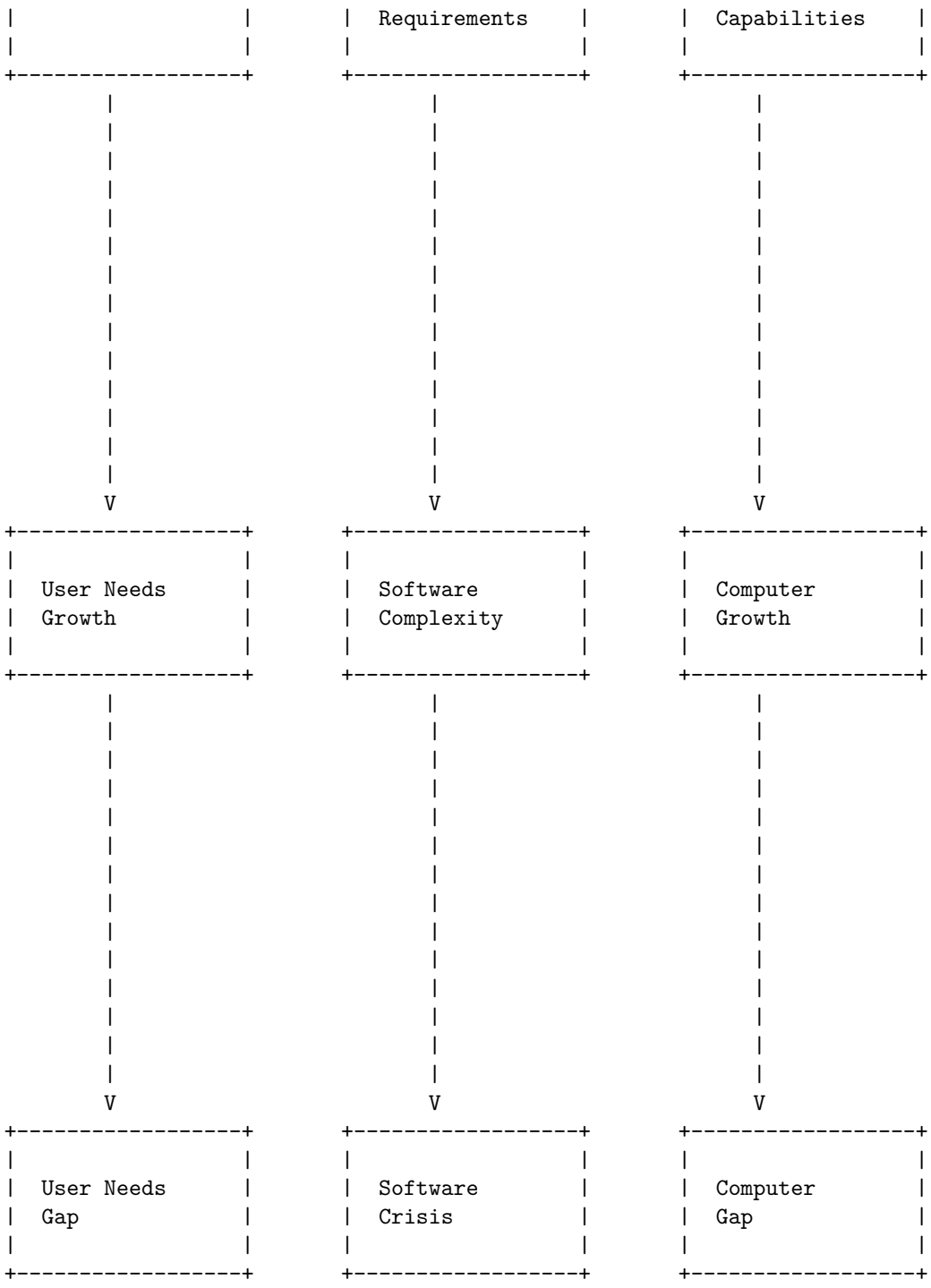
Software crisis is a term used in computer science for the difficulty of writing useful and efficient computer programs in the required time. The software crisis was due to the rapid increases in computer power and the complexity of the problems that could not be tackled . Some of the causes of the software crisis were:

- Lack of proper training and skills of the software developers
- Lack of standardization and documentation of the software development process
- Lack of effective tools and techniques for software design, testing, and maintenance
- Lack of communication and coordination among the software stakeholders
- Lack of understanding of the user requirements and expectations
- Lack of quality assurance and reliability of the software products
- Lack of management and control of the software projects

Software Crisis

The following diagram illustrates the software crisis using ASCII art:



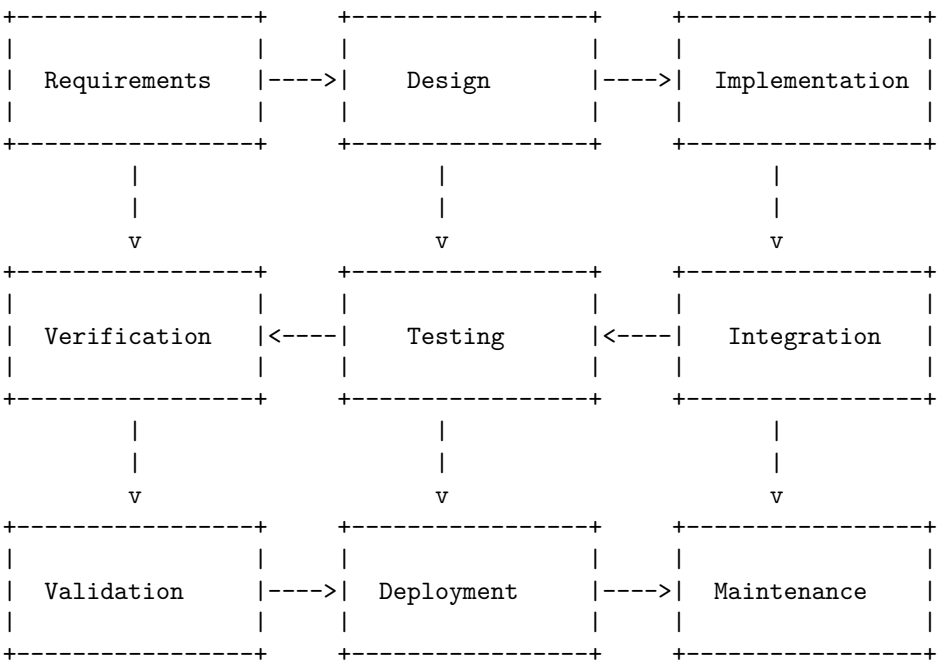

Requirements
Capabilities
User Needs Growth
Software Complexity
Computer Growth
User Needs Gap
Software Crisis
Computer Gap

The diagram shows that the user needs, the software requirements, and the computer capabilities are initially aligned, but they grow at different rates over time. The user needs and the computer capabilities grow faster than the software requirements, creating gaps between them. These gaps lead to the software crisis, which is the inability of the software to meet the user needs and the computer capabilities. The software crisis can result in software failures, delays, cost overruns, and dissatisfaction.

Software engineering processes refer to the methods and techniques used to develop and maintain software. They ensure that the final product meets the client's requirements and quality standards. There are different types of software engineering processes, such as waterfall, agile, lean, and traditional/waterfall. Each process has its own advantages and disadvantages, depending on the environmental, organizational, and product constraints.

The following diagram illustrates the basic architecture of a software engineering process using ASCII art:

Software Engineering Processes



Similarity and Differences from Conventional Engineering Processes

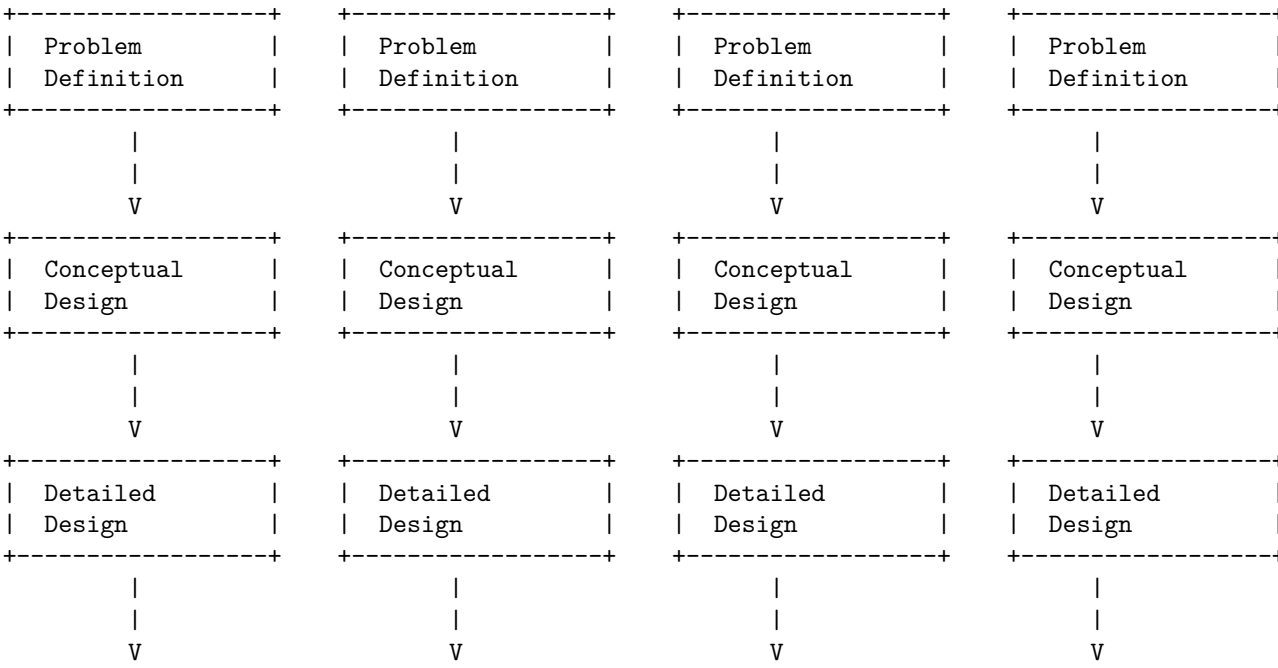
Conventional engineering processes are the methods and techniques used to design, build, test, and maintain physical systems or products, such as bridges, buildings, machines, vehicles, etc. Software engineering processes are the meth-

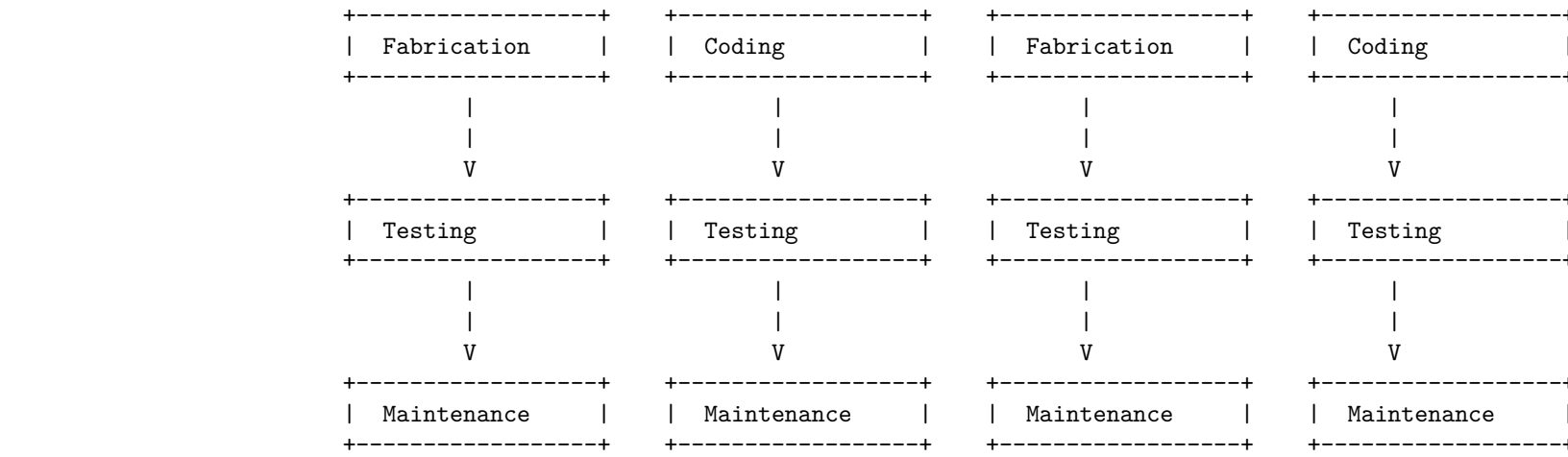
ods and techniques used to design, build, test, and maintain software systems or products, such as applications, websites, databases, etc.

Some of the similarities and differences between conventional engineering processes and software engineering processes are:

- Similarities:
  - Both are getting automated slowly.
  - Both require in-depth knowledge of their field.
  - Both follow a systematic and iterative approach to solve problems.
  - Both involve creativity, innovation, and teamwork.
- Differences:
  - Conventional engineering processes deal with tangible and physical entities, while software engineering processes deal with intangible and logical entities.
  - Conventional engineering processes have higher government sector opportunity, while software engineering processes have more opportunities of foreign settlement.
  - Conventional engineering processes have more standardized and regulated practices, while software engineering processes have more diverse and evolving practices.
  - Conventional engineering processes have a more physically active role, while software engineering processes have a typical office job.

The following diagram illustrates the basic architecture of a conventional engineering process and a software engineering process using ASCII art:





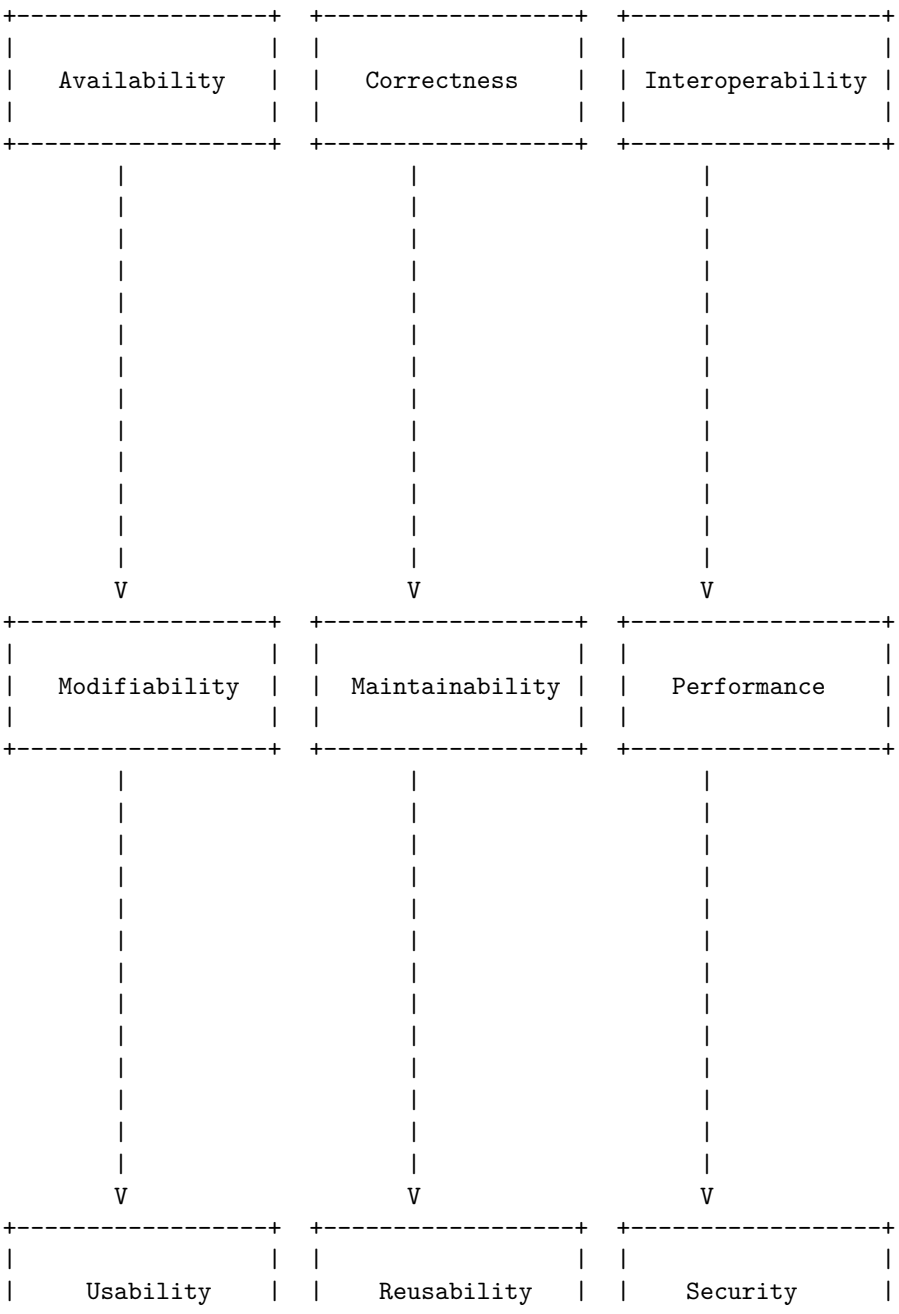
Conventional Engineering Process (Waterfall Model) | Software Engineering Process (Waterfall Model) | Conventional Engineering Process (Spiral Model) | Software Engineering Process (Spiral Model)

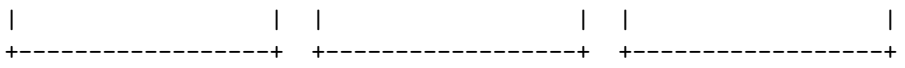
Software quality attributes are the non-functional requirements of software that can affect its quality, performance, usability, and maintainability. Some of the common software quality attributes are:

- Availability: The degree to which a software system is accessible and operational when required by the users.
- Correctness: The degree to which a software system performs its intended functions without errors or defects.
- Interoperability: The degree to which a software system can exchange data and services with other systems or components.
- Modifiability: The degree to which a software system can be modified or adapted to meet changing requirements or environments.
- Maintainability: The degree to which a software system can be repaired, updated, or improved with minimal effort and cost.
- Performance: The degree to which a software system meets the speed, response time, throughput, or resource consumption requirements of the users or stakeholders.
- Usability: The degree to which a software system is easy to learn, use, and understand by the users or stakeholders.
- Reusability: The degree to which a software system or component can be reused in other systems or contexts.
- Security: The degree to which a software system protects the confidentiality, integrity, and availability of the data and services it provides or consumes.

Software Quality Attributes

The following diagram illustrates the basic architecture of a software quality attributes model using ASCII characters:





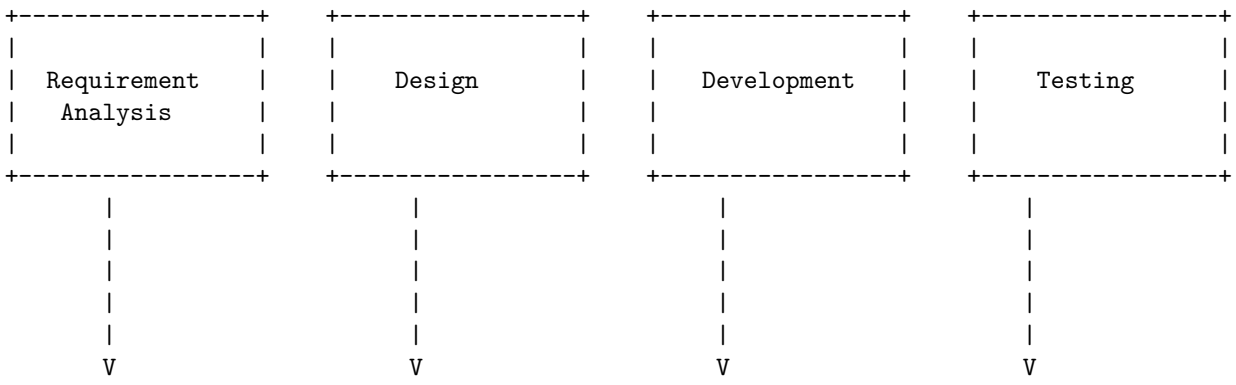
The diagram shows that each software quality attribute is connected to one or more other attributes, indicating that they are interdependent and influence each other. For example, improving the availability of a software system may affect its performance or security, and vice versa. Therefore, software quality attributes should be considered holistically and balanced according to the needs and priorities of the users and stakeholders.

Software Development Life Cycle (SDLC) Models are frameworks that describe the operations performed at each phase of a software development project. There are different types of SDLC models, such as waterfall, V-shaped, prototype, spiral, iterative incremental, agile, etc. Each model has its own advantages and disadvantages, depending on the nature and scope of the project.

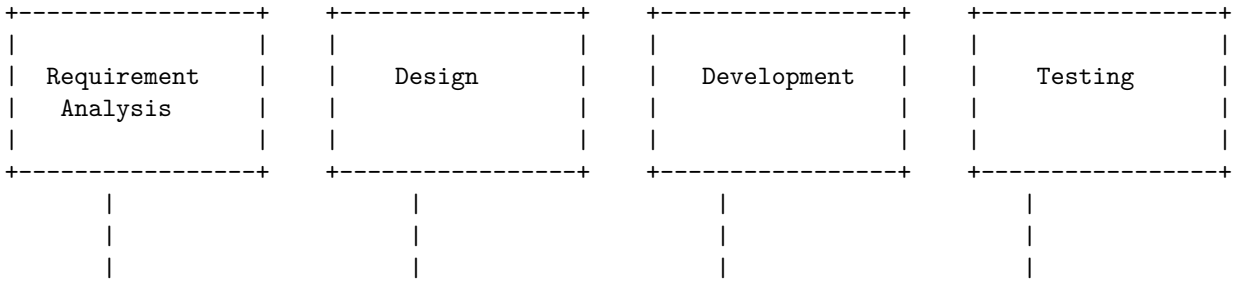
Software Development Life Cycle (SDLC) Models

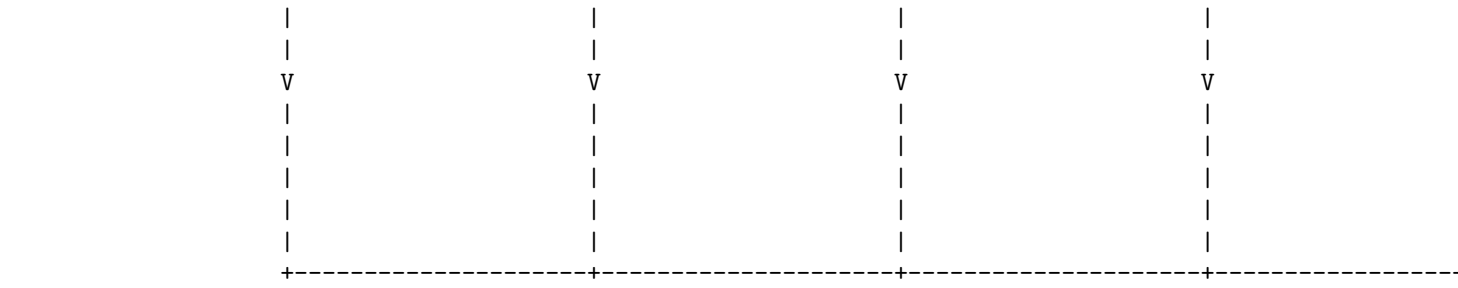
The following is a detailed ASCII diagram for some of the common SDLC models:

Waterfall Model:

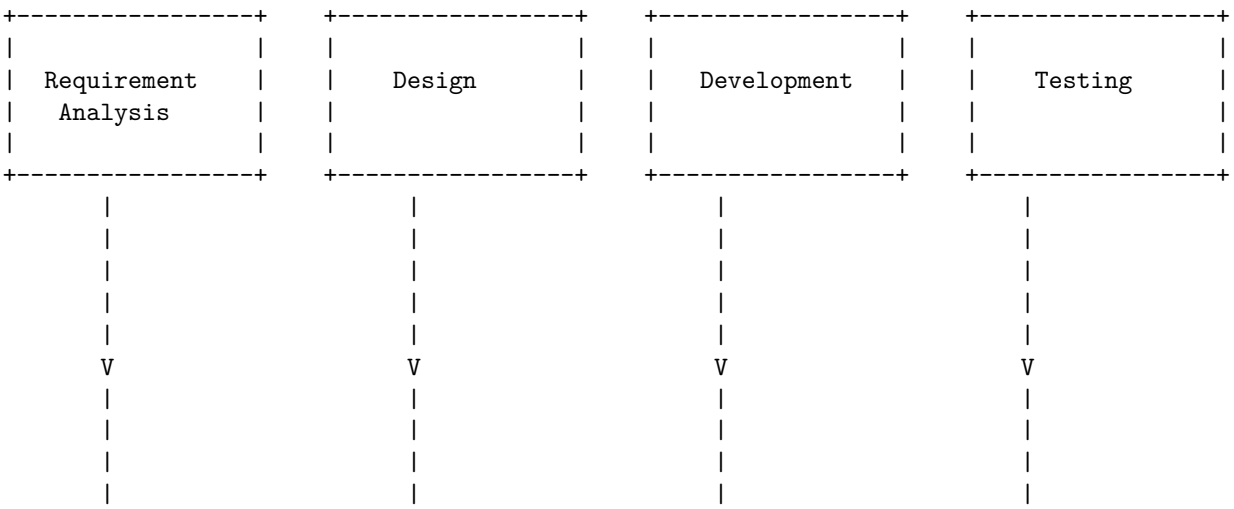


V-Shaped Model:





Prototype Model:



The waterfall model is a linear, sequential approach to the software development lifecycle (SDLC) that is popular in software engineering and product development. The waterfall model uses a logical progression of SDLC steps for a project, similar to the direction water flows over the edge of a cliff. The waterfall model is the earliest SDLC approach that was used for software development.

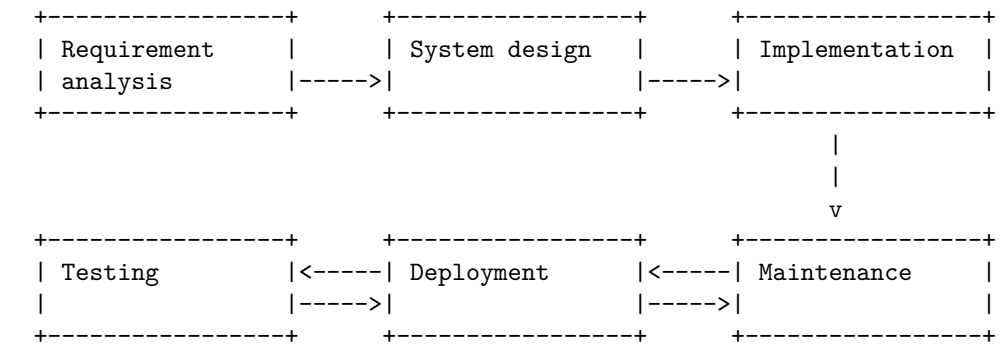
The waterfall model divides the software development process into separate phases. Each phase has a specific goal and deliverable. The outcome of one phase acts as the input for the next phase sequentially. The phases of the waterfall model are:

- **Requirement analysis**: In this phase, the project team gathers the requirements from the customer and documents them in a specification document.
- **System design**: In this phase, the project team designs the system architecture and the database schema based on the requirements.
- **Implementation**: In this phase, the project team writes the code and tests the modules of the system.

- **Testing**: In this phase, the project team performs system testing, integration testing, and user acceptance testing to ensure that the system meets the requirements and is free of defects.
- **Deployment**: In this phase, the project team deploys the system to the production environment and provides training and support to the end-users.
- **Maintenance**: In this phase, the project team provides bug fixes, enhancements, and updates to the system as per the customer feedback and changing needs.

Water Fall Model in SDLC

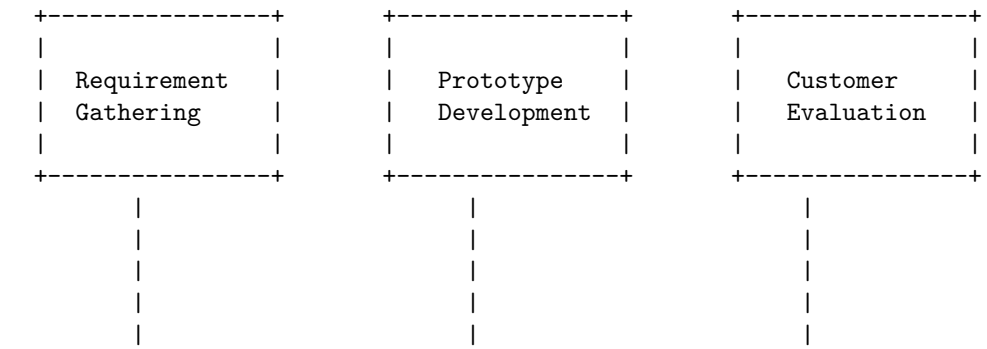
The following diagram illustrates the basic architecture of the waterfall model in SDLC using ASCII art:

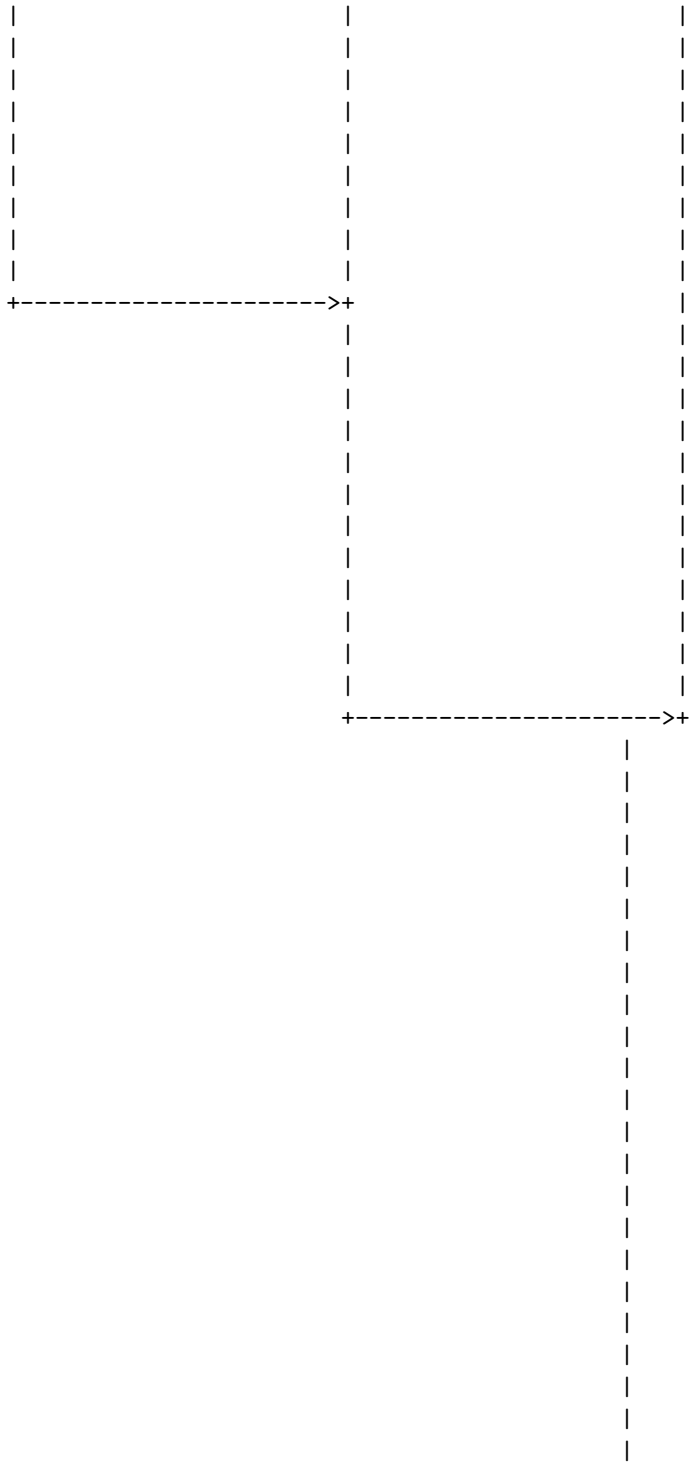


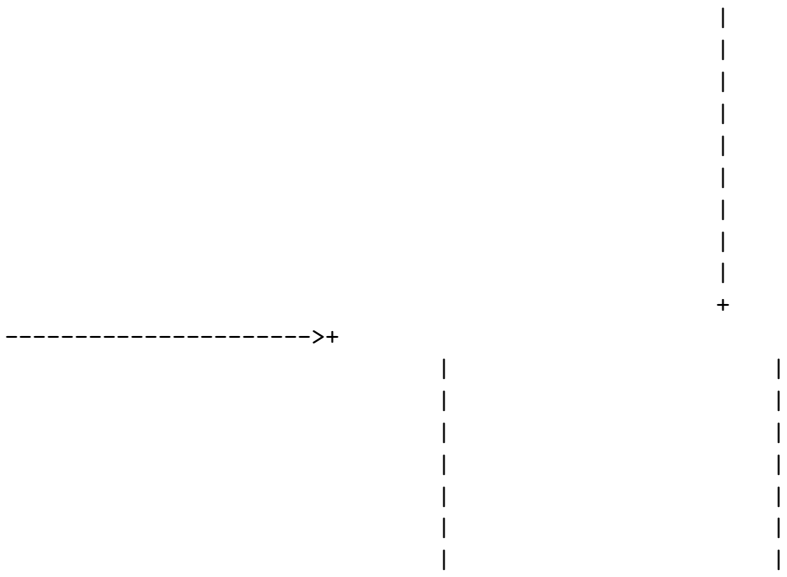
The prototype model is a software development life cycle (SDLC) model in which a prototype is built, tested, and then reworked as necessary until an acceptable prototype is finally achieved from which the complete system or product can be developed.

The following diagram illustrates the basic architecture of a prototype model in SDLC:

Prototype Model in SDLC



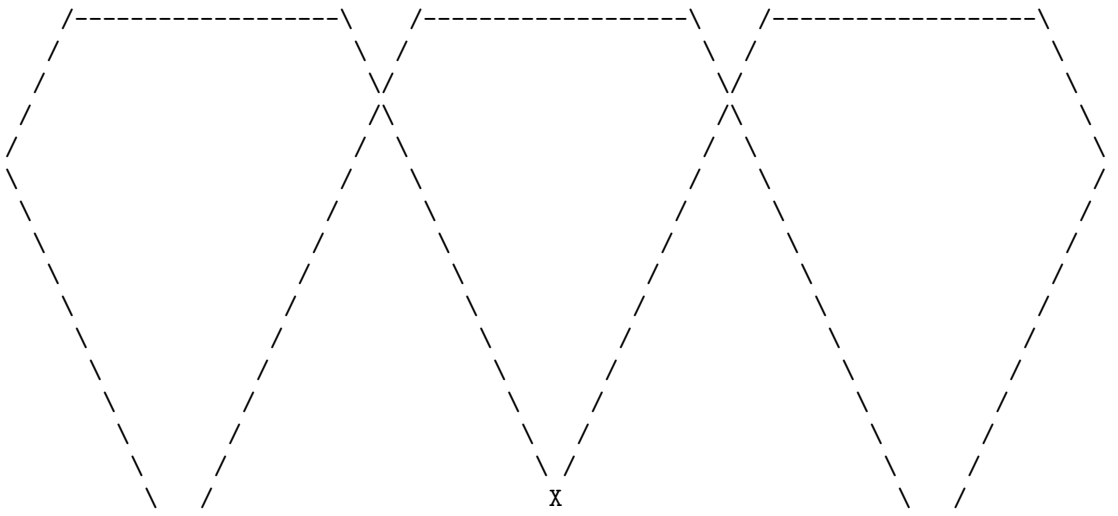


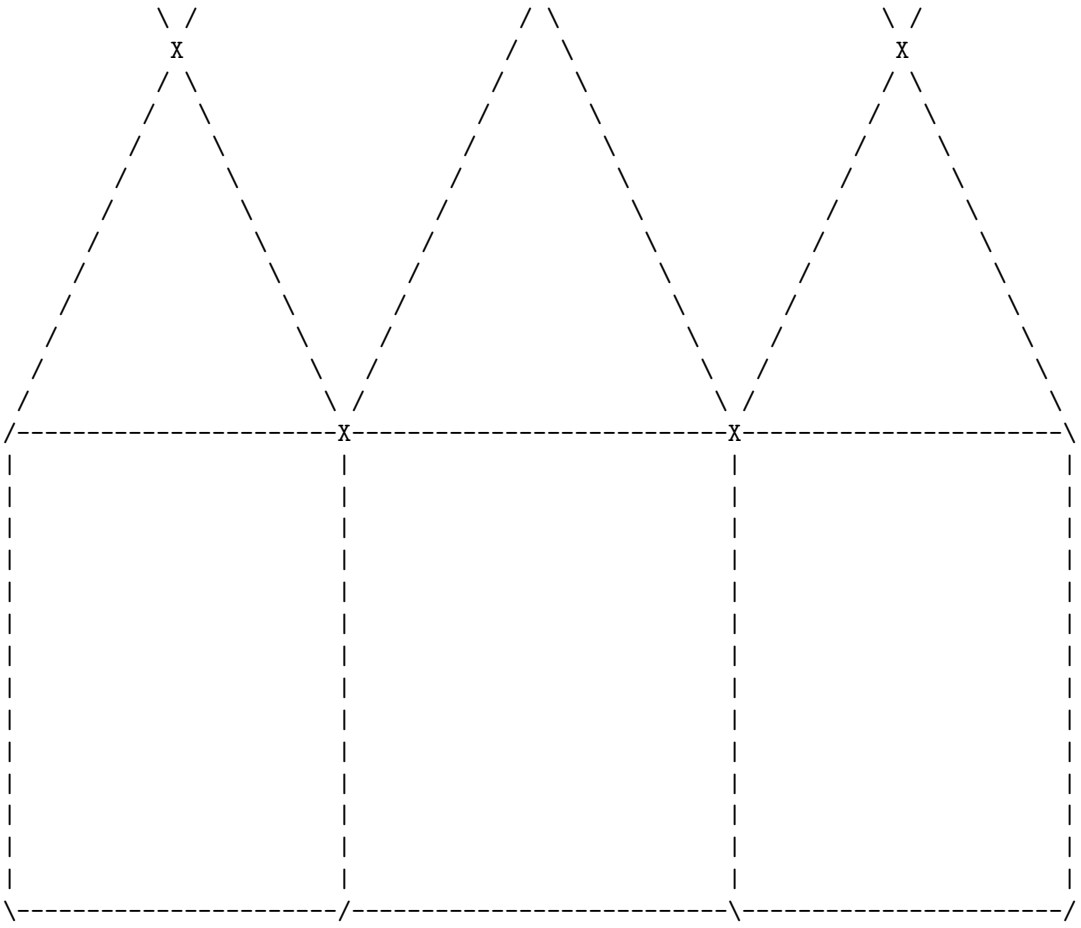


The spiral model is a software development life cycle (SDLC) model that provides a systematic and iterative approach to software development. It is based on the idea of a spiral, with each iteration of the spiral representing a complete software development cycle, from requirements gathering and analysis to design, implementation, testing, and deployment. The spiral model is used for risk management and is favored for large, expensive, and complicated projects.

Spiral Model in SDLC

The following diagram illustrates the basic architecture of a spiral model in SDLC:





Each loop of the spiral consists of four phases:

- **Planning**: In this phase, the objectives, alternatives, and constraints of the project are defined. The risks involved in the project are identified and analyzed, and a risk management plan is developed.
- **Risk analysis**: In this phase, the risks identified in the planning phase are evaluated and mitigated. The feasibility and cost-effectiveness of the project are assessed, and the best alternative is chosen.
- **Engineering**: In this phase, the software product is designed, implemented, tested, and integrated. The quality standards and requirements are ensured, and the deliverables are produced.
- **Evaluation**: In this phase, the software product is reviewed and evaluated by the customer and other stakeholders. The feedback and suggestions are collected, and the project is refined and improved.

The spiral model is repeated until the software product meets the customer's expectations and requirements. The number and size of the loops depend on

the complexity and scope of the project. The advantages of the spiral model are:

- It allows for flexibility and changes in the requirements and specifications of the project.
- It reduces the risk of failure and ensures customer satisfaction and involvement.
- It provides a realistic estimate of the cost, time, and resources required for the project.
- It supports the development of large and complex software systems.

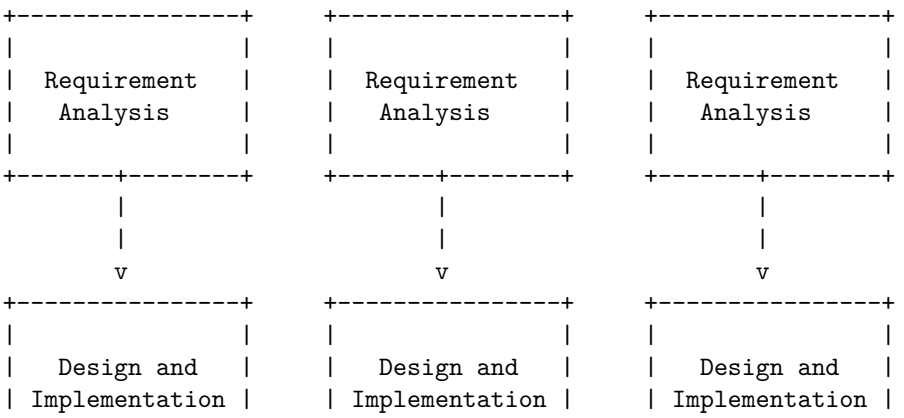
The disadvantages of the spiral model are:

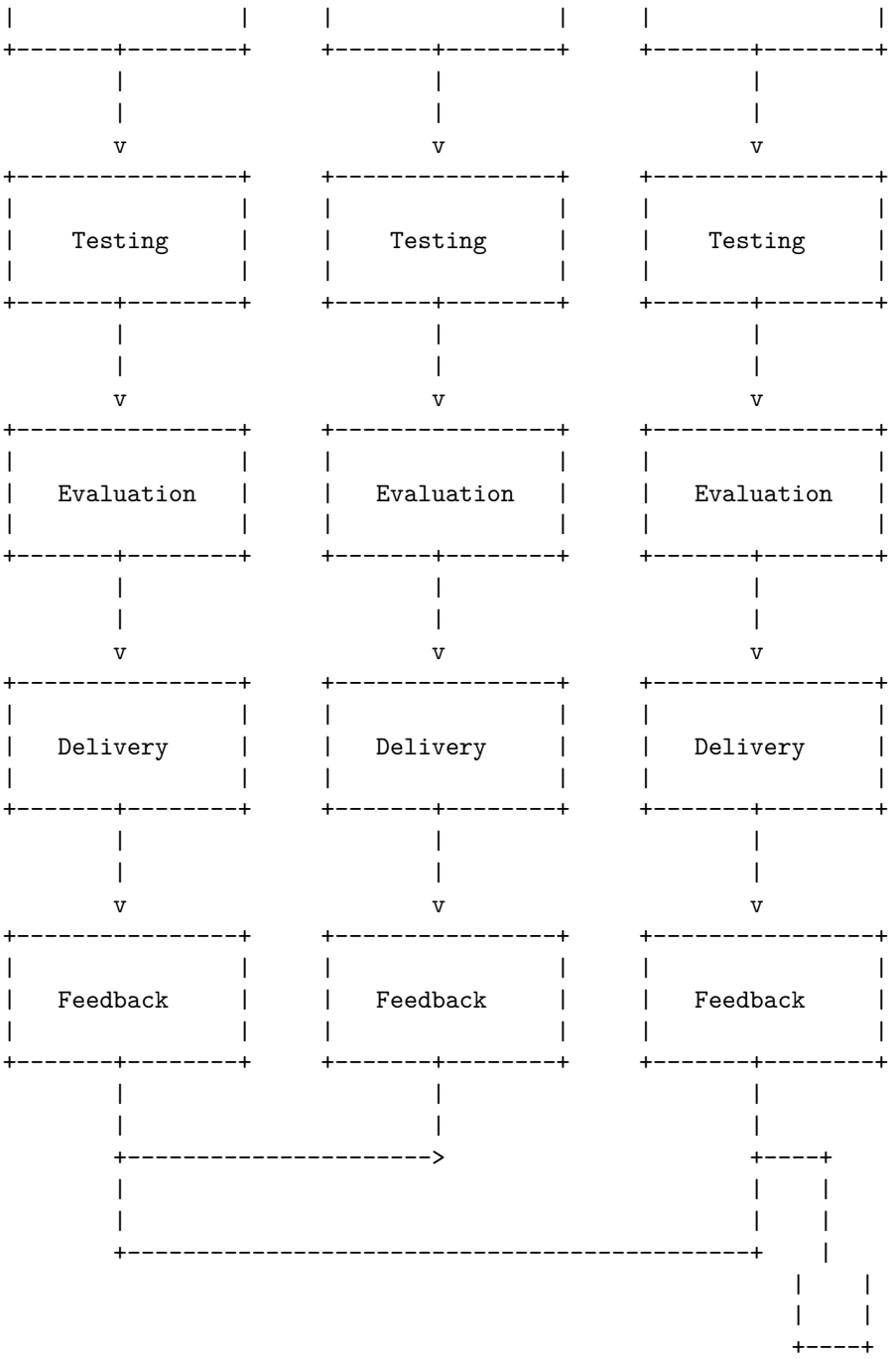
- It requires a high level of expertise and experience in risk management and analysis.
- It can be costly and time-consuming due to the frequent iterations and reviews.
- It can be difficult to define the objectives and scope of the project in the early stages.
- It can be challenging to maintain the documentation and control of the project.

Evolutionary Development Models in SDLC are a group of software development methodologies that aim to deliver software products incrementally, through a series of iterations, rather than in a single, final version. The main advantage of evolutionary models is that they can accommodate changing requirements and feedback from customers or users, and deliver software faster and with higher quality. Some examples of evolutionary models are prototyping, spiral, incremental, and agile models.

Evolutionary Development Models in SDLC

The following diagram shows a generic representation of an evolutionary development model in SDLC:





Each iteration consists of the following phases:

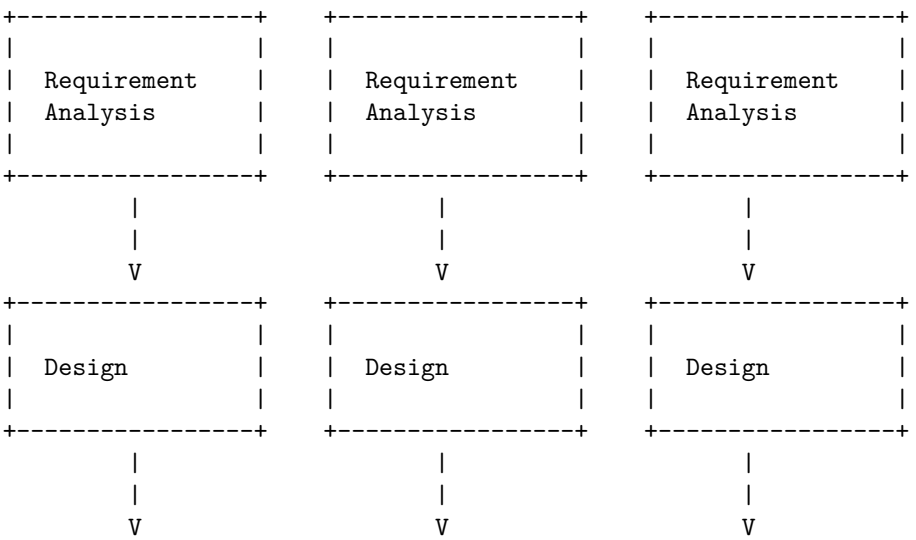
- Requirement Analysis: The requirements of the software are gathered

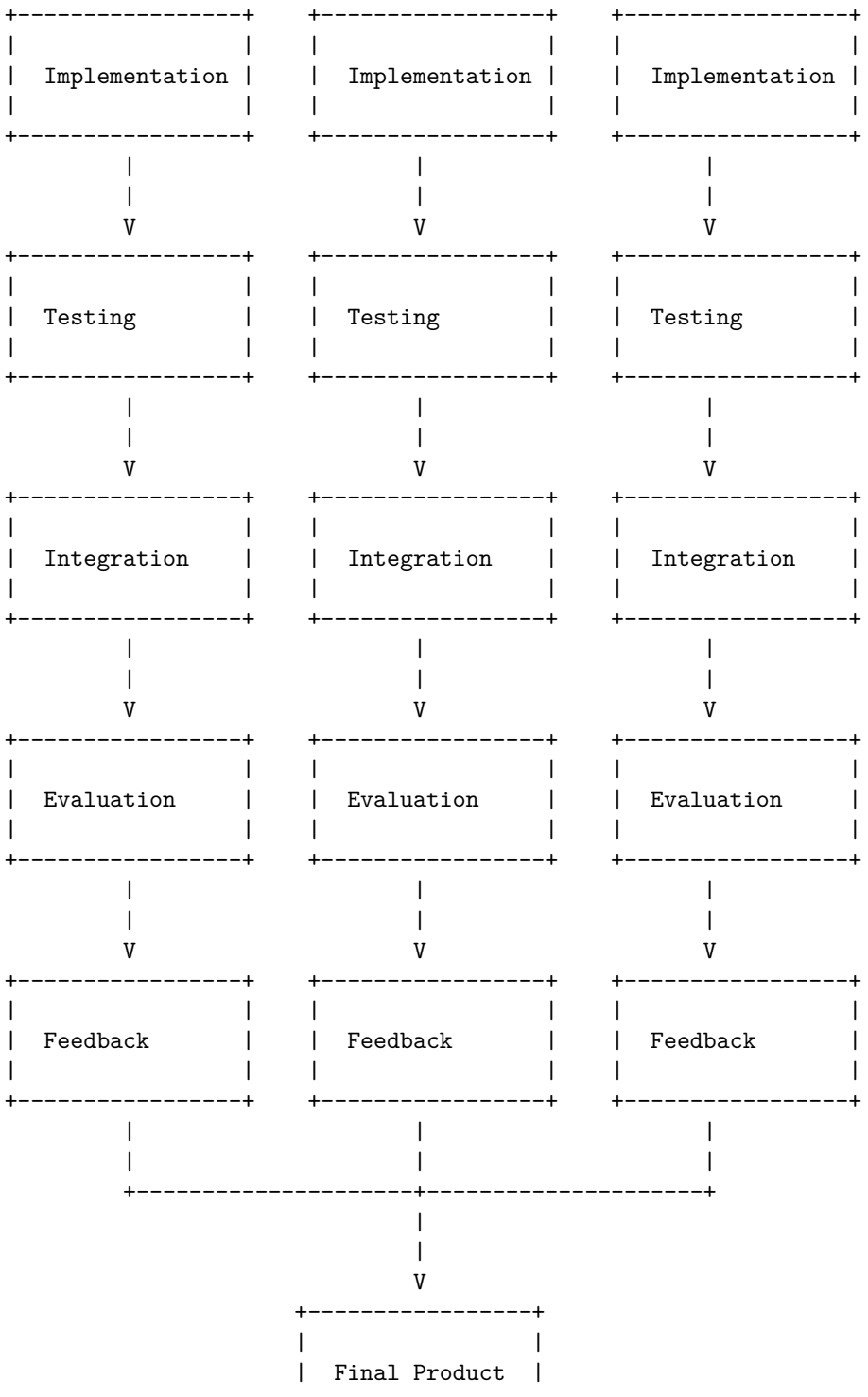
and analyzed, either from the customer or from the previous iteration's feedback.
- Design and Implementation: The software is designed and implemented according to the requirements and the chosen architecture.
- Testing: The software is tested to ensure that it meets the quality standards and the functional and non-functional requirements.
- Evaluation: The software is evaluated by the customer or the user, and the feedback is collected for the next iteration.
- Delivery: The software is delivered to the customer or the user, either as a prototype or as a final product.
- Feedback: The feedback from the customer or the user is used to improve the software or to change the requirements for the next iteration.

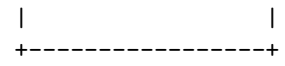
The number and duration of iterations depend on the complexity and scope of the software, the customer's expectations, and the development team's capabilities. The iterations can be planned in advance or decided dynamically, depending on the chosen evolutionary model. The iterations can also overlap or run in parallel, to speed up the development process and reduce the risks. The final iteration should deliver a complete and satisfactory software product that meets all the requirements and expectations of the customer or the user.

Iterative Enhancement Models in SDLC are a way to create software by breaking down the build into manageable components. Each component is implemented, tested and integrated in an iterative cycle until the complete system is ready. The iterative model allows for feedback and changes in the requirements during the development process. The following diagram illustrates the basic architecture of an iterative enhancement model in SDLC :

Iterative Enhancement Models in SDLC




Implementation
Testing
Integration
Evaluation
Feedback
Implementation
Testing
Integration
Evaluation
Feedback
Implementation
Testing
Integration
Evaluation
Feedback
Final Product



Unit 2 - Software Requirement Specifications (SRS)

A software requirement specification (SRS) is a document that describes the features, functions, and constraints of a software system. It also specifies the quality attributes, performance criteria, and design constraints of the system.

A possible ascii diagram for an SRS document is:

```
+------------------------+
| SRS Document           |
+------------------------+
| Introduction           |
| Functional Requirements|
| Non-functional Requirements|
| System Models          |
| Glossary               |
| References             |
+------------------------+
```

The introduction section provides an overview of the software system, its purpose, scope, objectives, and assumptions. It also identifies the stakeholders, users, and intended audience of the system.

The functional requirements section describes the services and behaviors that the system must provide to the users and other systems. It also specifies the inputs, outputs, and interactions of the system.

The non-functional requirements section describes the quality attributes and constraints that the system must satisfy, such as performance, reliability, security, usability, maintainability, etc.

The system models section provides graphical and textual representations of the system, such as use case diagrams, data flow diagrams, entity-relationship diagrams, state transition diagrams, etc. These models help to illustrate the structure, behavior, and interactions of the system.

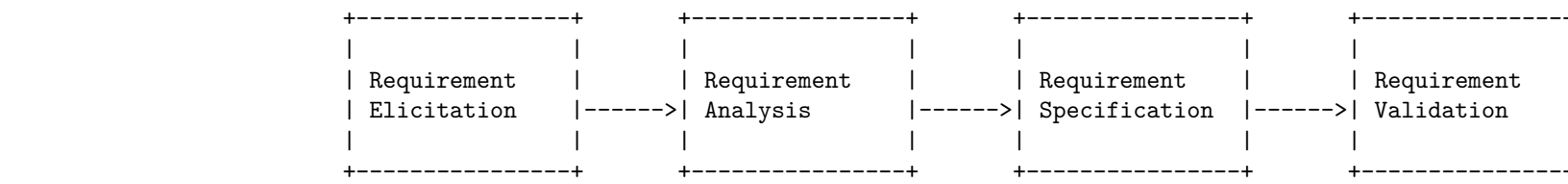
The glossary section defines the terms and acronyms used in the document that may be unfamiliar or ambiguous to the readers.

The references section lists the sources of information that were used to create the document, such as standards, specifications, books, articles, etc.

The Requirement Engineering Process in SRS is the process of eliciting, analyzing, specifying, validating, and managing the requirements of a software project. It is a crucial step in the software development life cycle, as it defines the scope, functionality, quality, and constraints of the software system. The Requirement Engineering Process in SRS can be divided into four main activities:

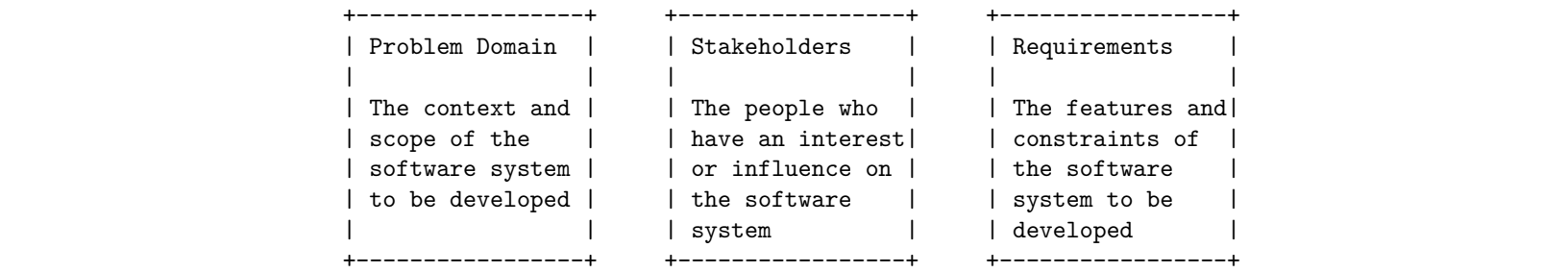
- **Requirement Elicitation**: This is the process of gathering the requirements from various sources, such as stakeholders, users, domain experts, existing systems, documents, etc. The goal of this activity is to identify the needs, expectations, and objectives of the software system.
- **Requirement Analysis**: This is the process of analyzing the requirements to ensure that they are clear, consistent, complete, feasible, verifiable, and prioritized. The goal of this activity is to resolve any conflicts, ambiguities, or gaps in the requirements, and to refine them into a more detailed and structured form.
- **Requirement Specification**: This is the process of documenting the requirements in a formal and standardized way, such as using a Software Requirements Specification (SRS) document. The goal of this activity is to provide a clear and unambiguous description of the software system, its features, functions, interfaces, constraints, and quality attributes.
- **Requirement Validation**: This is the process of verifying that the requirements meet the needs and expectations of the stakeholders and users, and that they conform to the standards and regulations. The goal of this activity is to ensure that the requirements are correct, complete, consistent, and acceptable.

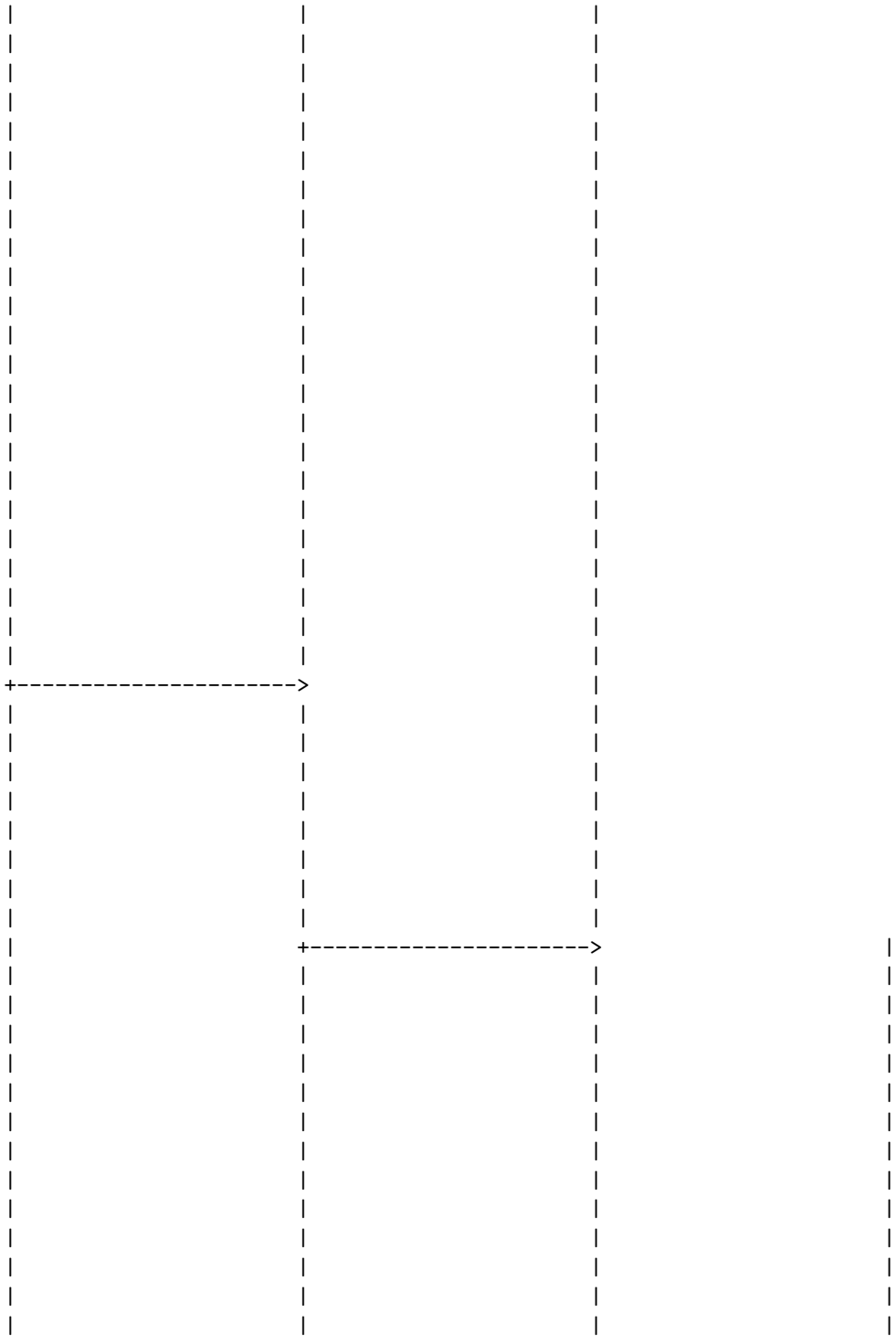
The following diagram illustrates the Requirement Engineering Process in SRS using ASCII art:

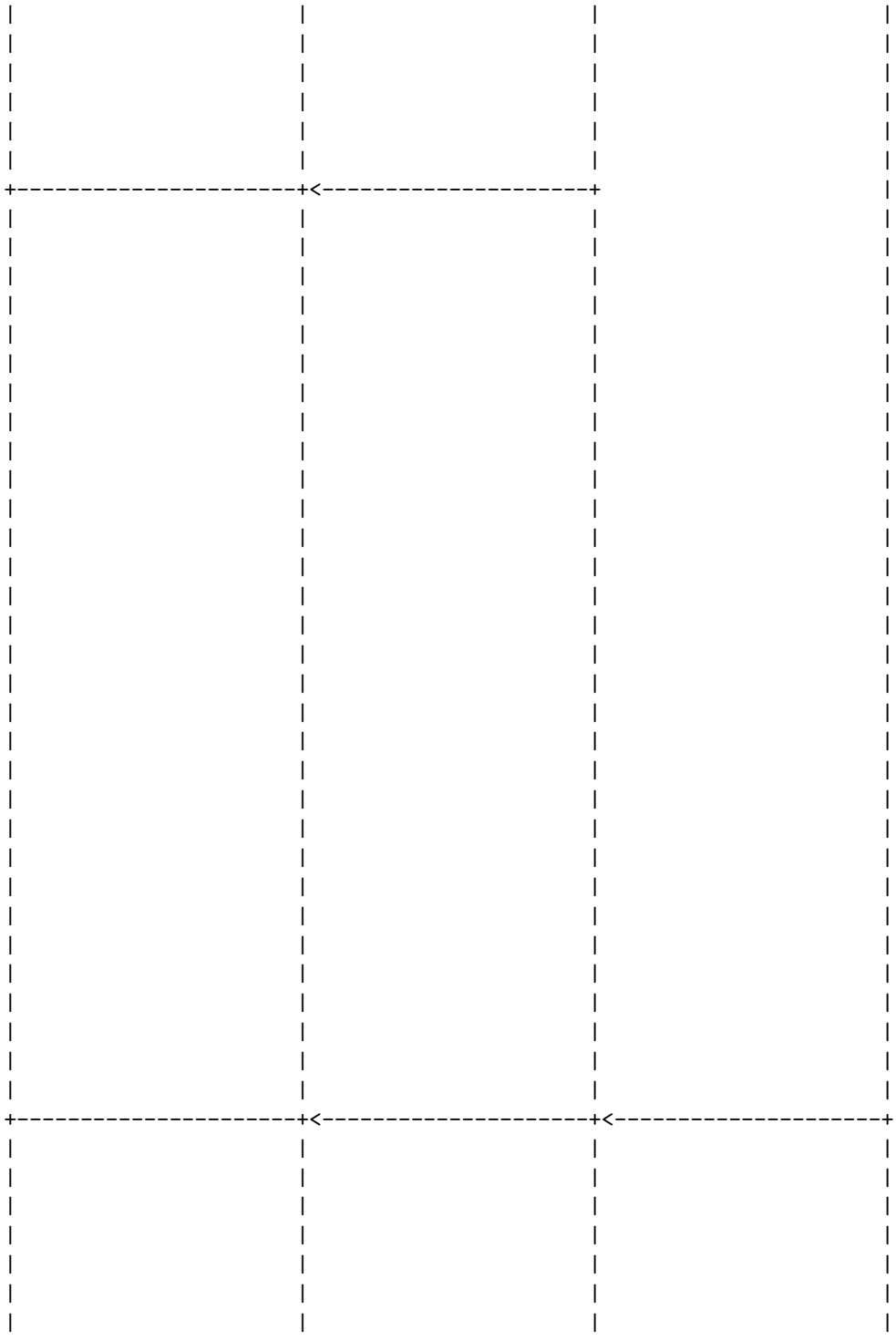


Elicitation in Requirement Engineering Process in SRS is the first step of the Requirement Engineering process. It helps the analyst to gain knowledge about the problem domain which in turn is used to produce a formal specification of the software. There are a number of techniques and challenges involved in this process .

The following diagram illustrates the basic steps of elicitation in Requirement Engineering Process in SRS using ASCII art:

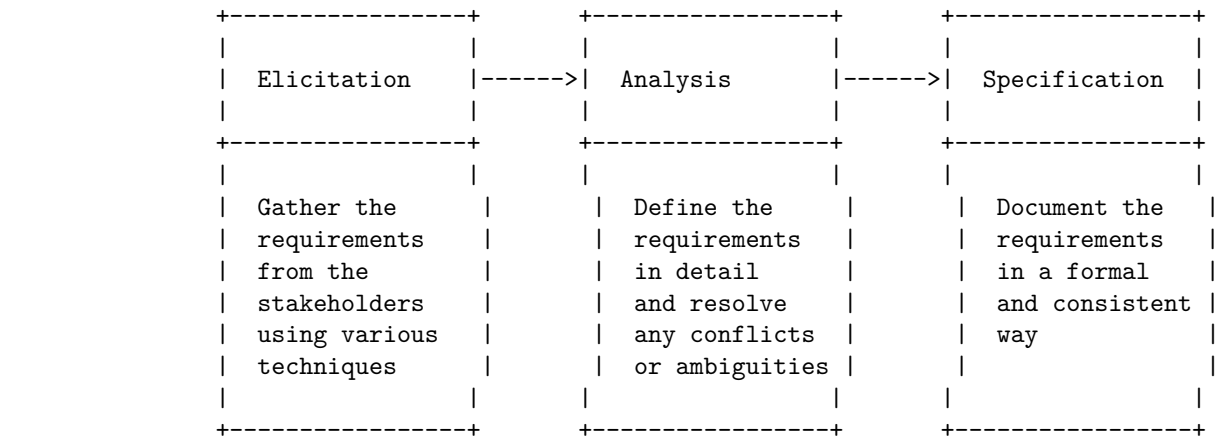






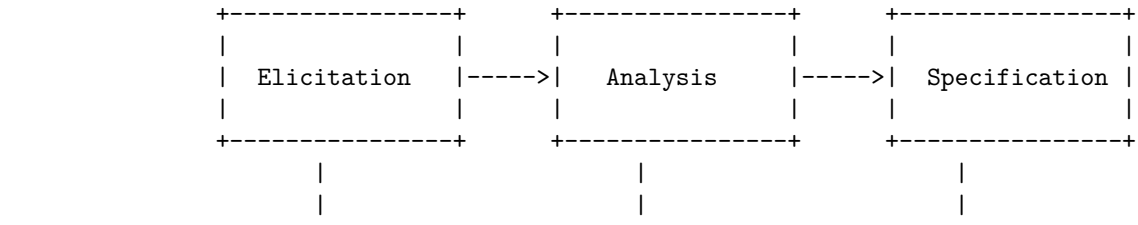
According to the search results, analysis in requirement engineering process in SRS is the process of defining user expectations for a new software being built or modified. It is performed by the requirement analyst or system analyst and the final outcome is a software requirements specification (SRS) document. The SRS fully describes what the software will do and how it will be expected to perform.

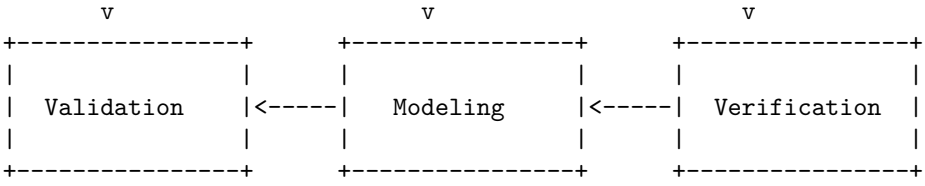
The following diagram illustrates the basic steps of analysis in requirement engineering process in SRS using ASCII art:



Documentation in Requirement Engineering Process in SRS is the activity of producing a document that describes the software requirements for a system or a software product. The document is also known as Software Requirements Specification (SRS) or Software Requirement Document (SRD). The SRS serves as a basis for software engineering activities, such as design, development, testing, and maintenance. It also serves as a contract between the development team and the customer, and as a product validation check.

The following diagram illustrates the basic steps of documentation in requirement engineering process in SRS using ASCII art:





Elicitation: The process of gathering the requirements from the stakeholders, such as customers, users, domain experts, etc.

Analysis: The process of refining, prioritizing, and organizing the requirements, and resolving any conflicts or inconsistencies.

Specification: The process of documenting the requirements in a formal or informal way, using natural language, diagrams, models, etc.

Modeling: The process of creating abstract representations of the requirements, such as use cases, data models, state diagrams, etc.

Verification: The process of checking the correctness, completeness, and consistency of the specification, and ensuring that it meets the stakeholder needs and expectations.

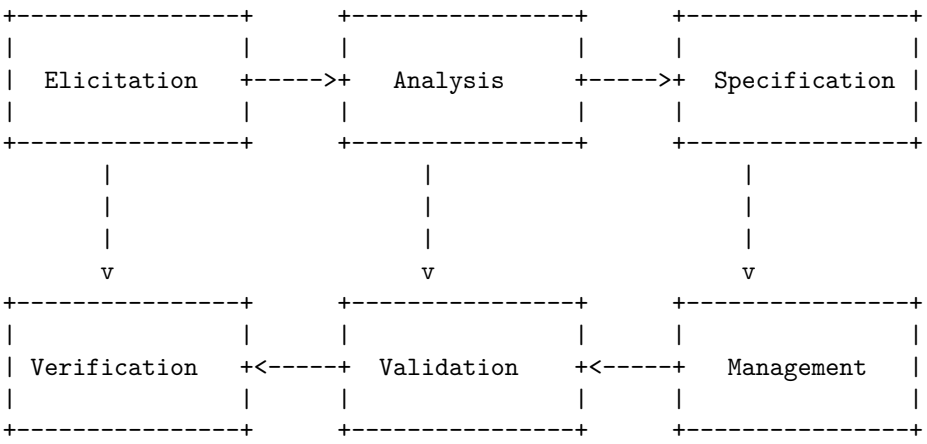
Validation: The process of confirming that the specification matches the actual requirements of the system or the product, and that it satisfies the quality criteria.

The Review and Management of User Needs in Requirement Engineering Process in SRS is a process that aims to ensure that the software requirements specification (SRS) document accurately reflects the needs and expectations of the stakeholders, and that it is consistent, complete, correct, and verifiable. The process involves the following steps:

- Elicitation: This is the process of gathering the user needs from various sources, such as interviews, surveys, observations, documents, etc. The elicitation techniques should be appropriate for the context and the type of stakeholders involved.
- Analysis: This is the process of refining, organizing, prioritizing, and validating the user needs. The analysis techniques should help to identify the functional and non-functional requirements, the constraints, the assumptions, and the risks associated with the software project.
- Specification: This is the process of documenting the user needs in a formal and structured way, using a standard notation and language. The specification techniques should ensure that the SRS document is clear, concise, unambiguous, and testable.
- Verification: This is the process of checking that the SRS document meets the quality criteria and conforms to the standards and guidelines. The verification techniques should include reviews, inspections, walkthroughs, and audits by different stakeholders, such as users, developers, testers, managers, etc.

- Validation: This is the process of confirming that the SRS document satisfies the user needs and expectations, and that it is feasible and realistic. The validation techniques should include feedback, prototyping, simulation, testing, and evaluation by the end-users and the clients.
- Management: This is the process of controlling and monitoring the changes and updates to the SRS document throughout the software development life cycle. The management techniques should include configuration management, change management, traceability management, and version control.

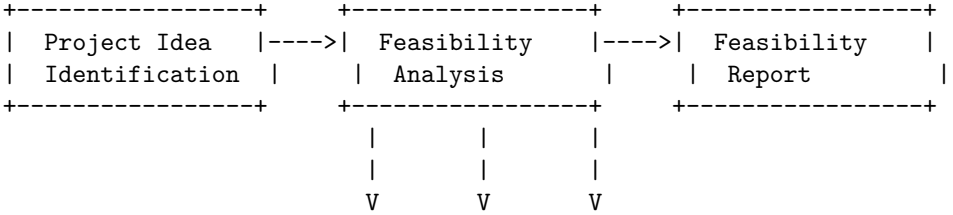
The following diagram illustrates the basic architecture of the Review and Management of User Needs in Requirement Engineering Process in SRS:

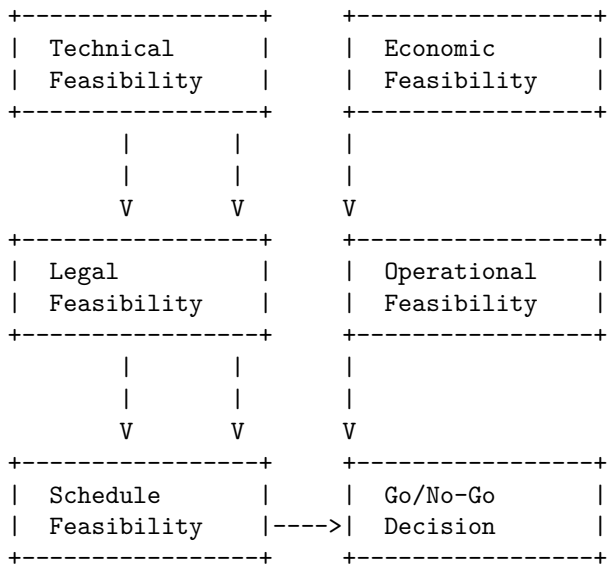


A feasibility study is a stage in the software project management process that evaluates the technical, economic, legal, operational, and schedule aspects of a proposed software product. It helps to determine whether the project is viable, feasible, and worth pursuing. A feasibility study can be conducted before or after the software requirements specification (SRS) document is written, depending on the purpose and scope of the project.

A feasibility study can be represented by a diagram that shows the main steps and outcomes of the process. One possible diagram is shown below, using ASCII characters to draw the shapes and arrows.

Feasibility Study in Software Requirement Specification (SRS)





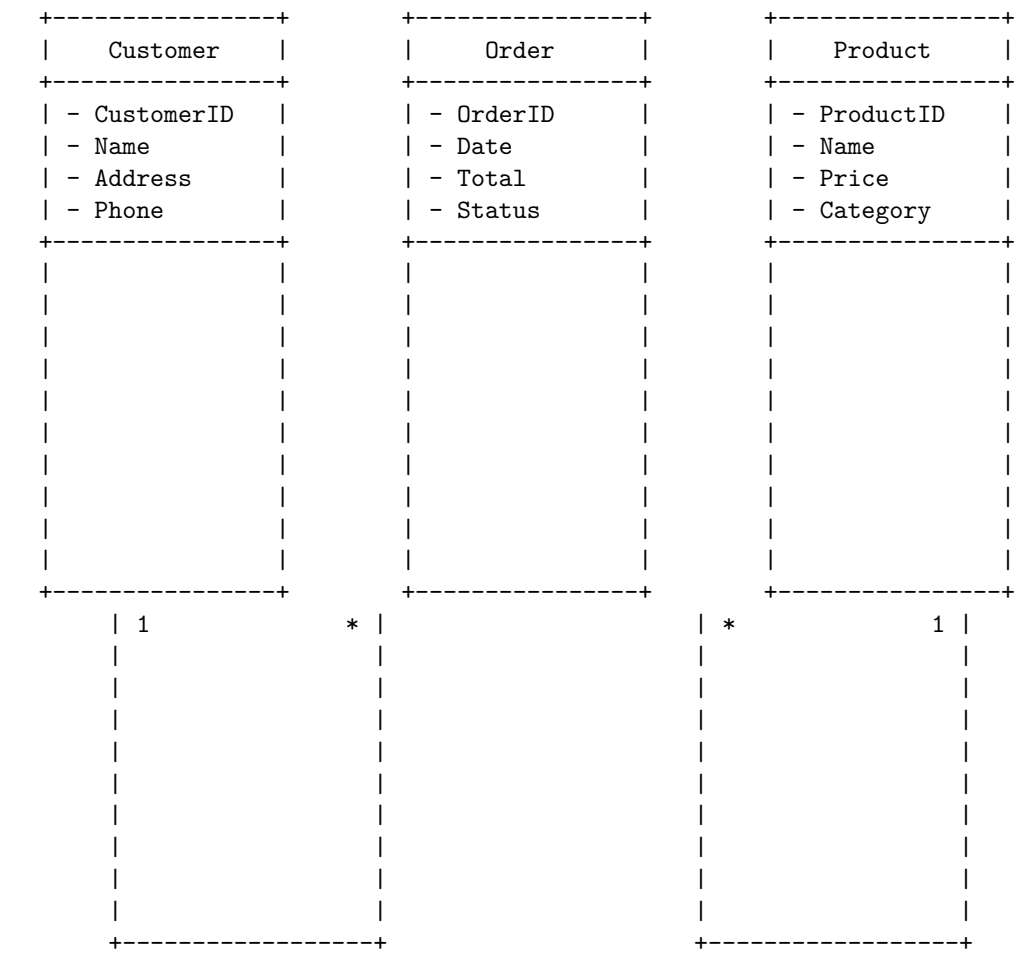
The diagram illustrates the basic steps of a feasibility study, starting from the project idea identification, to the feasibility analysis, to the feasibility report, and finally to the go/no-go decision. The feasibility analysis consists of four sub-steps: technical, economic, legal, and operational feasibility. Each sub-step evaluates a different aspect of the project and its potential risks and benefits. The schedule feasibility is also considered as a sub-step, as it assesses the time and resources needed to complete the project. The feasibility report summarizes the findings and recommendations of the feasibility analysis, and the go/no-go decision is the final outcome of the feasibility study, indicating whether the project should proceed or not.

Information Modelling in Software Requirement Specification (SRS) is the process of identifying and defining the data and information that are relevant and necessary for the software product to be developed. It involves creating a conceptual model of the data and information, as well as their relationships, constraints, and operations. Information Modelling helps to ensure that the software requirements are clear, complete, consistent, and verifiable.

One of the common techniques for Information Modelling is the Entity-Relationship (ER) model, which uses graphical symbols to represent the entities, attributes, and relationships in the information domain. An entity is a thing or object that has significance for the software product, such as a customer, a product, or an order. An attribute is a property or characteristic of an entity, such as a name, a price, or a quantity. A relationship is an association or link between two or more entities, such as a customer placing an order, or a product belonging to a category.

The following diagram illustrates the basic structure of an ER model using ASCII symbols:

Information Modelling in Software Requirement Specification (SRS)



The diagram shows that a customer can place zero or more orders, and each order can contain one or more products. Each product can belong to one category. The numbers on the lines indicate the cardinality of the relationships, which means the minimum and maximum number of occurrences of one entity for each occurrence of the related entity. For example, the 1 on the line between Customer and Order means that each order must have one and only one customer, while the * on the same line means that each customer can have zero or more orders. The attributes of each entity are listed below the entity name, preceded by a dash. The primary key of each entity, which is the attribute that uniquely identifies each instance of the entity, is underlined. For example, the primary key of Customer is CustomerID, which means that no two customers can have the same CustomerID.

Some additional information that can be added to the ER model are the data

types and constraints of the attributes, the names of the relationships, and the optional or mandatory participation of the entities in the relationships. For example, the data type of CustomerID could be integer, the name of the relationship between Customer and Order could be places, and the participation of Order in the places relationship could be optional, meaning that a customer can exist without placing any order. These details can be shown using different symbols or notations, depending on the convention or standard used for ER modelling.

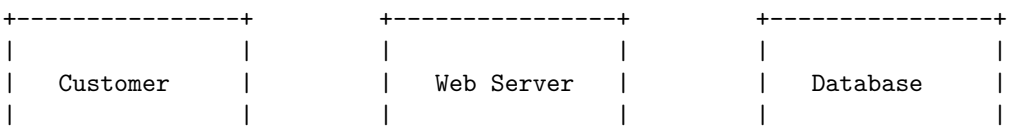
A data flow diagram (DFD) is a graphical representation of the flow of data through a system or process. It shows how data is input, processed, stored, and output in a system or process. A DFD can be used to document the current or desired state of a system or process, to identify problems or opportunities for improvement, or to communicate requirements and specifications.

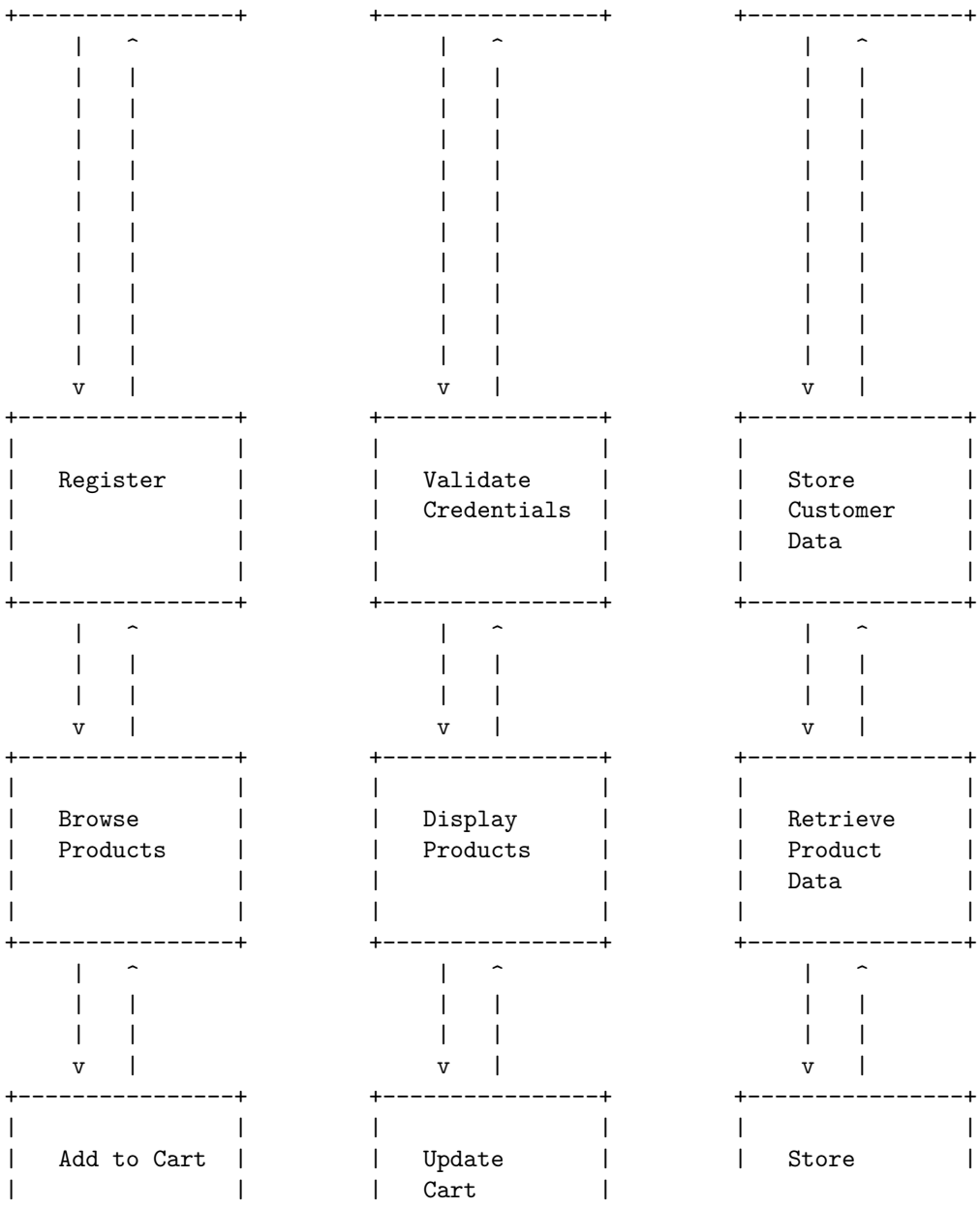
To draw a data flow diagram, you need to follow these steps:

1. Define the scope and boundary of the system or process you want to model. This can be done by drawing a context DFD, which shows the system or process as a single process with inputs and outputs from external entities (such as users, other systems, or data sources).
2. Decompose the system or process into smaller and more detailed processes. This can be done by drawing a level 1 DFD, which shows the main processes and data flows within the system or process. Each process in the level 1 DFD can be further decomposed into sub-processes in lower level DFDs, until the desired level of detail is reached.
3. Identify the data stores and data flows in the system or process. A data store is a place where data is stored, such as a database, a file, or a memory. A data flow is a movement of data from one process, data store, or external entity to another. Data flows are labeled with the name and description of the data being transferred.
4. Draw the diagram using standard symbols and notation. A DFD consists of four main elements: processes, data stores, data flows, and external entities. Processes are represented by circles or rounded rectangles, data stores are represented by open-ended rectangles, data flows are represented by arrows, and external entities are represented by squares or rectangles. Each element should have a unique name and number, and each data flow should have a descriptive label.

Data Flow Diagrams in Software Requirement Specification (SRS)

The following diagram illustrates the basic architecture of a web-based online shopping system:





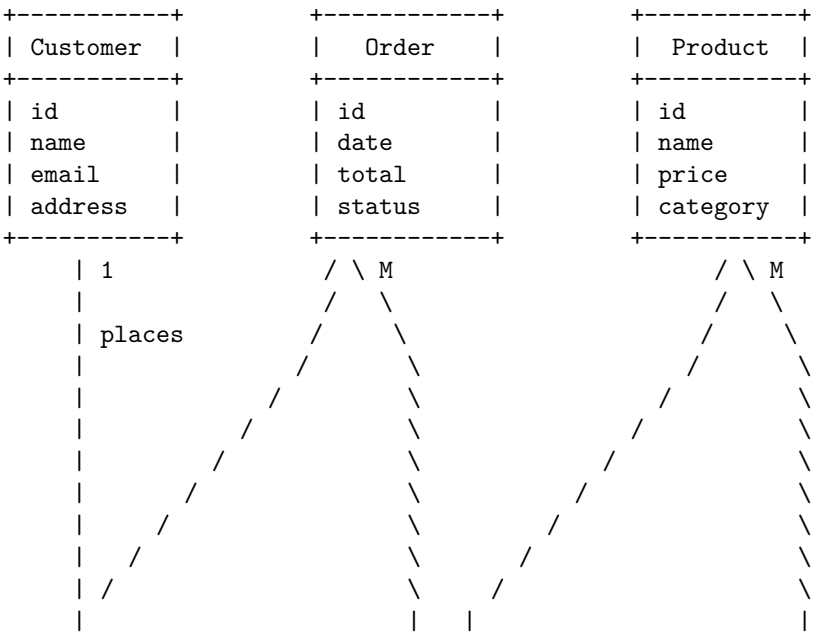
An entity relationship diagram (ERD) is a graphical representation of the entities and relationships in a system or database. It shows the types of entities, their attributes, and the cardinality and optionality of the relationships between them.

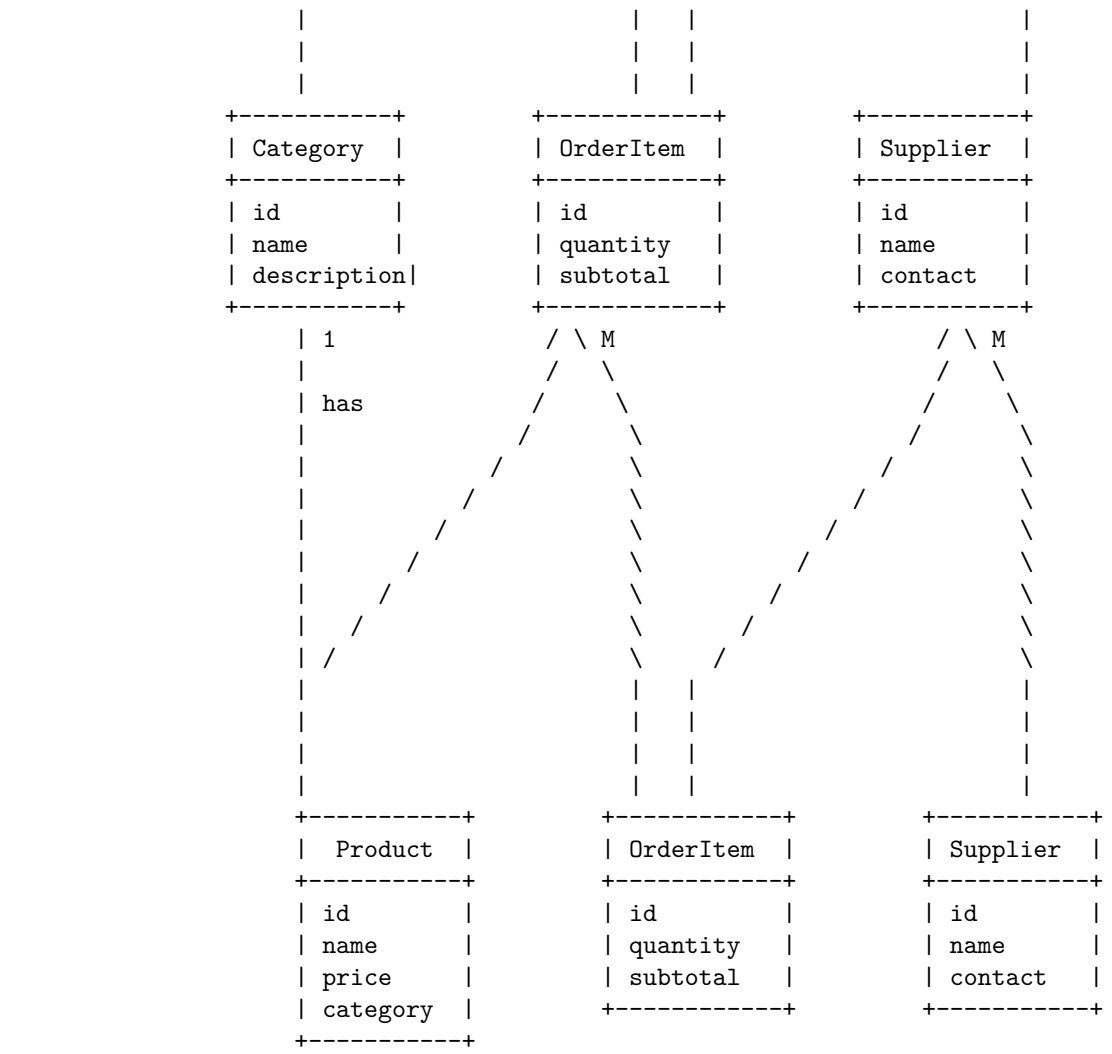
To draw an entity relationship diagram, you need to follow these steps:

1. Determine the entities in your system or database. Entities are the things or objects that you want to store information about, such as customers, products, orders, etc. They are usually represented by rectangles with the entity name inside.
2. Add attributes to each entity. Attributes are the properties or characteristics of each entity, such as name, address, price, quantity, etc. They are usually represented by ovals connected to the entity rectangle by a line.
3. Define the relationships between the entities. Relationships are the associations or interactions between the entities, such as customer places order, product belongs to category, order contains product, etc. They are usually represented by diamonds with the relationship name inside, connected to the entity rectangles by lines.
4. Specify the cardinality and optionality of each relationship. Cardinality is the number of instances of one entity that can be related to one instance of another entity, such as one-to-one, one-to-many, or many-to-many. Optionality is the degree of dependency or obligation of one entity to another entity, such as mandatory or optional. They are usually represented by symbols or words on the relationship lines, such as 1, M, N, (0,1), (1,1), etc.

Entity Relationship Diagrams in Software Requirement Specification (SRS)

The following diagram illustrates the basic architecture of a simple online shopping system:





A decision table is a tool that helps to specify the behavior of a software system based on different combinations of input conditions and actions. It is a tabular representation of logical rules that can be used to document the requirements of a software system. A decision table consists of four parts: condition stubs, action stubs, condition entries, and action entries. The condition stubs are the input conditions that affect the behavior of the system. The action stubs are the output actions that the system performs. The condition entries are the possible values of the input conditions, usually represented by Y (yes), N (no), or - (don't care). The action entries are the expected outcomes of the output actions, usually represented by X (execute) or - (don't execute).

The following diagram illustrates the basic structure of a decision table:

Condition Stub	Condition Entry	Condition Entry	Condition Entry
Condition 1	Y	N	-
Condition 2	Y	-	N
Condition 3	-	Y	N
Action Stub	Action Entry	Action Entry	Action Entry
Action 1	X	-	-
Action 2	-	X	-
Action 3	-	-	X

The diagram shows that the system performs different actions depending on the values of the input conditions. For example, if condition 1 and condition 2 are both true, then the system executes action 1. If condition 1 is false and condition 3 is true, then the system executes action 2. If condition 2 and condition 3 are both false, then the system executes action 3.

A decision table can be used to document the requirements of a software system in a Software Requirement Specification (SRS) document. An SRS is a document that defines what a given software system needs to do and takes care of various requirements. It is written according to the needs of the software and ensures that the software does not cause any problems to the end-users. The different features of the software are clearly detailed and given particular attention. A decision table can help to specify the functional requirements of the software, which describe the behavior and functionality of the system. A decision table can also help to avoid ambiguity and inconsistency in the requirements, as it shows all the possible scenarios and outcomes of the system. A decision table can also help to verify and validate the requirements, as it can be used to test the system against the expected behavior and actions. A decision table can also help to communicate the requirements to the developers, as it provides a clear and concise representation of the logic and rules of the system.

Decision Tables in Software Requirement Specification (SRS)

Condition Stub	Condition Entry	Condition Entry	Condition Entry
User is logged in	Y	N	-
User has access to file	Y	-	N

```
+----------------+----------------+----------------+----------------+
| File is encrypted| -              | Y              | N              |
+------------------+----------------+----------------+----------------+
| Action Stub      | Action Entry   | Action Entry   | Action Entry   |
+------------------+----------------+----------------+----------------+
| Open file        | X              | -              | -              |
+------------------+----------------+----------------+----------------+
| Request password| -               | X              | -              |
+------------------+----------------+----------------+----------------+
| Display error message| -              | -              | X              |
+----------------+----------------+----------------+----------------+
```

The diagram above shows an example of

An SRS document is a software requirements specification document that describes what the software will do and how it will be expected to perform. It also describes the functionality the product needs to fulfill the needs of all stakeholders (business, users). An SRS document can be thought of as a blueprint or roadmap for the software you're going to build.

SRS Document

An SRS document typically consists of the following sections:

- Introduction: This section provides an overview of the document, its purpose, scope, definitions, acronyms, abbreviations, references, and overview of the software product.
- Overall Description: This section provides a general description of the software product, its perspective, functions, user characteristics, constraints, assumptions, and dependencies.
- Specific Requirements: This section provides a detailed description of the functional and non-functional requirements of the software product, such as user interface, performance, security, reliability, etc. This section may also include use cases, data flow diagrams, state transition diagrams, or other graphical representations of the requirements.
- Appendices: This section provides any additional information that may be relevant to the SRS document, such as glossary, index, bibliography, etc.

An example of an SRS document in ASCII format is shown below:

```
+------------------------------------------------------------------------------+
| Software Requirements Specification (SRS) Document                           |
+------------------------------------------------------------------------------+
| Document ID: SRS-001                                                         |
| Version: 1.0                                                                 |
| Date: 13/03/2023                                                             |
| Author: Sydney                                                               |
```

+--+

1. Introduction

1.1 Purpose
This document specifies the software requirements for the Online Shopping System (OSS), a we

1.2 Scope
The OSS is intended to provide a convenient and user-friendly online shopping experience for

1.3 Definitions, Acronyms, and Abbreviations
- OSS: Online Shopping System
- SRS: Software Requirements Specification
- UI: User Interface
- DB: Database
- API: Application Programming Interface

1.4 References
- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications
- Online Shopping System Use Case Diagram
- Online Shopping System Data Flow Diagram
- Online Shopping System State Transition Diagram

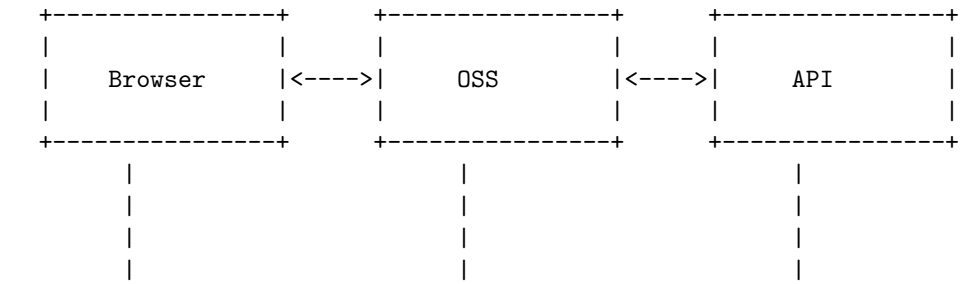
1.5 Overview
The rest of the document is organized as follows:

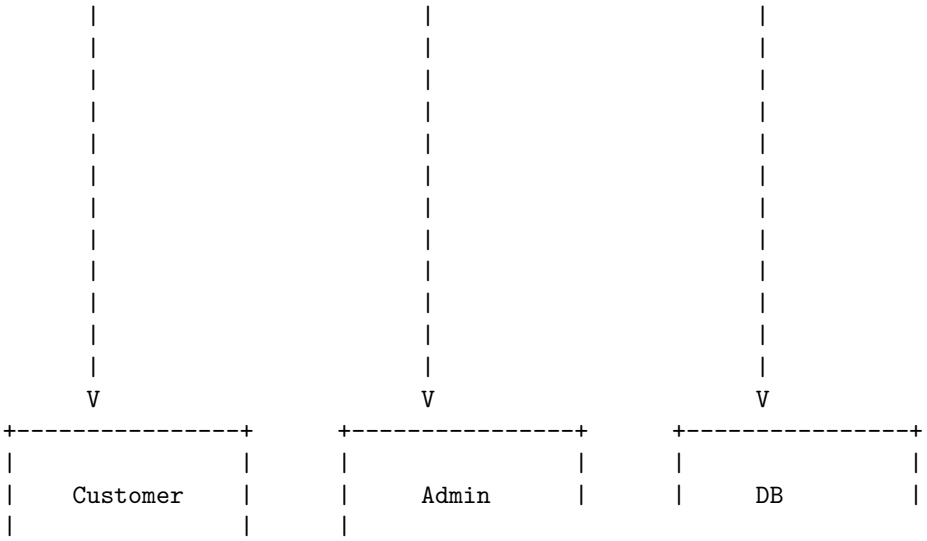
- Section 2 provides an overall description of the OSS, its perspective, functions, user cha
- Section 3 provides a detailed description of the functional and non-functional requirement
- Section 4 provides any additional information that may be relevant to the SRS document, su

2. Overall Description

2.1 Product Perspective
The OSS is a standalone web-based application that interacts with a DB and an API. The OSS i

The following diagram illustrates the basic architecture of the OSS:



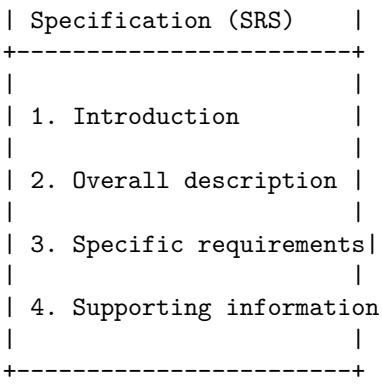


According to the IEEE standard 29148, a software requirements specification (SRS) document should contain the following sections:

1. Introduction
   - Purpose
   - Scope
   - Definitions, acronyms, and abbreviations
   - References
   - Overview
2. Overall description
   - Product perspective
   - Product functions
   - User characteristics
   - Constraints
   - Assumptions and dependencies
3. Specific requirements
   - External interface requirements
   - Functional requirements
   - Performance requirements
   - Design constraints
   - Software system attributes
   - Other requirements
4. Supporting information
   - Table of contents and index
   - Appendices

The following diagram illustrates the basic structure of a SRS document:

```
+-----------------------+
| Software Requirements  |
```



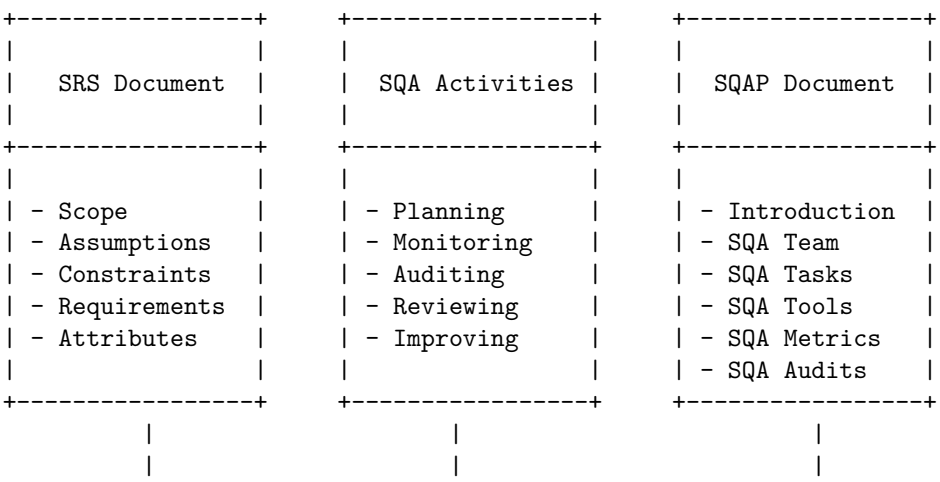
Software Quality Assurance (SQA) in SRS

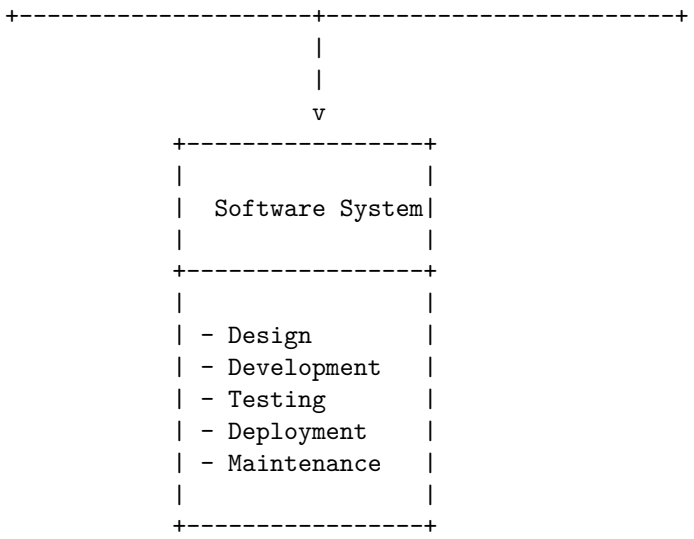
Software Quality Assurance (SQA) is a process that assures that all software engineering processes, methods, activities, and work items are monitored and comply with the defined standards. These defined standards could be one or a combination of any like ISO 9000, CMMI model, ISO15504, etc.

Software Requirement Specification (SRS) is a document that describes the functional and non-functional requirements of a software system. It also defines the scope, assumptions, constraints, and quality attributes of the system.

A Software Quality Assurance Plan (SQAP) is a document that defines the procedures, techniques, and tools that are employed to make sure that a product or service aligns with the requirements defined in the SRS. It also describes the roles and responsibilities, quality metrics, quality audits, quality reviews, and quality improvement actions of the SQA team.

The following diagram illustrates the basic architecture of a Software Quality Assurance Plan in relation to the Software Requirement Specification:

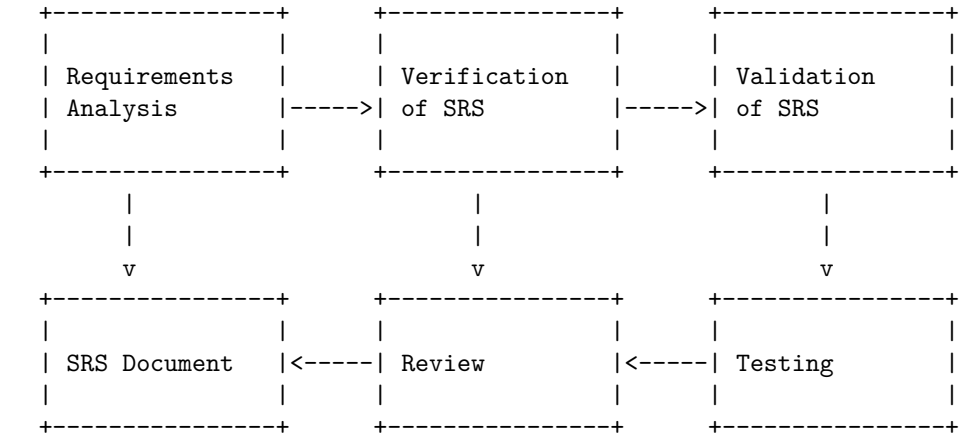




Verification and validation are two important activities in the software development process. Verification is the process of checking whether the software meets the specified requirements. Validation is the process of checking whether the software meets the user's needs and expectations. Both verification and validation are performed throughout the software development life cycle, from the initial requirements analysis to the final testing and deployment.

Verification and Validation in SRS

The following diagram illustrates the basic steps of verification and validation in the software requirement specification (SRS) document.



The SRS document is the output of the requirements analysis phase, where the user's needs and expectations are elicited, analyzed, and documented. The

SRS document should be clear, complete, consistent, verifiable, realistic, and unambiguous.

The verification of the SRS document is the process of checking whether the SRS document meets the specified requirements. This can be done by using various methods, such as peer reviews, walkthroughs, inspections, etc. The verification of the SRS document should ensure that the SRS document is correct, consistent, and complete.

The validation of the SRS document is the process of checking whether the SRS document meets the user's needs and expectations. This can be done by using various methods, such as testing, prototyping, simulation, etc. The validation of the SRS document should ensure that the SRS document is relevant, feasible, and usable.

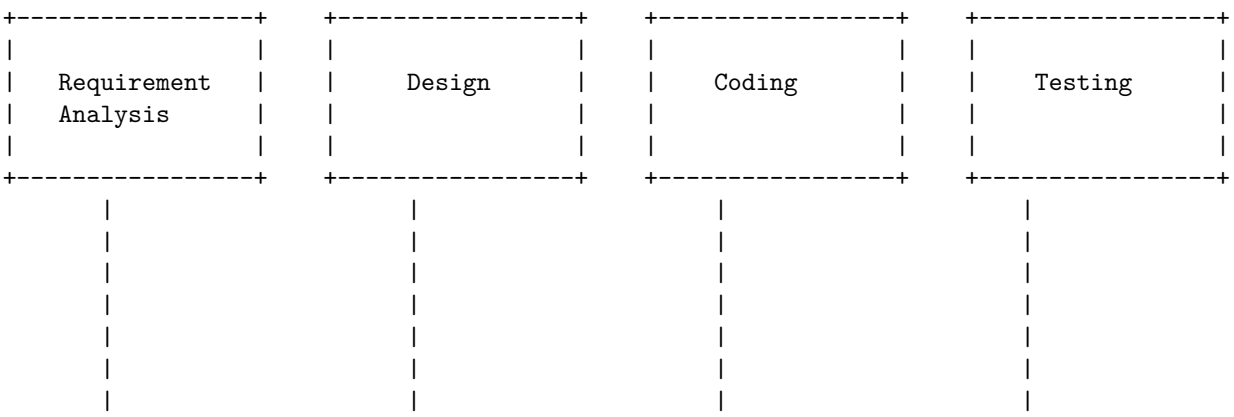
SQA Plans in SRS

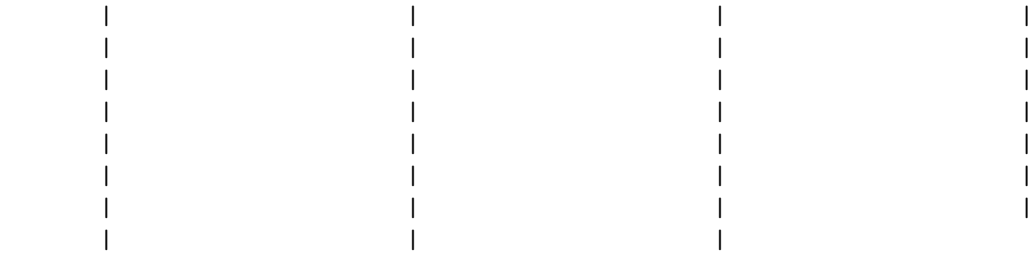
A software quality assurance plan (SQAP) is a document that describes the procedures, techniques, and tools that are used to ensure that a software product or service meets the requirements defined in the software requirement specification (SRS). A SQAP typically includes the following sections:

- Purpose: This section states the objectives and scope of the SQAP, and identifies the software project and the organization responsible for its development and quality assurance.
- Reference documents: This section lists the relevant standards, guidelines, policies, and regulations that are applicable to the software project and the SQAP.
- Management: This section describes the organizational structure, roles, and responsibilities of the software project team and the quality assurance team, and defines the communication and reporting mechanisms among them.
- Documentation: This section specifies the types, formats, contents, and quality criteria of the software project documentation, such as the SRS, design documents, test plans, user manuals, etc.
- Standards, practices, conventions, and metrics: This section defines the standards, practices, conventions, and metrics that are followed by the software project team and the quality assurance team to ensure the consistency, completeness, correctness, and maintainability of the software product or service.
- Reviews and audits: This section describes the methods, procedures, and criteria for conducting reviews and audits of the software project activities and deliverables, such as the SRS, design documents, code, test cases, test results, etc.
- Testing: This section describes the testing strategy, methodology, tools, and environment for verifying and validating the software product or service, and defines the test levels, types, cases, scenarios, and procedures.

- Problem reporting and corrective action: This section describes the process and tools for identifying, reporting, tracking, resolving, and preventing software defects and non-conformities, and defines the roles and responsibilities of the software project team and the quality assurance team in this process.
- Tools, techniques, and methodologies: This section describes the tools, techniques, and methodologies that are used or recommended by the quality assurance team to support the software project activities and deliverables, such as the SRS, design documents, code, test cases, test results, etc.
- Code control: This section describes the process and tools for managing the software source code, such as the version control, configuration management, change control, and release management.
- Media control: This section describes the process and tools for handling and storing the software media, such as the disks, tapes, CDs, DVDs, etc., that contain the software product or service or its documentation.
- Supplier control: This section describes the process and criteria for selecting, evaluating, and monitoring the software suppliers, subcontractors, or vendors, if any, that provide software components, services, or tools to the software project.
- Records collection, maintenance, and retention: This section describes the process and tools for collecting, maintaining, and retaining the software quality records, such as the SQAP, review reports, audit reports, test reports, problem reports, etc.
- Training: This section describes the training needs, plans, and programs for the software project team and the quality assurance team, and defines the training objectives, contents, methods, and evaluation.
- Risk management: This section describes the process and tools for identifying, analyzing, prioritizing, mitigating, and monitoring the software project risks, and defines the risk categories, factors, sources, and impacts.

The following diagram illustrates the basic architecture of a SQAP in relation to the software project life cycle:



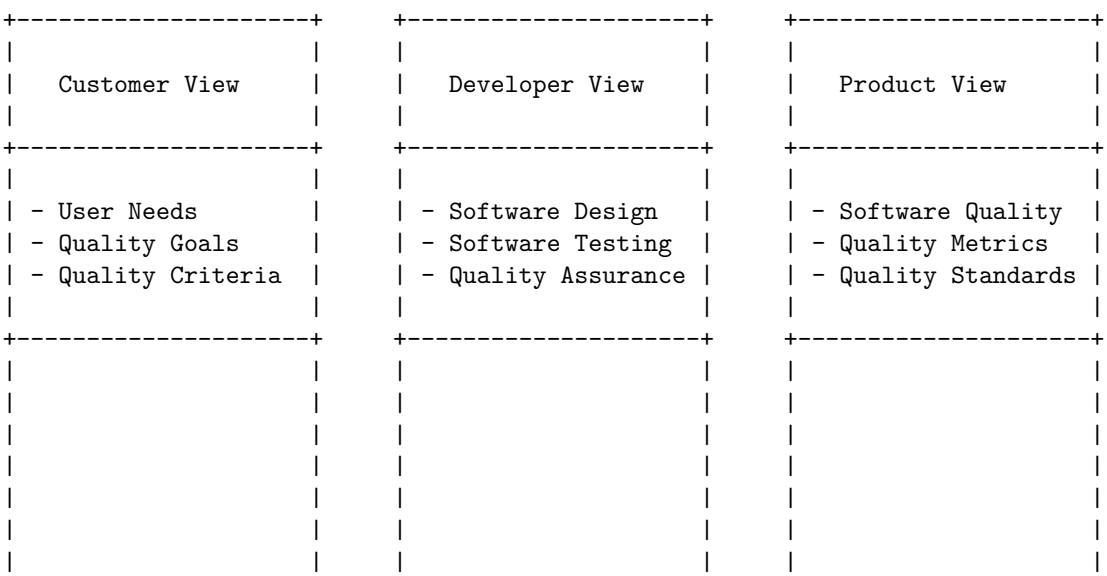


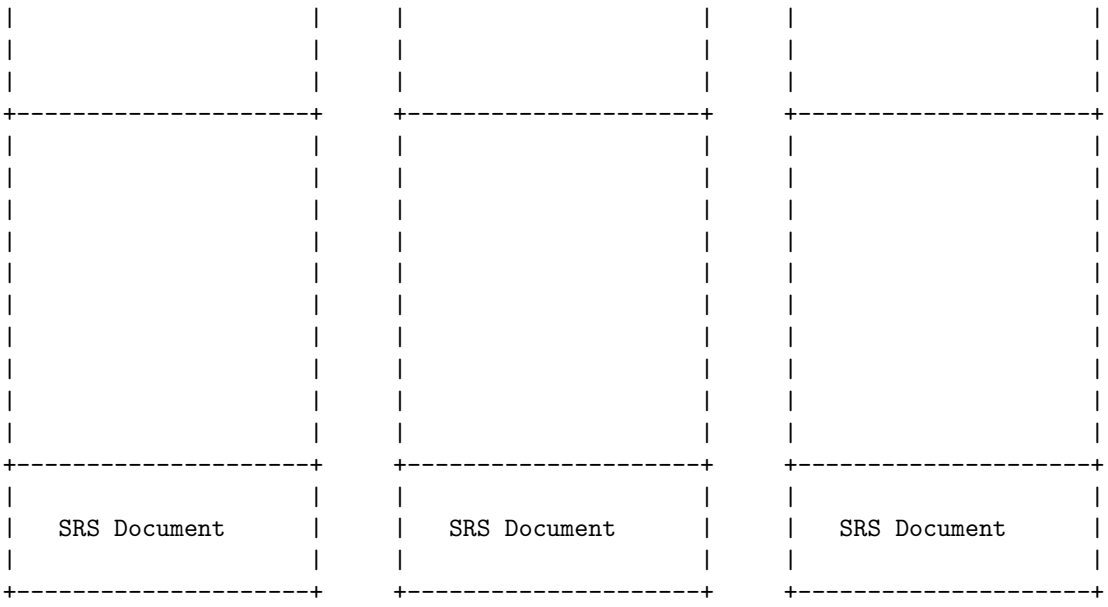
A Software Quality Framework (SQF) is a model for software quality by connecting and integrating the different views of software quality. It connects the customer view with the developer view of software quality and it treats software as a product. An SQF can be used to define, measure, and improve the quality of software products and processes.

A Software Requirements Specification (SRS) is a document that describes the features, functions, and constraints of a software system. It provides a clear and complete description of what the software should do and how it should behave. It also serves as a contract between the stakeholders and the developers of the software system.

An SQF can be incorporated in an SRS by defining the quality attributes and criteria that the software system should meet. These can include functional and non-functional requirements, such as reliability, security, performance, maintainability, usability, etc. An SQF can also provide a framework for testing and verifying the software quality throughout the development process.

The following diagram illustrates the basic architecture of an SQF in an SRS using ASCII art:





ISO 9000 Models in SRS

ISO 9000 is a series of standards that provide guidelines and principles for quality management systems (QMS) in various domains, including software engineering. ISO 9000 models in SRS refer to the application of ISO 9000 standards to the software requirements specification (SRS) document, which defines the functional and non-functional requirements of a software system or product.

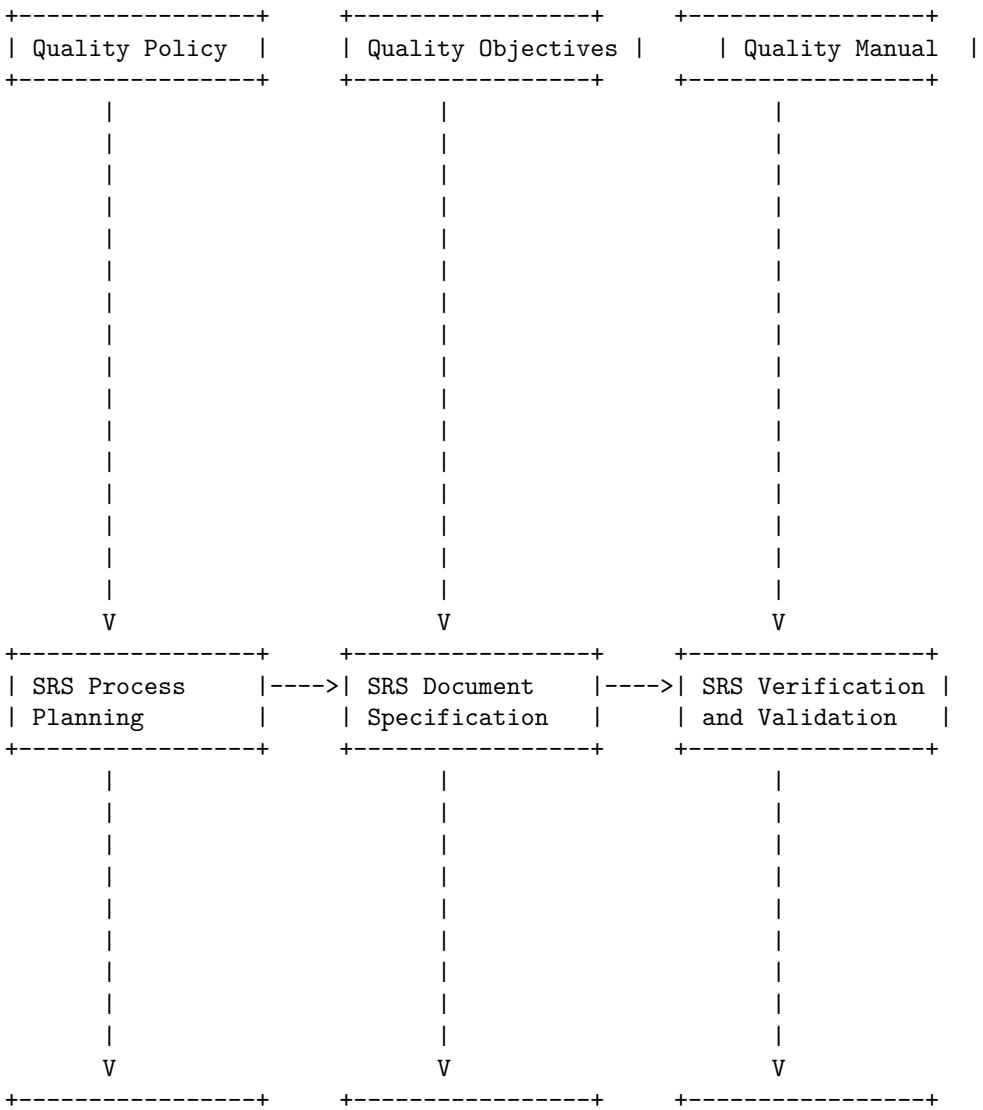
One of the ISO 9000 standards that is relevant to SRS is ISO 9001, which specifies the requirements for a QMS that can be used to demonstrate the ability to consistently provide products and services that meet customer and regulatory requirements. ISO 9001 can be applied to the SRS process by ensuring that the SRS document is:

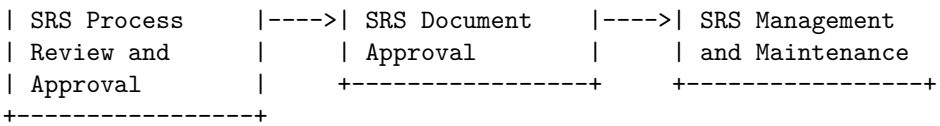
- Planned and controlled according to the quality objectives and policies of the organization
- Reviewed and approved by the relevant stakeholders
- Verified and validated to ensure that the requirements are clear, complete, consistent, and testable
- Managed and maintained throughout the software development life cycle
- Traced and linked to the design, implementation, and testing activities and artifacts

Another ISO 9000 standard that is relevant to SRS is ISO 9000-3, which provides guidelines for the application of ISO 9001 to the development, supply, installation, and maintenance of computer software. ISO 9000-3 can be applied to the SRS process by providing guidance on:

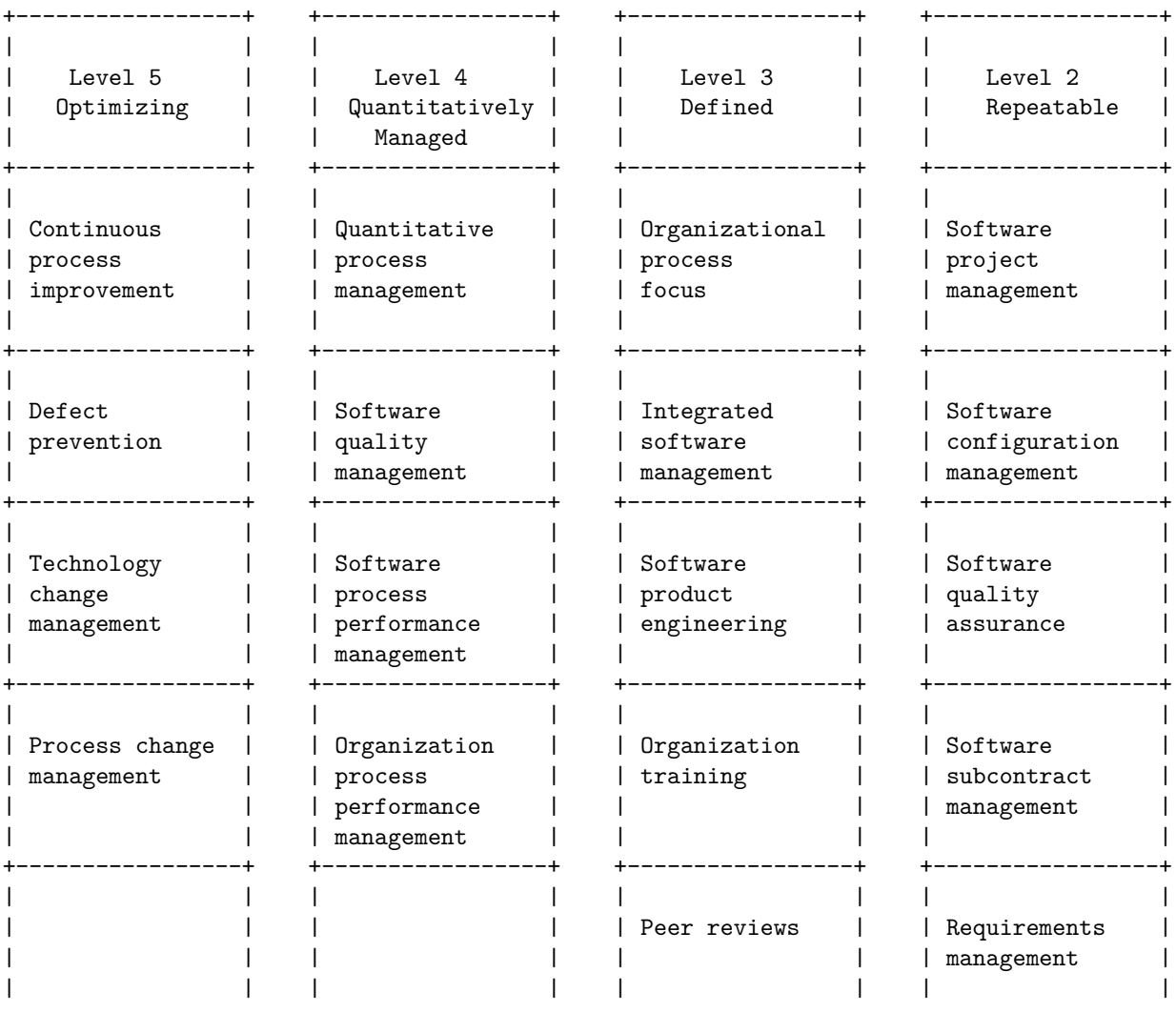
- The definition and documentation of the software requirements
- The use of appropriate methods and tools for eliciting, analyzing, specifying, and validating the software requirements
- The allocation and traceability of the software requirements to the software components and modules
- The management of changes and configuration of the software requirements
- The evaluation and improvement of the software requirements process

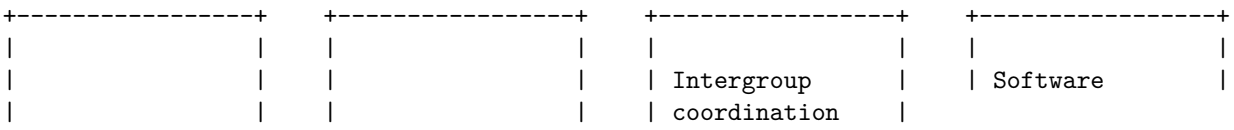
The following diagram illustrates the basic architecture of a QMS for SRS based on ISO 9000 standards:





The SEI-CMM Model in SRS is a framework that describes the essential elements of an organization's software engineering process that must exist to ensure good software products. It is based on the Capability Maturity Model (CMM) developed by the Software Engineering Institute (SEI) at Carnegie Mellon University. The model defines five levels of process maturity, from initial to optimizing, and identifies the key process areas and goals for each level. The following diagram illustrates the basic architecture of the SEI-CMM Model in SRS:





Unit 3 - Software Design

One possible way to draw a detailed ASCII diagram for software design is to use a tool called Graphviz, which can generate graphical representations of abstract graphs and networks. Graphviz can output various formats, including ASCII, using the dot language. For example, the following dot code:

```
digraph G {
  node [shape=box];
  A [label="User Interface"];
  B [label="Business Logic"];
  C [label="Data Access"];
  D [label="Database"];
  A -> B [label="Input"];
  B -> A [label="Output"];
  B -> C [label="Query"];
  C -> B [label="Result"];
  C -> D [label="Read/Write"];
}
```

Can be converted to ASCII using the command:

```
cat dotfile.dot | graph-easy --from=dot --as_ascii
```

Which produces the following output:

```
+-----------------+     Input    +-----------------+     Query    +-----------------+     Re
| User Interface  | -----------> | Business Logic  | -----------> | Data Access     | -----
+-----------------+ <----------- +-----------------+ <----------- +-----------------+ <-----
      Output            Result
```

This is a simple example of a software design diagram for a three-tier architecture, where the user interface, the business logic, and the data access are separated into different layers. The arrows indicate the flow of data and control between the components. The labels on the arrows describe the type of interaction.

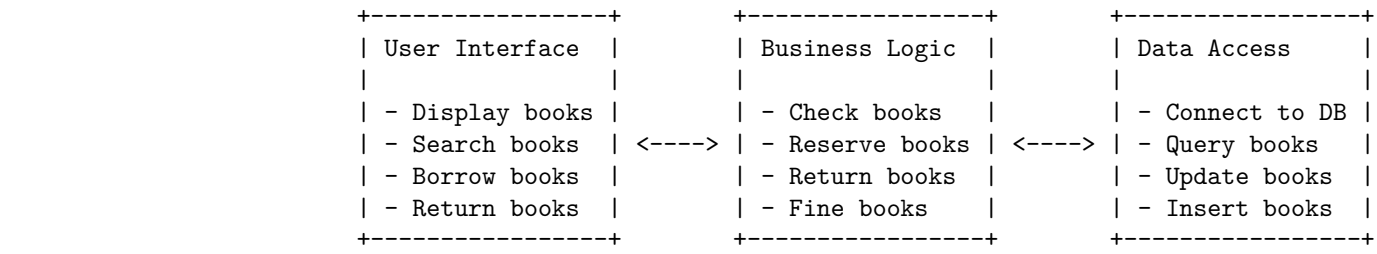
There are other tools and methods to draw ASCII diagrams for software design, such as Asciidoctor, Monodraw, Asciiflow, or even plain text editors. The choice of tool depends on the complexity and style of the diagram, as well as the personal preference of the designer. The main advantage of ASCII diagrams is that they can be easily embedded in code comments, documentation, or communication channels, without requiring any special software or format. The

main disadvantage is that they may not be as expressive or clear as graphical diagrams, especially for large or complex systems.

The basic concept of software design is to transform user requirements into a suitable form that helps the programmer in software coding and implementation. Software design is the process of envisioning and defining software solutions to one or more sets of problems, using a set of primitive components and subject to constraints. Software design also involves software requirements analysis, which is the process of listing specifications used in software engineering.

Basic Concept of Software Design

The following diagram illustrates the basic concept of software design using an example of a software system that manages a library:



The diagram shows the three main components of software design: user interface, business logic, and data access. The user interface is the part of the software that interacts with the user and provides input and output. The business logic is the part of the software that implements the rules and functionality of the system. The data access is the part of the software that communicates with the data source, such as a database or a file. The arrows show the direction of data flow and interaction between the components.

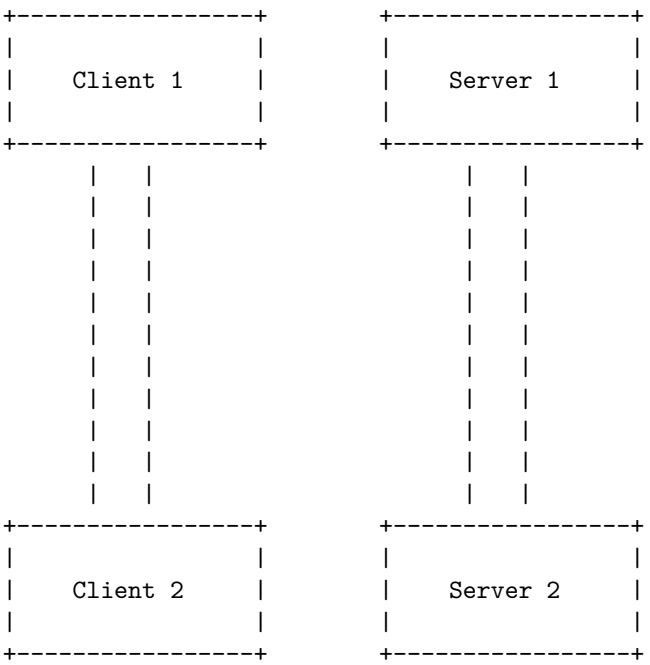
Architectural design in software engineering is the process of defining a collection of hardware and software components and their interfaces to establish the framework for the development of a computer system. It is expressed as a block diagram defining an overview of the system structure, features of the components, and how these components communicate with each other to share data.

There are many different types of architectural design patterns that can be used to represent the software architecture, such as layered, client-server, microservices, event-driven, etc. Each pattern has its own advantages and disadvantages, and the choice of the best pattern depends on the requirements, goals, and constraints of the software project.

The following diagram illustrates the basic architecture of a client-server pattern, which is one of the most common and simple patterns. In this pattern, the software system is divided into two components: a client and a server. The client is the component that requests services from the server, and the server is

the component that provides services to the client. The client and the server communicate with each other using a network protocol, such as HTTP, TCP, etc. The client and the server can be deployed on different machines, and there can be multiple clients and servers in the system.

Architectural Design in Software Design



Low-level design (LLD) is a component-level design process that follows a step-by-step refinement process. This process can be used for designing data structures, required software architecture, source code and ultimately, performance algorithms.

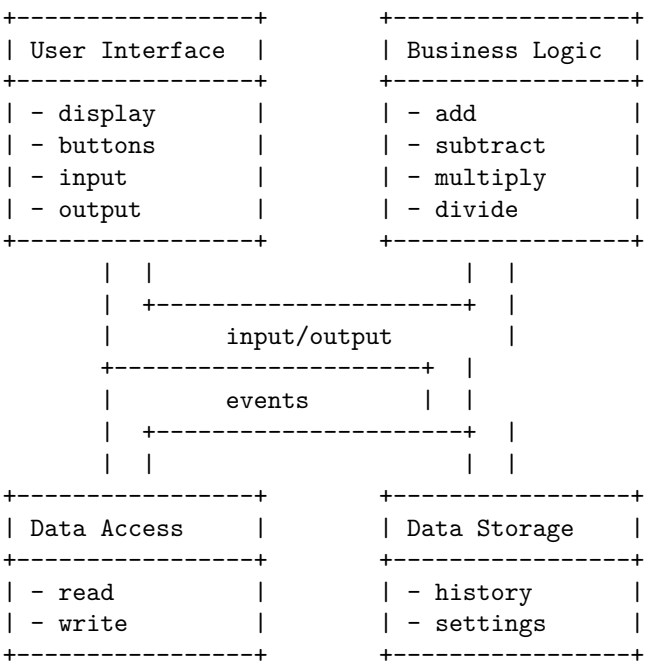
A low-level design document (LLD) typically contains the following sections:

- Introduction: This section provides an overview of the system or component, its purpose, scope, objectives, and assumptions.
- Architecture: This section describes the overall architecture of the system or component, its components, interfaces, dependencies, and interactions.
- Modules: This section describes each module of the system or component in detail, including its name, description, inputs, outputs, functionality, algorithms, data structures, and pseudocode.
- Test cases: This section describes the test cases that will be used to verify the functionality and performance of the system or component, including the test inputs, expected outputs, and test steps.

A low-level design diagram is a graphical representation of the system or com-

ponent, showing its components, interfaces, dependencies, and interactions. It can be drawn using various tools or notations, such as UML, ERD, or ASCII art.

The following is an example of a low-level design diagram for a simple calculator application, drawn using ASCII art:

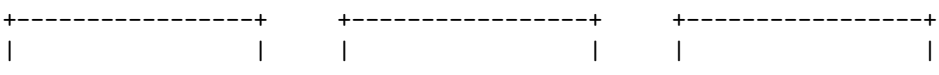


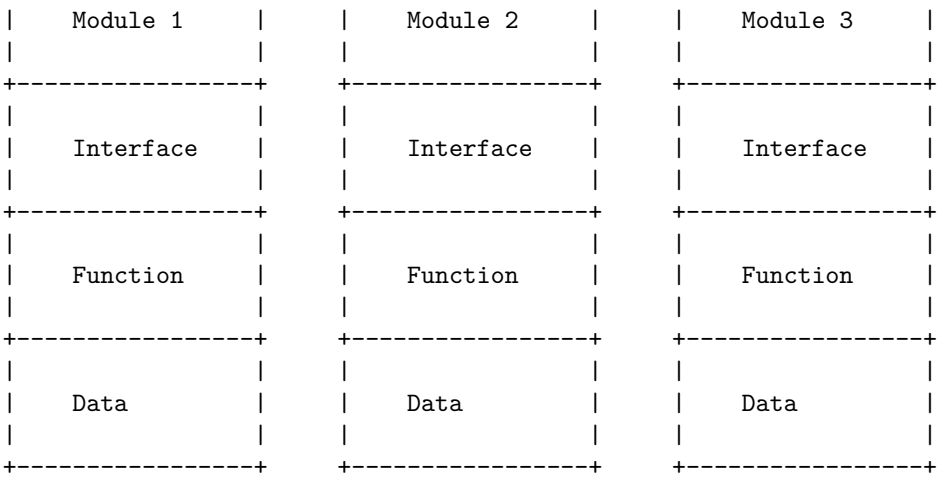
: Low-level design - Wikipedia

Modularization in Software Design is a technique to divide a software system into multiple discrete and independent modules, which are expected to be capable of carrying out task (s) independently. These modules may work as basic constructs for the entire software. Modularization improves the efficiency, reliability, and maintainability of software projects by organizing code into modules. A module is defined as the unique and addressable component of the software which can be solved and modified independently without disturbing (or affecting in very small amount) other modules of the software. Thus every software design should follow modularity.

Modularization in Software Design

The following diagram illustrates the basic architecture of a modularized software system:





Each module has three parts: interface, function, and data. The interface defines how the module communicates with other modules. The function defines what the module does. The data defines what the module stores. The modules are independent and interchangeable, meaning that they can be replaced or modified without affecting the rest of the system. The modules are also cohesive, meaning that they have a single responsibility and a clear purpose. The modules are loosely coupled, meaning that they have minimal dependencies on other modules. The modules are reusable, meaning that they can be used in different contexts and scenarios. The modules are testable, meaning that they can be verified and validated individually. The modules are maintainable, meaning that they can be easily updated and fixed. The modules are scalable, meaning that they can handle increasing demands and loads.

A design structure chart is a diagram that shows the hierarchical decomposition of a software system into its modules and the data flow between them. It is a tool for structured design that helps to identify the modules, their interfaces, and the information that passes between them.

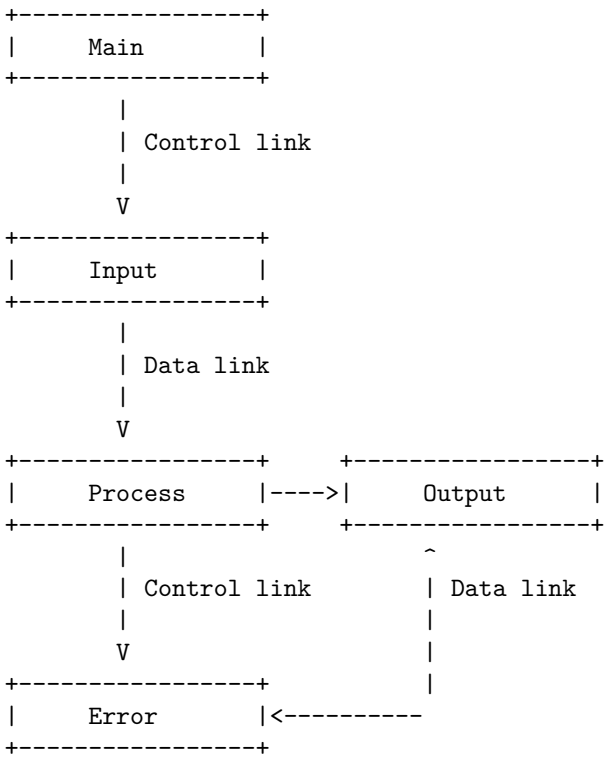
The basic elements of a design structure chart are:

- A module, represented by a rectangle with the module name inside.
- A control link, represented by a solid line with an arrowhead, that shows the calling relationship between modules.
- A data link, represented by a dashed line with an arrowhead, that shows the data flow between modules.
- A data couple, represented by a small circle on a data link, that shows the data item or structure that is passed between modules.
- A flag, represented by a diamond on a control link, that shows a condition or a parameter that affects the control flow between modules.
- A loop, represented by a curved line with an arrowhead, that shows a repeated execution of a module.

- A fan-out, represented by a fork on a control link, that shows a module calling multiple modules.
- A fan-in, represented by a join on a control link, that shows multiple modules calling a module.

Design Structure Charts in Software Design

The following diagram illustrates the basic architecture of a design structure chart:



The diagram shows that the Main module calls the Input module, which reads the data from the user or a file. The Input module passes the data to the Process module, which performs some calculations or transformations on the data. The Process module passes the results to the Output module, which displays or writes the results to the user or a file. The Process module also calls the Error module, which handles any errors or exceptions that may occur during the processing. The Error module passes the error message to the Output module, which displays or writes the error message to the user or a file.

Pseudo codes are a way of describing the logic and steps of an algorithm without using the syntax of a specific programming language. They are useful for designing and planning a solution to a problem, as well as for communicating

the approach to others. Pseudo codes are independent of any programming language, which makes them easier to translate into various languages.

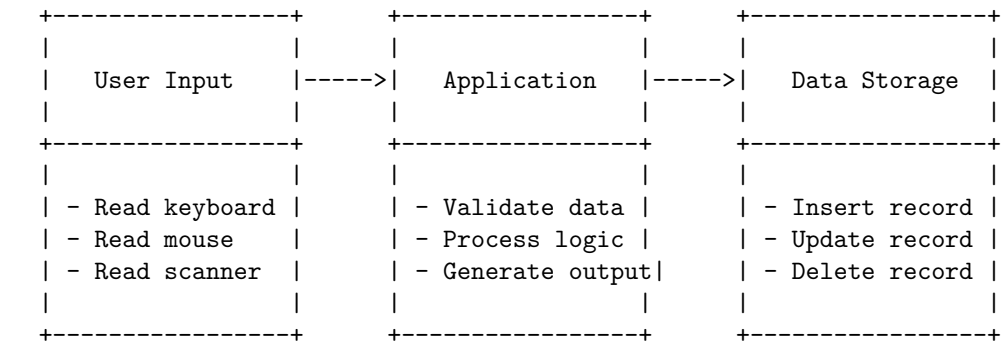
There is no standard format or rules for writing pseudo codes, but they usually follow some common conventions, such as:

- Using indentation to show the structure and hierarchy of the code.
- Using keywords such as IF, THEN, ELSE, FOR, WHILE, REPEAT, UNTIL, etc. to indicate the control flow of the code.
- Using comments to explain the purpose and meaning of the code.
- Using variables, constants, operators, and expressions to represent the data and operations of the code.
- Using input and output statements to interact with the user or other systems.

A pseudo code can be represented as a text or a diagram. A diagram can help to visualize the flow of the code and the relationships between the different parts. A common type of diagram used for pseudo codes is a flowchart, which uses symbols and arrows to show the sequence and branching of the code.

Pseudo Codes in Software Design

The following diagram illustrates the basic architecture of a software system using pseudo code:

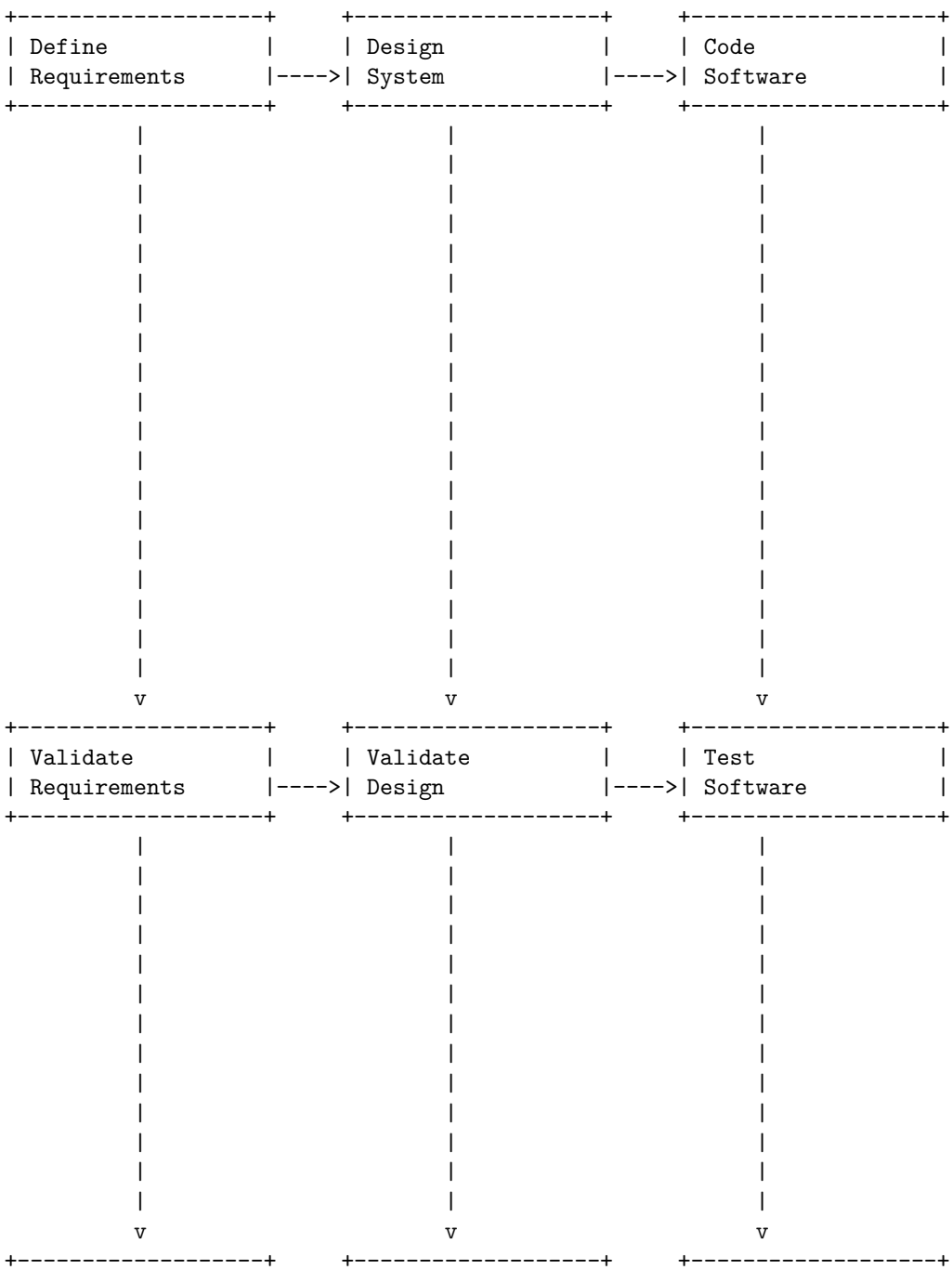


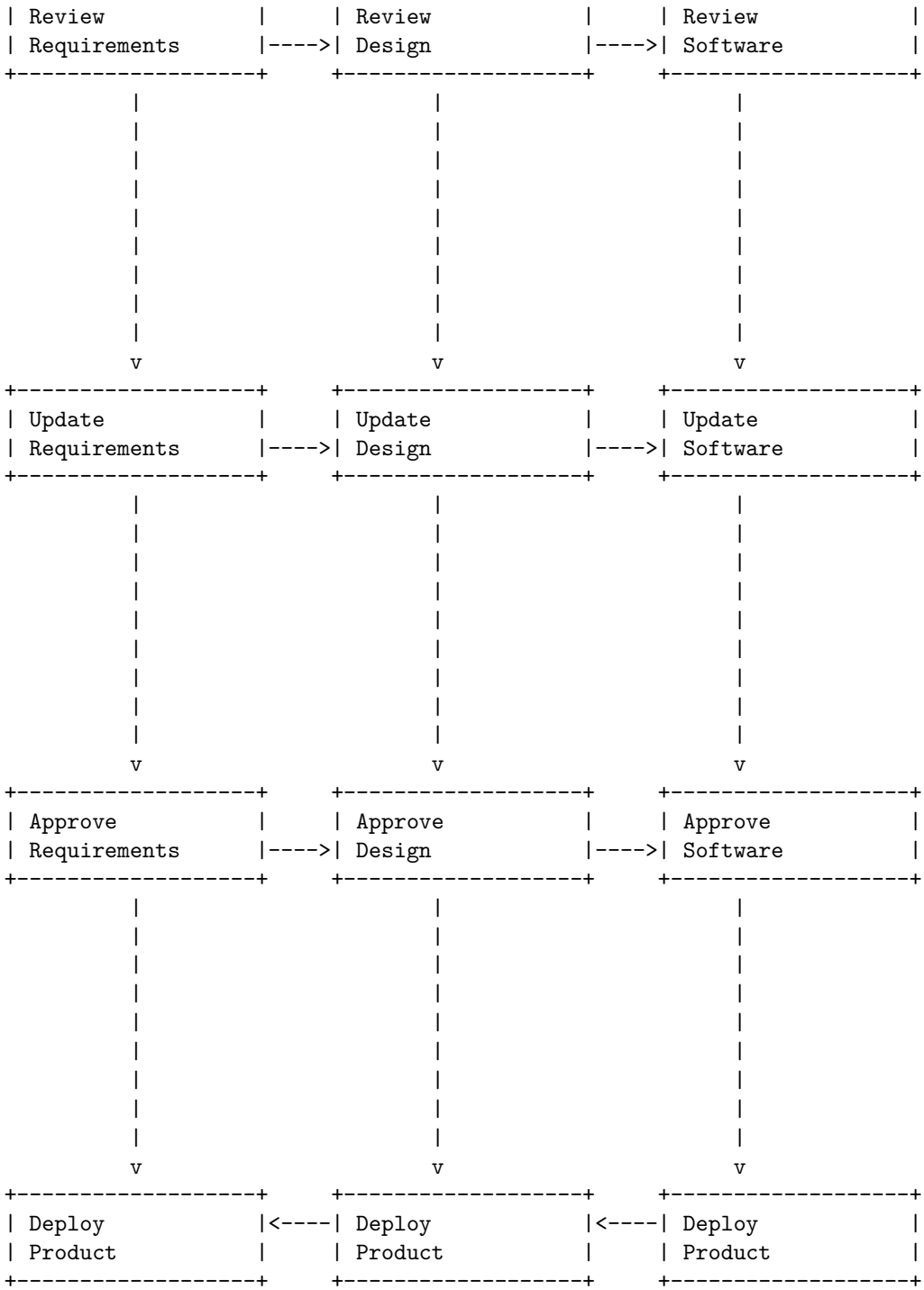
A flowchart is a diagram that shows the steps of a process or an algorithm in a logical order. Flowcharts are useful for designing, explaining, and documenting software processes. A flowchart typically uses different shapes and symbols to represent different types of actions, decisions, inputs, outputs, and flows.

Flow Charts in Software Design

The following diagram illustrates the basic architecture of a software design process using a flowchart. The flowchart shows the main steps involved in developing a software product, from defining the requirements, to designing the system, to coding, testing, and deploying the product. The flowchart also

shows the feedback loops and the decision points that may affect the flow of the process.



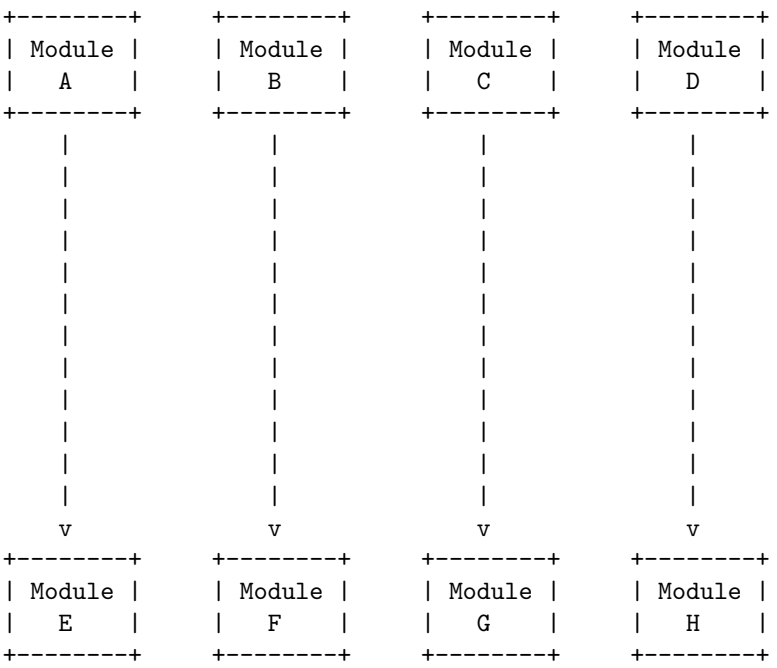

Review Requirements
Review Design
Review Software
Update Requirements
Update Design
Update Software
Approve Requirements
Approve Design
Approve Software
Deploy Product
Deploy Product
Deploy Product

Coupling in software engineering is the degree of interdependence between software modules; a measure of how closely connected two routines or modules are; the strength of the relationships between modules. Coupling is usually contrasted with cohesion, which is the degree of how well the elements of a module belong together. Low coupling often correlates with high cohesion, and vice versa .

There are different types of coupling, such as common coupling, content coupling, data coupling, stamp coupling, control coupling, and message coupling. Each type of coupling has a different level of dependency and complexity between modules.

Coupling in Software Design

The following diagram illustrates the basic concept of coupling in software design using ASCII characters. The boxes represent modules and the arrows represent dependencies. The direction of the arrow indicates which module depends on which other module. The number of arrows indicates the degree of coupling. More arrows mean higher coupling and less arrows mean lower coupling.




```
Low coupling: Module A depends only on Module E
High coupling: Module B depends on Module E, F, G, and H
```

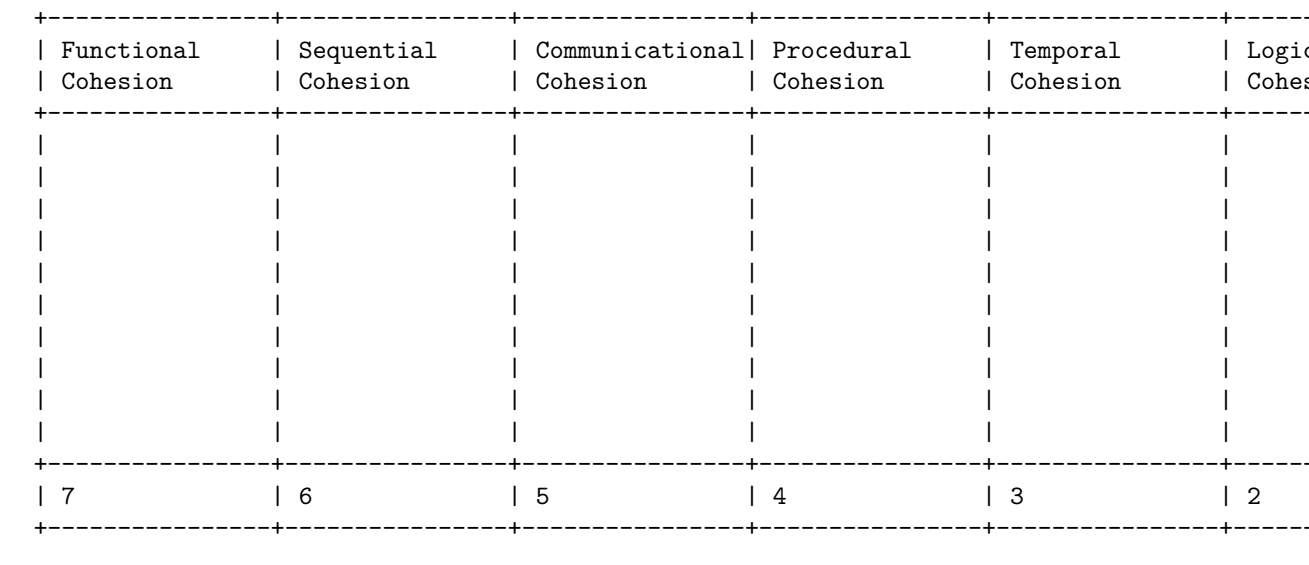
Cohesion is a measure of how strongly related and focused the various responsibilities of a software module are. It shows the functional strength of a module of

software. Cohesion is an ordinal type of measurement and is usually described as “high cohesion” or “low cohesion”. Modules with high cohesion tend to be preferable, because high cohesion is associated with several desirable traits of software including robustness, reliability, reusability, and understandability. In contrast, low cohesion is associated with undesirable traits such as being difficult to maintain, test, reuse, or even understand.

There are different types of cohesion, such as functional cohesion, sequential cohesion, communicational cohesion, procedural cohesion, temporal cohesion, logical cohesion, and coincidental cohesion. These types can be arranged in a hierarchy from the most desirable (functional cohesion) to the least desirable (coincidental cohesion).

Cohesion Measures in Software Design

The following diagram illustrates the different types of cohesion and their relative desirability using a scale from 1 (low) to 7 (high):



A brief description of each type of cohesion is given below:

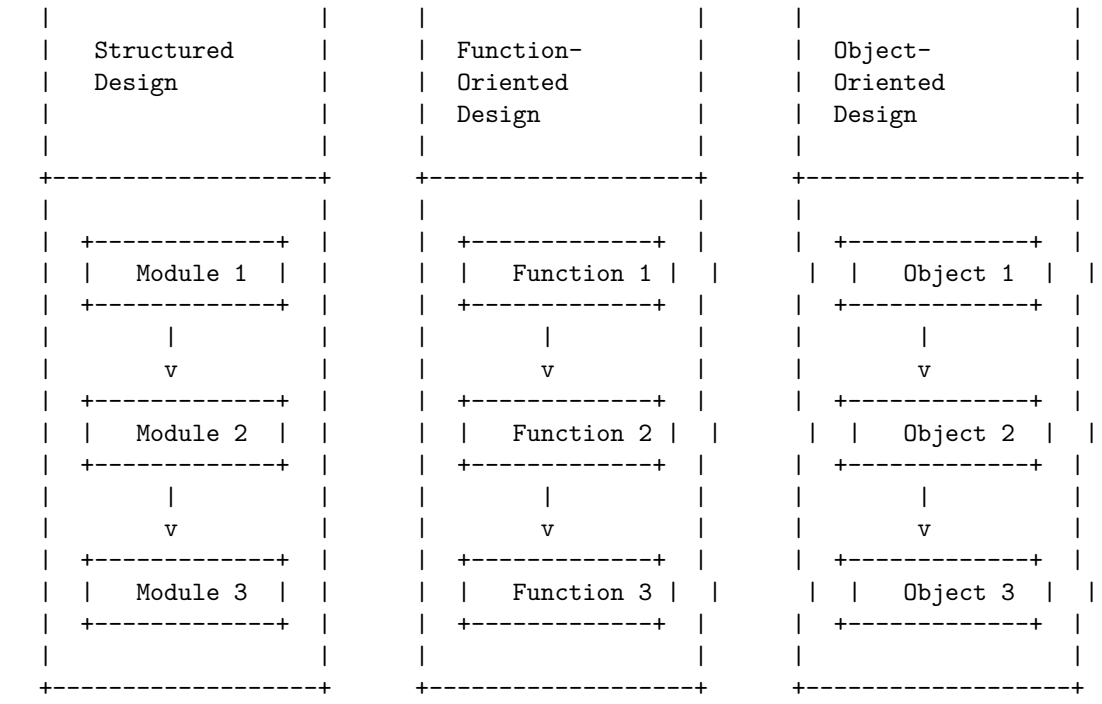
- Functional cohesion: The module performs a single specific task or function. For example, a module that calculates the area of a circle.
- Sequential cohesion: The module performs a series of related tasks or functions that must be executed in a specific order. For example, a module that reads data from a file, processes it, and writes the output to another file.
- Communicational cohesion: The module performs a series of related tasks or functions that operate on the same data or input/output device. For example, a module that performs different calculations on the same set of data.

- Procedural cohesion: The module performs a series of related tasks or functions that are grouped together because they follow a certain sequence of steps or a common procedure. For example, a module that validates user input, performs some calculations, and displays the results.
- Temporal cohesion: The module performs a series of related tasks or functions that are grouped together because they are executed at the same time or within the same time span. For example, a module that initializes a system, loads configuration files, and sets up connections.
- Logical cohesion: The module performs a series of related tasks or functions that are grouped together because they share some logical category or condition. For example, a module that handles different types of errors or exceptions.
- Coincidental cohesion: The module performs a series of unrelated tasks or functions that are grouped together arbitrarily or by coincidence. For example, a module that performs some calculations, prints a report, and sends an email.

Design strategies in software design are methods or approaches to solve software design problems. They help in defining the structure, behavior, and interactions of software components. Some of the common design strategies in software design are:

- Structured design: This is a conceptualization of problems into several well-organized elements of solutions. It is mainly concerned about the solution design. It uses a top-down approach to decompose the problem into smaller and simpler subproblems. It focuses on the functional aspects of the software and ignores the data aspects. It uses graphical tools such as data flow diagrams and structure charts to represent the software design.
- Function-oriented design: This is one of the classical methods of software design, where decomposition centers on identifying the major software functions and then elaborating and refining them in a top-down manner. It also considers the data aspects of the software and uses data dictionaries and entity-relationship diagrams to model the data. It uses functional abstraction and information hiding to achieve modularity and reusability. It follows the principle of stepwise refinement to design the software.
- Object-oriented design: This is a modern method of software design, where decomposition centers on identifying the major software objects and then defining their attributes and behaviors. It uses a bottom-up approach to combine the objects into larger and more complex systems. It focuses on the data aspects of the software and encapsulates the data and the functions that operate on them into a single unit called an object. It uses graphical tools such as class diagrams and sequence diagrams to represent the software design.

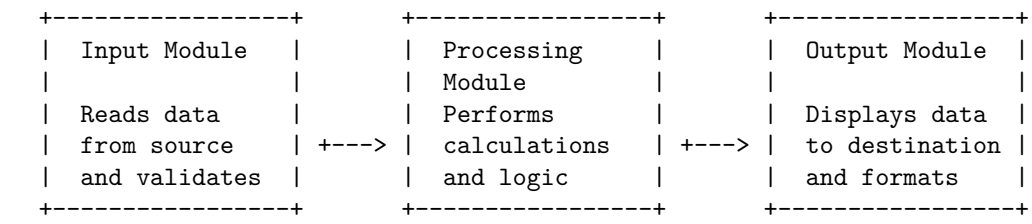
The following diagram illustrates the basic architecture of a software system using each of these design strategies:



Function Oriented Design is a method to software design where the model is decomposed into a set of interacting units or modules where each unit or module has a clearly defined function . The system is designed from a functional viewpoint .

Function Oriented Design in Software Design

The following diagram illustrates the basic architecture of a Function Oriented Design in Software Design using ASCII characters:



Each module has a specific function and communicates with other modules through data flows. The data flows are represented by arrows and show the direction and nature of data movement. The modules can be further decomposed into submodules if needed.

Object-oriented design (OOD) is the process of using an object-oriented methodology to design a computing system or application. This technique enables the

implementation of a software solution based on the concepts of objects. OOD serves as part of the object-oriented programming (OOP) process or lifecycle.

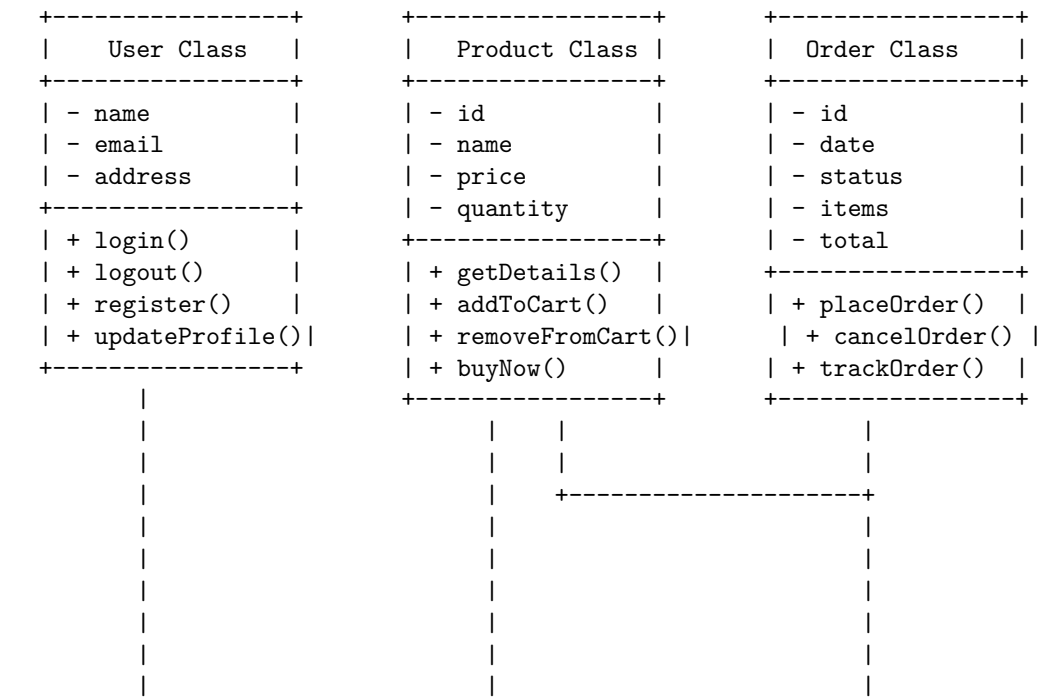
An object is a software entity that contains encapsulated data and procedures grouped together to represent an entity. Objects interact with each other through well-defined interfaces, which specify the services provided by an object and the messages that an object can receive and send.

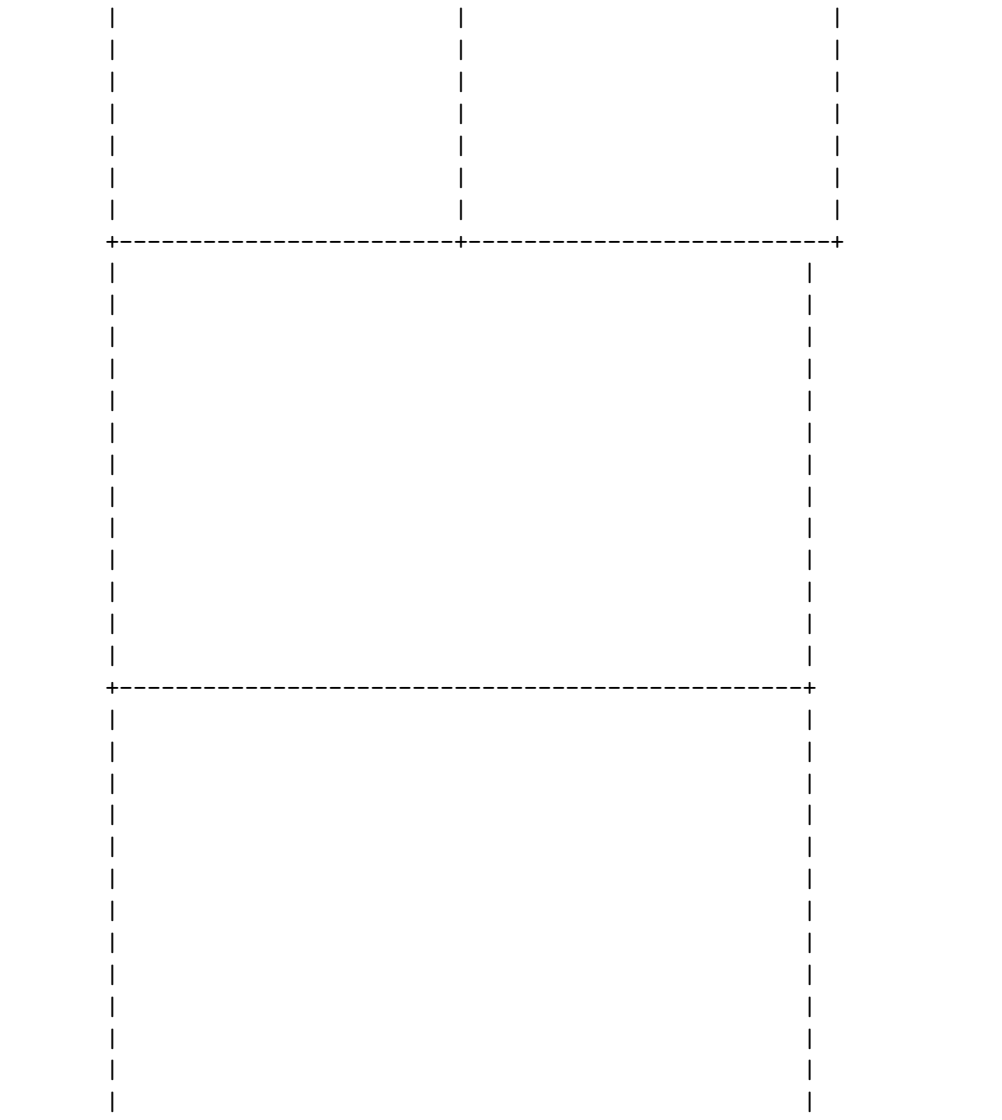
One of the main goals of OOD is to achieve high cohesion and low coupling among the objects in a system. Cohesion refers to the degree of relatedness of the elements within an object, while coupling refers to the degree of dependency of an object on other objects. High cohesion and low coupling make the system easier to maintain, extend, and reuse.

There are several principles and techniques that can guide the OOD process, such as abstraction, encapsulation, inheritance, polymorphism, modularity, and design patterns. These concepts help to define the structure and behavior of the objects, as well as their relationships and interactions .

The following diagram illustrates the basic architecture of a typical object-oriented system, using the Unified Modeling Language (UML) notation. UML is a standard graphical language for modeling and documenting software systems, especially those based on OOD.

Object Oriented Design in Software Design



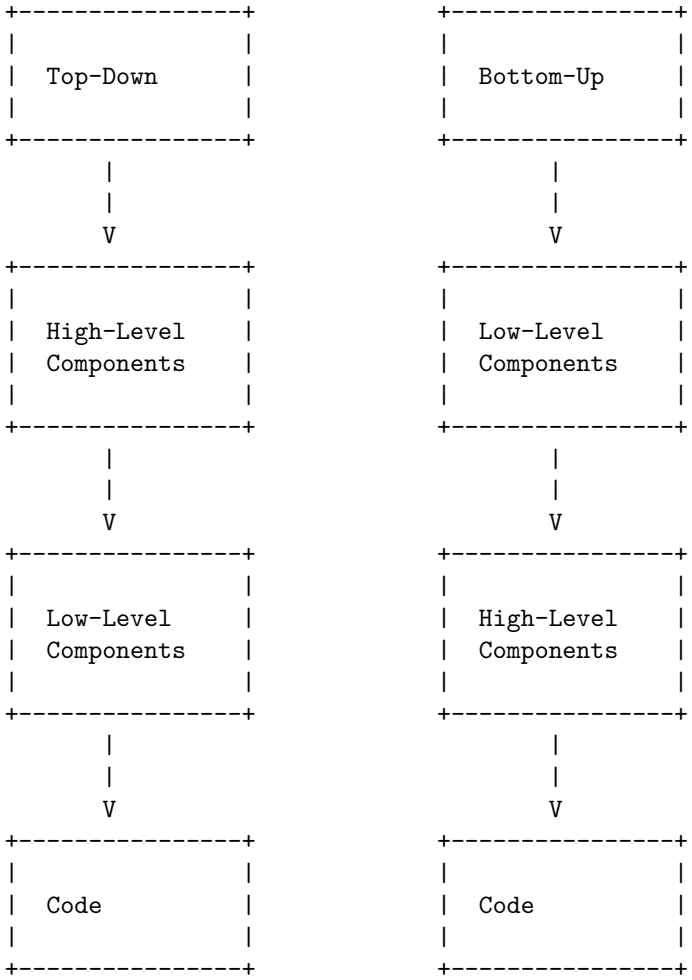

Database Class
- connection
- query
+ connect()
+ execute()
+ close()
Email Class
- sender
- receiver
- subject
- body
+ send()
Payment Class
- amount
- method
- status
+ pay()
+ refund()

The diagram shows three main classes that represent the entities in an online shopping system: User, Product, and Order. Each class has some

Top-down and bottom-up design are two strategies of software design that can be used in a variety of fields. Top-down design emphasizes planning and a complete understanding of the system, while bottom-up design starts with the most specific and basic components and proceeds with composing higher level components by using them.

Top-Down and Bottom-Up Design in Software Design

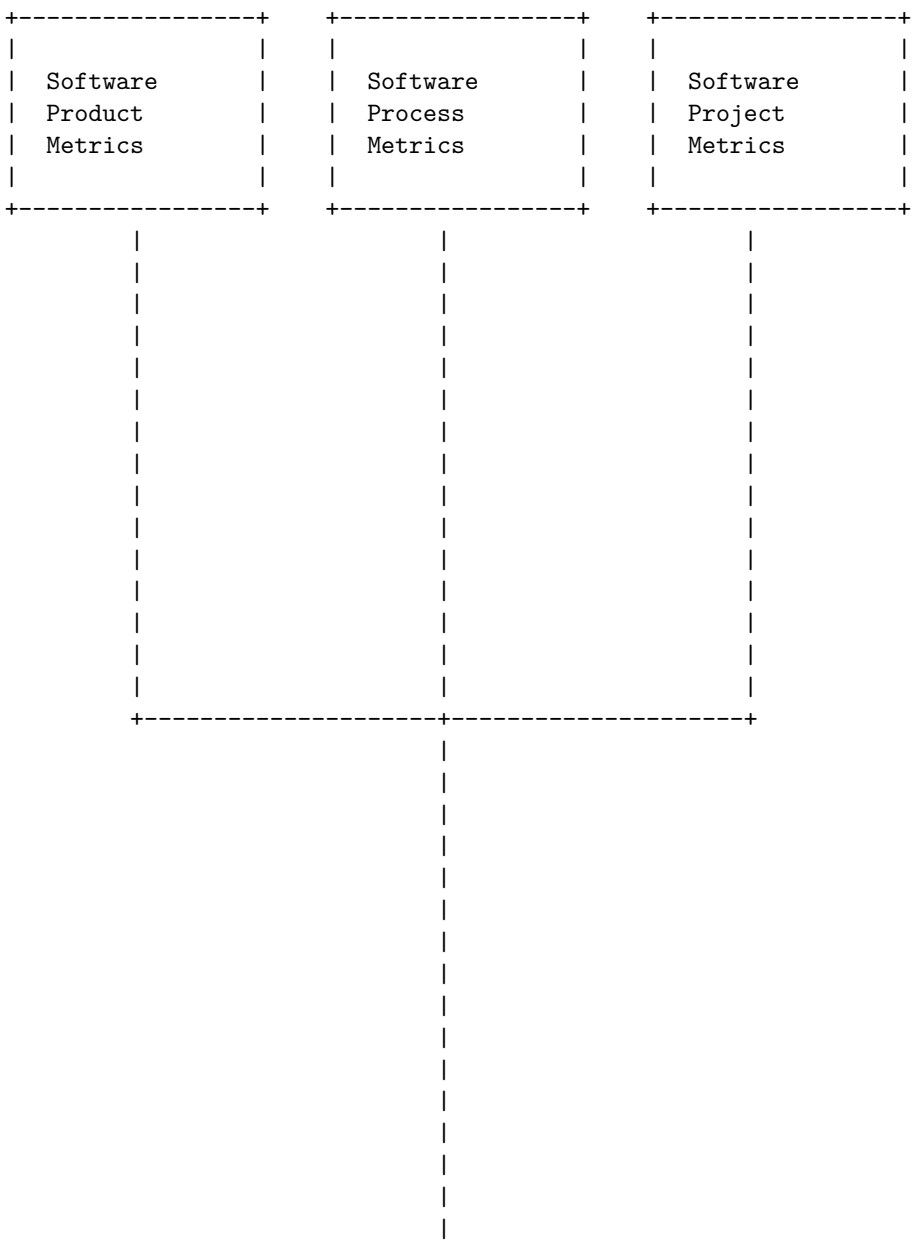
The following diagram illustrates the basic difference between top-down and bottom-up design in software design using ASCII art:



Software measurement and metrics are used to evaluate the quality, performance,

reliability, usability, and other attributes of software products, processes, and projects. Software metrics are functions that map software characteristics to numerical values, while software measurements are the numbers obtained by applying software metrics.

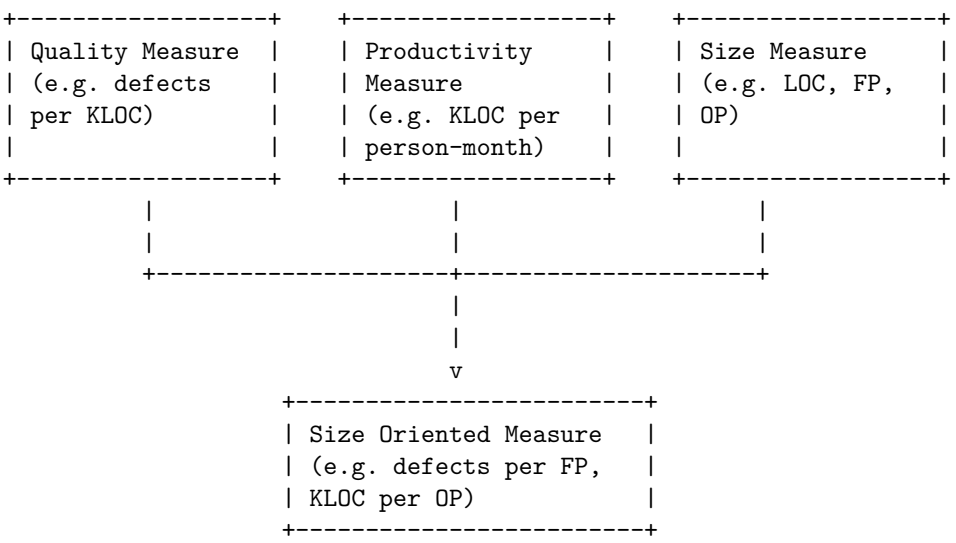
The following diagram shows a general framework for software measurement and metrics in software design, using ASCII characters:



Various size oriented measures are derived by normalizing quality and produc-

tivity measures by considering the size of the software that has been produced. Size is a direct and easily measurable attribute of software. However, size can be measured in different ways, such as lines of code, function points, object points, etc. Each of these measures has its own advantages and disadvantages, and may be suitable for different types of software projects.

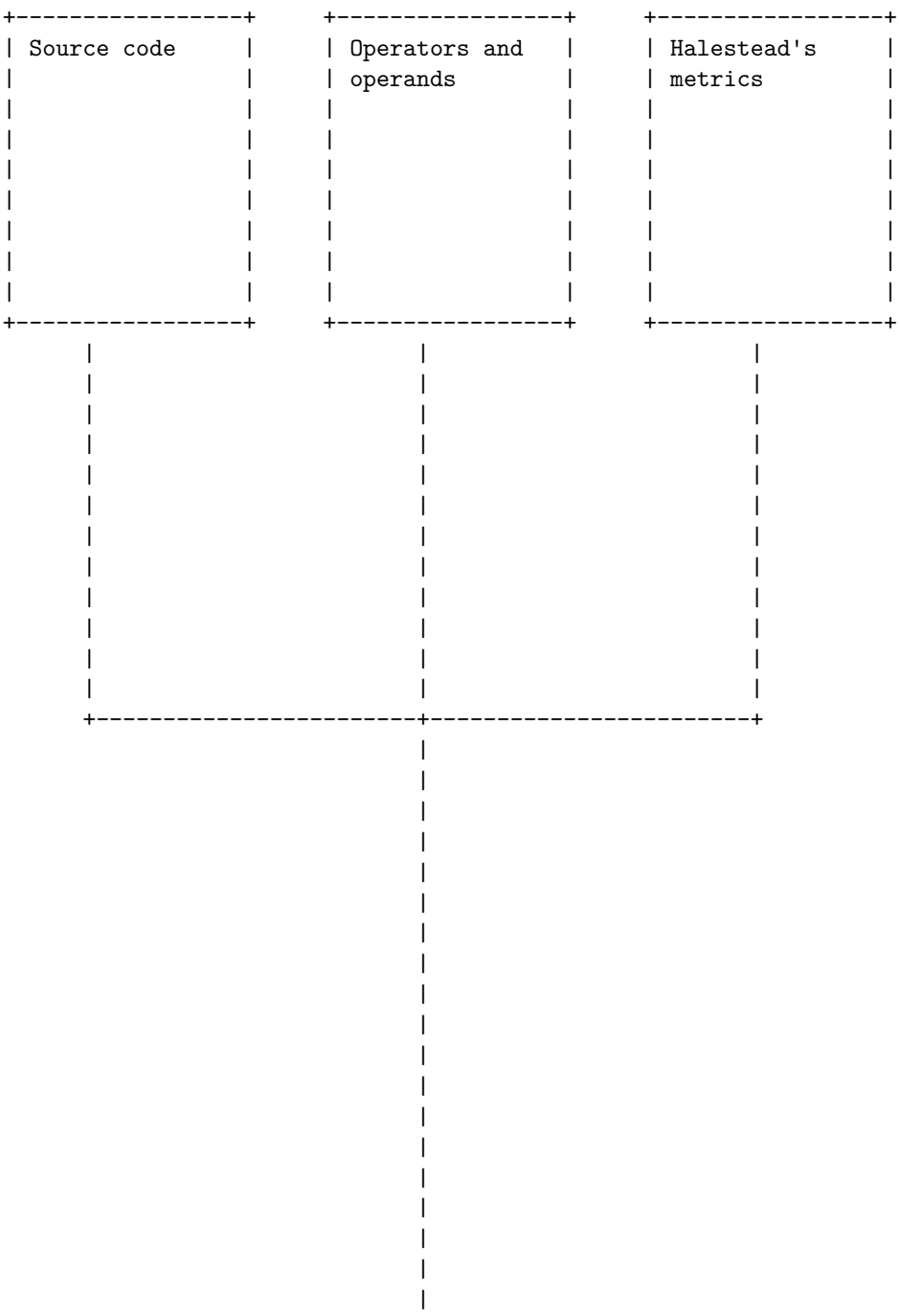
The following diagram illustrates the basic architecture of a size oriented measure in software design:



Halestead's Software Science is a set of software metrics that measure the complexity and quality of a program based on the number and types of operators and operands used in the source code. The basic idea is that a program can be seen as a collection of tokens, which are either operators (symbols that perform some action) or operands (data or variables that are acted upon). Halestead's Software Science defines the following measures:

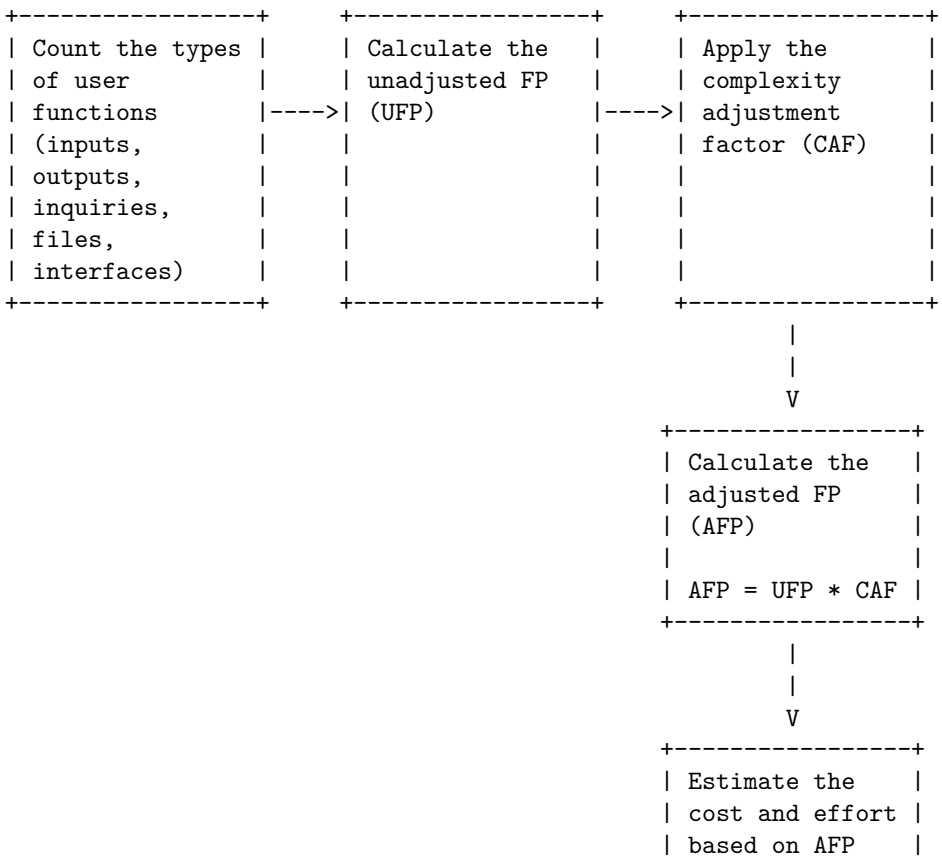
- n1: the number of distinct operators
- n2: the number of distinct operands
- N1: the total number of operators
- N2: the total number of operands
- n: the program vocabulary, defined as n = n1 + n2
- N: the program length, defined as N = N1 + N2
- V: the program volume, defined as V = N * log2(n)
- D: the program difficulty, defined as D = (n1/2) * (N2/n2)
- E: the program effort, defined as E = D * V
- T: the program time, defined as T = E / S, where S is a constant that represents the speed of the programmer
- L: the program level, defined as L = 1 / D
- B: the program error estimate, defined as B = E^(2/3) / 3000

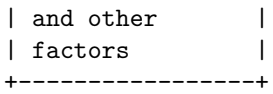
The following diagram illustrates the basic architecture of Halestead's Software Science in software design using ASCII art:



Function Point (FP) Based Measures are a technique to estimate the cost and effort of software development and maintenance based on the functionality that the user requests and receives. FP are derived using an empirical relationship based on countable measures of software's information domain and assessments of software complexity .

The following diagram illustrates the basic steps of FP based estimation in software design:

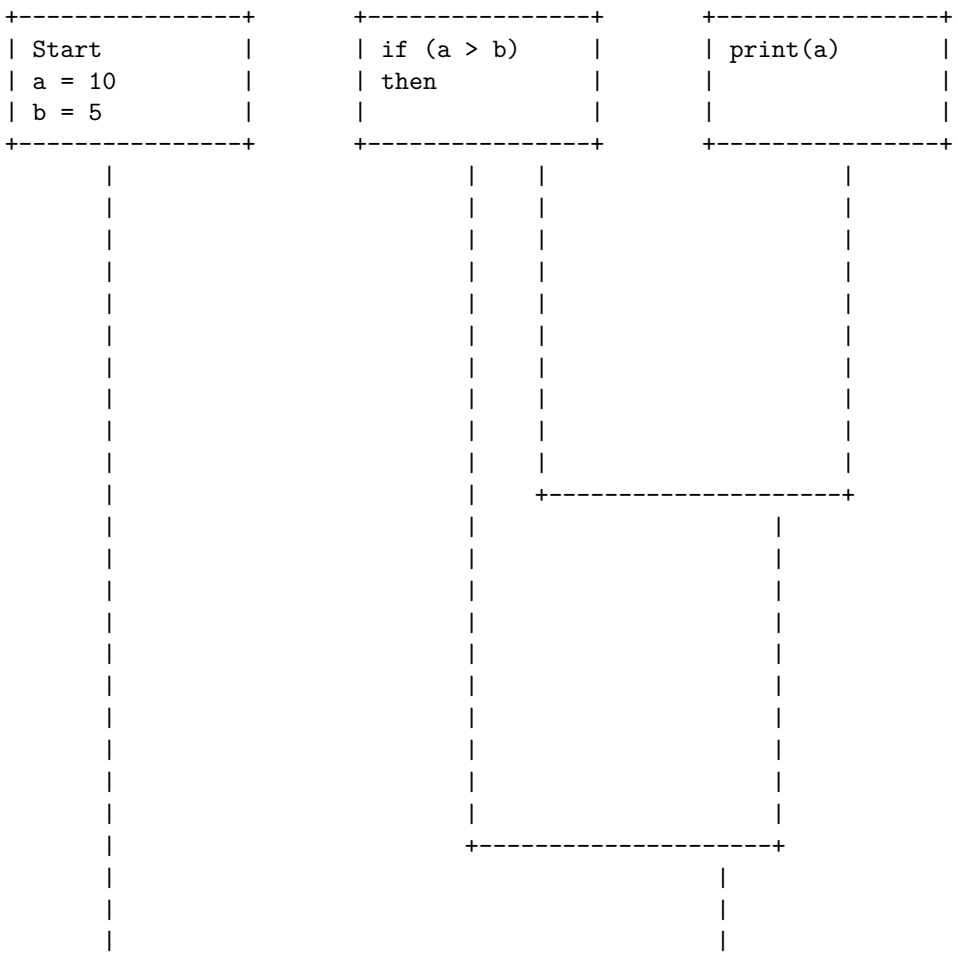


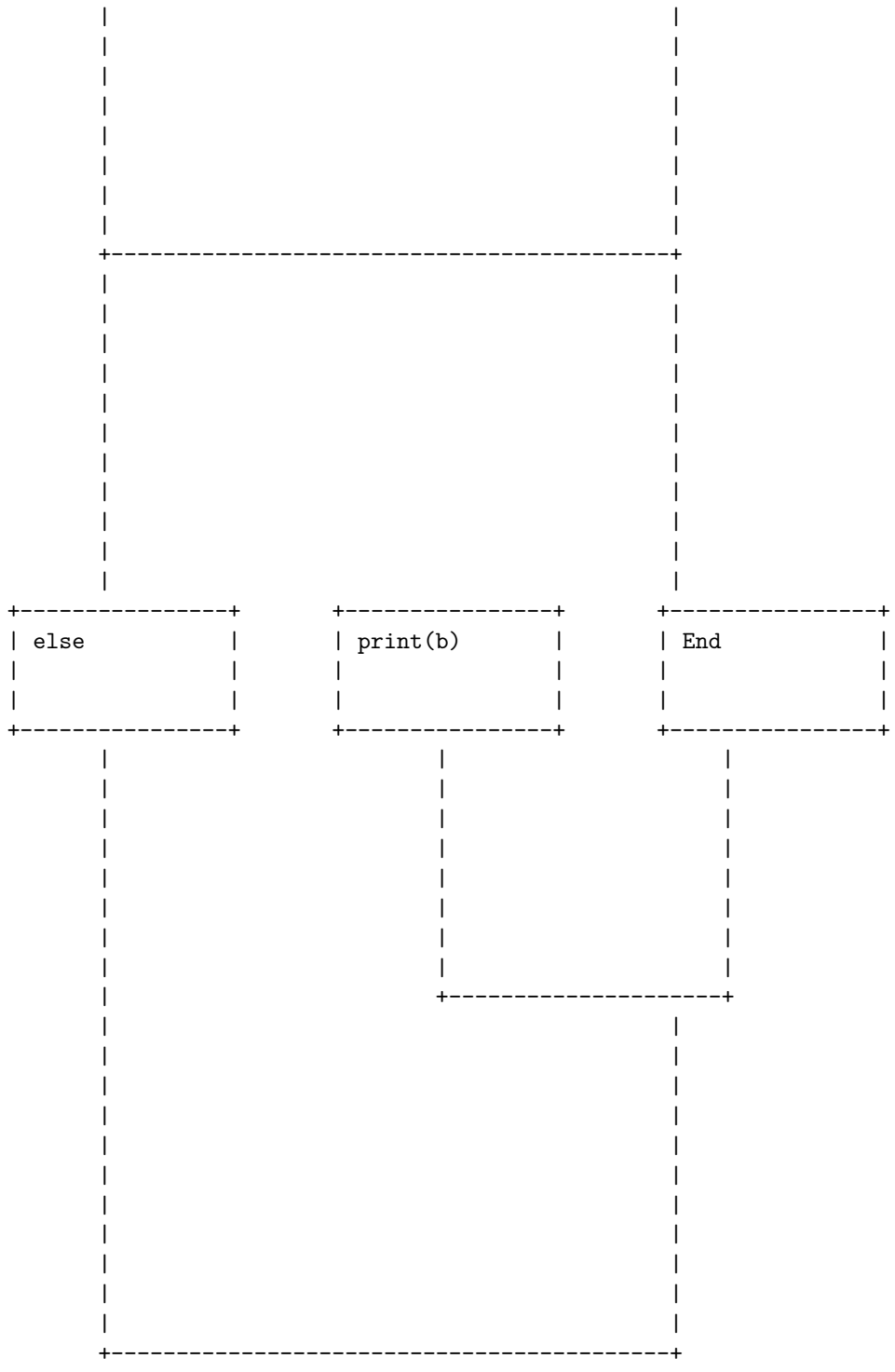


Cyclomatic complexity is a software metric that measures the number of independent paths through a program's source code. It is calculated as the number of edges minus the number of nodes plus two in the control flow graph of the program. A control flow graph is a representation of the program's structure, where each node is a basic block of code and each edge is a possible flow of control between the blocks. The cyclomatic complexity can be used to estimate the testing effort and the maintainability of the program.

Cyclomatic Complexity Measures in software design

The following diagram illustrates the basic architecture of a control flow graph and how to calculate the cyclomatic complexity using an example program.




else
print(b)
End

In this diagram, there are seven nodes (basic blocks of code) and nine edges (possible flows of control). Therefore, the cyclomatic complexity is 9 - 7 + 2 = 4. This means that there are four independent paths through the program, which can be identified as:

- Start -> if (a > b) -> print(a) -> End
- Start -> if (a > b) -> else -> print(b) -> End
- Start -> if (a > b) -> print(a) -> print(b) -> End
- Start -> if (a > b) -> else -> print(b) -> print(a) -> End

These paths correspond to the different combinations of the condition (a > b) being true or false. To test the program thoroughly, each path should be executed at least once with appropriate input values. The higher the cyclomatic complexity, the more paths there are and the more testing effort is required. The cyclomatic complexity can also indicate the maintainability of the program, as a high complexity may imply a high risk of errors and a low readability. A general guideline is to keep the cyclomatic complexity below 10 for each function or module.

A control flow graph (CFG) is a graphical representation of the possible paths of execution of a program or a function. It consists of nodes and edges, where nodes represent basic blocks of code (sequences of statements that are always executed together) and edges represent the flow of control between them. The entry node is the starting point of the program or function, and the exit node is the end point. A CFG can be used for various purposes, such as static analysis, compiler optimization, testing, debugging, and simulation.

A basic block is a maximal sequence of statements that has a single entry point and a single exit point. A basic block can be identified by finding the leaders, which are the first statements of a basic block. The leaders are:

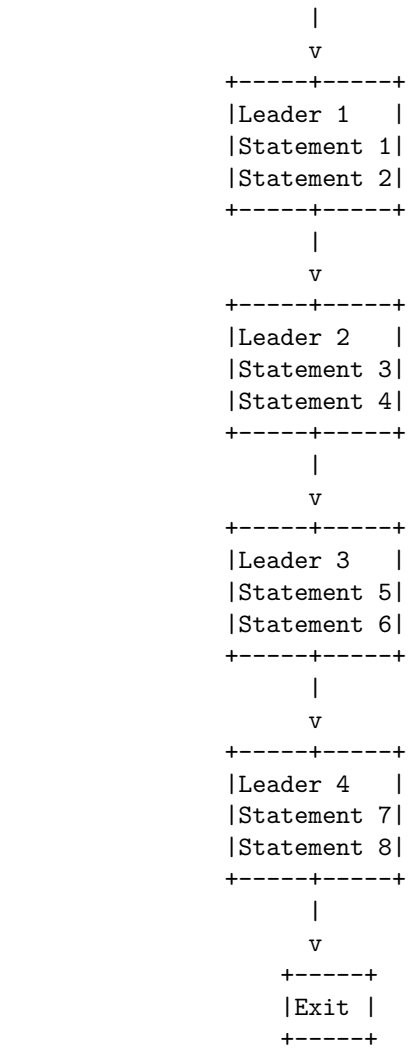
- The first statement of the program or function.
- Any statement that is the target of a jump, branch, or loop instruction.
- Any statement that immediately follows a jump, branch, or loop instruction.

To construct a CFG, the following steps can be followed:

- Identify the basic blocks and their leaders.
- Draw a node for each basic block and label it with the leader's line number or name.
- Draw an edge from one node to another if there is a possible flow of control from the first node's basic block to the second node's basic block.
- Mark the entry node and the exit node with special symbols.

The following diagram illustrates the basic architecture of a CFG:

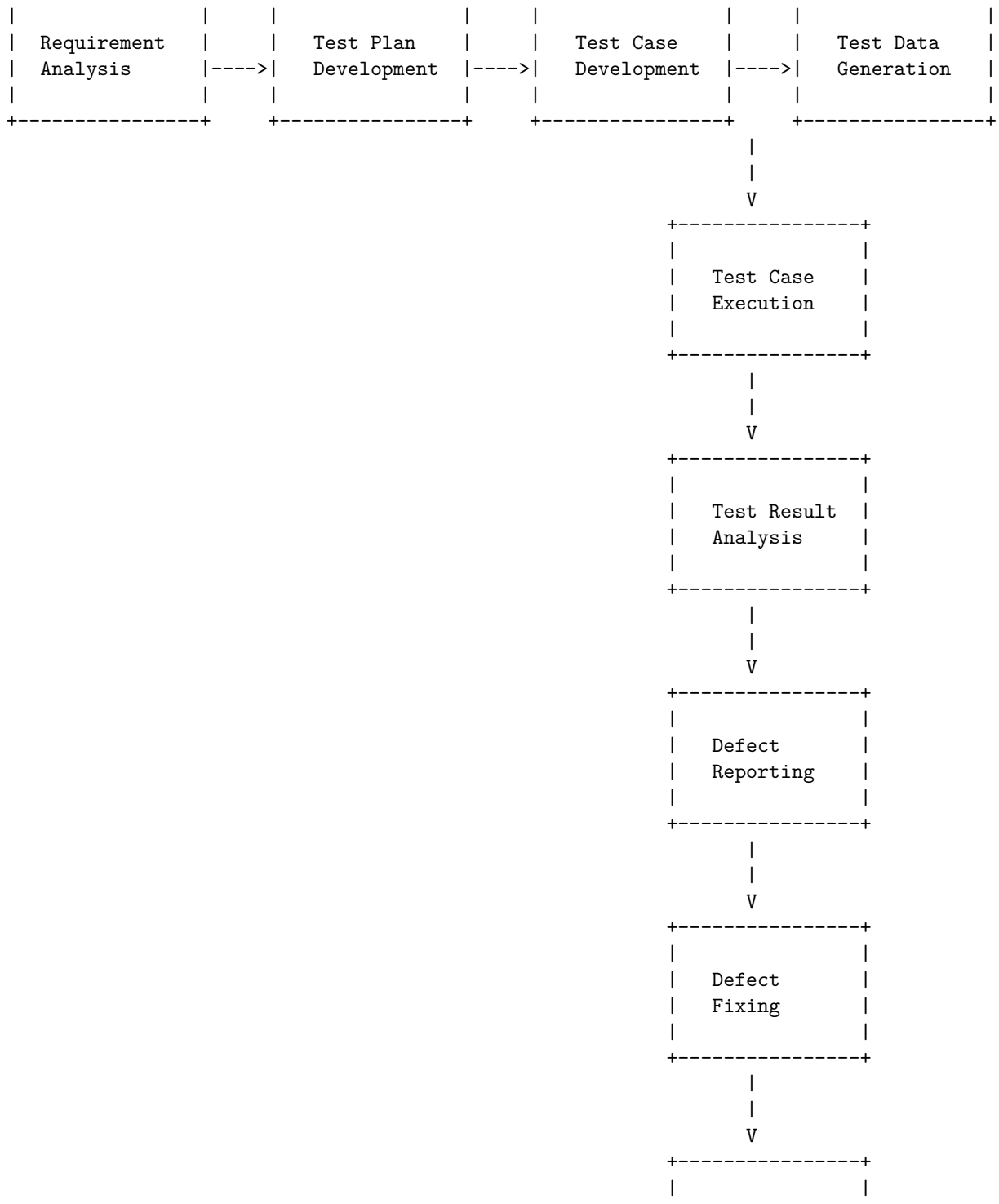
```
+-----+
|Entry|
+-----+
```

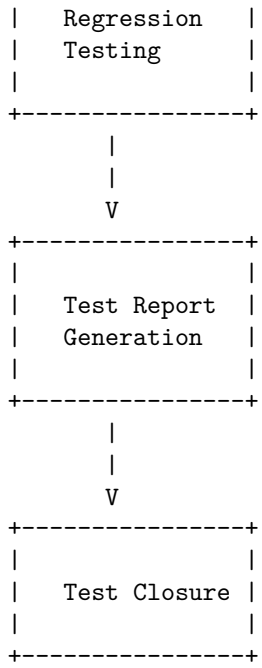


Unit 4 - Software Testing

Software testing is the process of verifying and validating that a software product or service meets the specified requirements and expectations of the stakeholders. Software testing can be performed at different levels of granularity, such as unit testing, integration testing, system testing, and acceptance testing. Software testing can also follow different approaches, such as black-box testing, white-box testing, and gray-box testing. Software testing can be done manually or with the help of automated tools.

The following diagram illustrates the basic phases of a software testing process:


Requirement Analysis
Test Plan Development
Test Case Development
Test Data Generation
Test Case Execution
Test Result Analysis
Defect Reporting
Defect Fixing



The diagram shows the following steps:

- Requirement analysis: The process of understanding the business and technical requirements of the software product or service, and identifying the test objectives and scope.
- Test plan development: The process of defining the test strategy, test environment, test resources, test schedule, test deliverables, and test risks.
- Test case development: The process of designing and documenting the test scenarios, test steps, test inputs, and expected outputs for each test objective.
- Test data generation: The process of creating or obtaining the data sets that are required to execute the test cases.
- Test case execution: The process of running the test cases on the software product or service under test, and recording the actual outputs and test status.
- Test result analysis: The process of comparing the actual outputs with the expected outputs, and determining the test outcome (pass or fail) and test coverage.
- Defect reporting: The process of logging and tracking the defects that are found during the test case execution, and assigning them to the responsible developers or testers.
- Defect fixing: The process of resolving the defects that are reported by the testers, and verifying that they are fixed correctly.
- Regression testing: The process of re-testing the software product or service after the defects are fixed, to ensure that no new defects are introduced

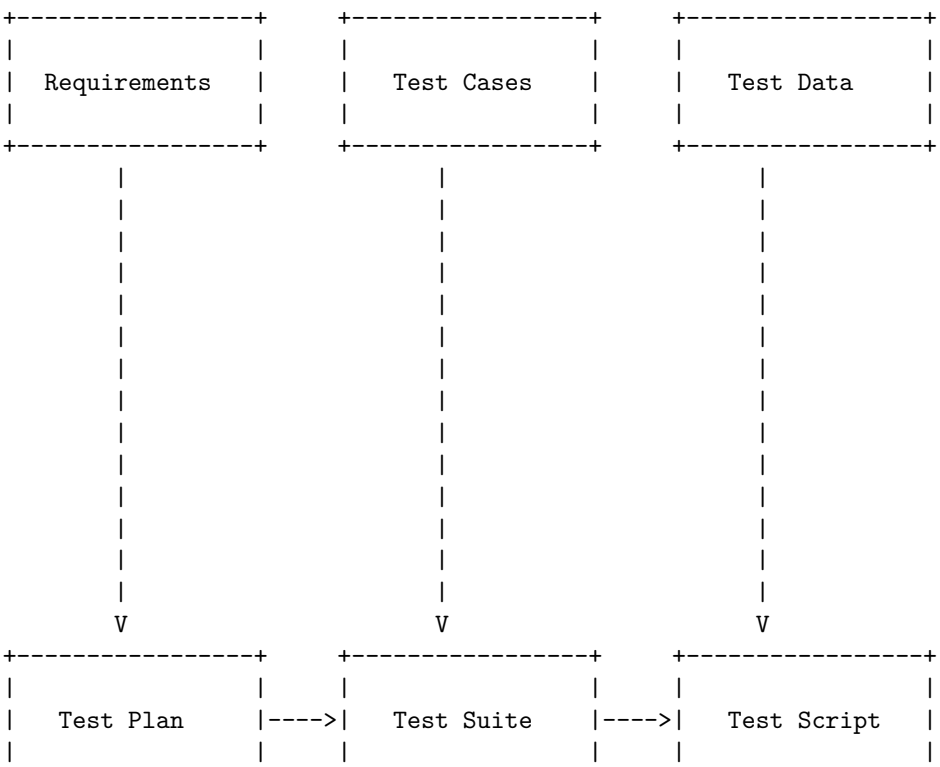
and the existing functionality is not affected.
- Test report generation: The process of summarizing and presenting the test results, test coverage, defect status, and test metrics in a formal document or dashboard.
- Test closure: The process of evaluating the test process and deliverables, and identifying the lessons learned and best practices for future improvement.

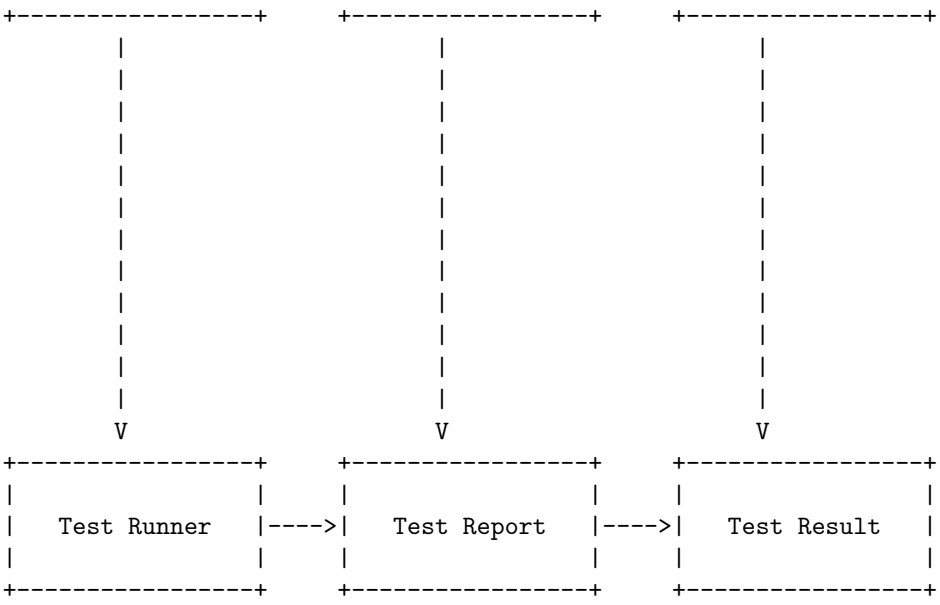
Testing Objectives in Software Testing are the goals and expectations that guide the testing process and its outcomes. Some of the common objectives are:

- To check whether the software meets the requirements and specifications
- To find and fix defects before the software is delivered to the customers
- To gain confidence in and provide information about the quality and reliability of the software
- To prevent defects from occurring or recurring in the future
- To ensure that the software is usable, secure, efficient and maintainable

The following diagram illustrates the basic architecture of a software testing process using ASCII art:

Testing Objectives in Software Testing





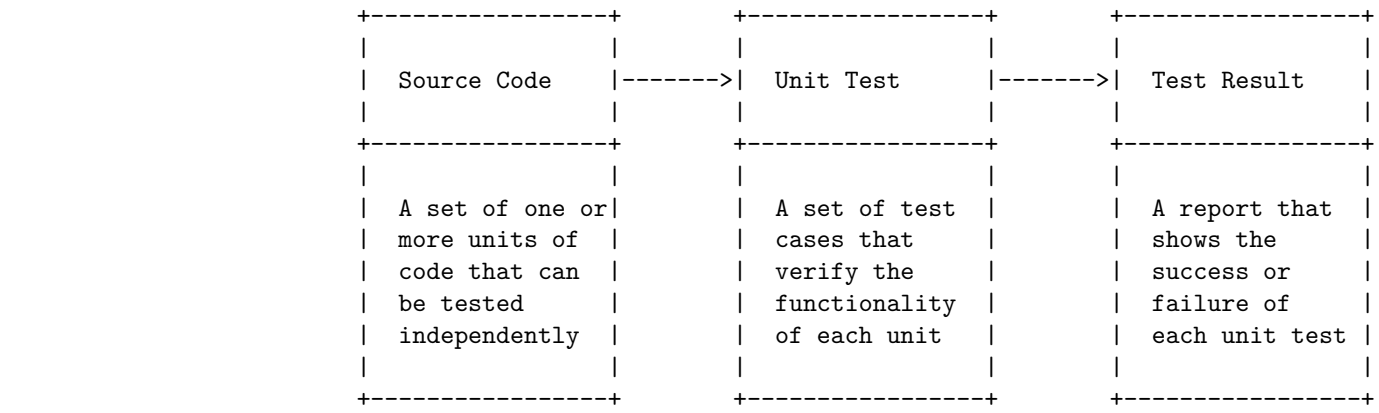
The diagram shows the following steps:

- The requirements are the expectations and specifications of the software that are defined by the stakeholders and customers.
- The test cases are the scenarios and conditions that are used to verify the requirements and find defects in the software.
- The test data are the inputs and outputs that are used to execute the test cases and simulate the real-world situations.
- The test plan is the document that describes the scope, objectives, strategy, resources, schedule and risks of the testing process.
- The test suite is the collection of test cases that are grouped by functionality, feature, module or component of the software.
- The test script is the code or command that automates the execution of the test cases and test data.
- The test runner is the tool or framework that runs the test script and interacts with the software under test.
- The test report is the document that summarizes the test execution, test coverage, test metrics and test findings.
- The test result is the outcome of the test execution, which can be pass, fail, error or skip.

Unit testing is a software testing method by which individual units of source code are tested to determine whether they are fit for use. A unit can be a function, method, module, object, or other entity in an application's source code. Unit testing is performed during the coding stage of a software development project to test individual units of the application. It's designed to test that each unit of the software code performs as required.

Unit Testing in Software Testing

The following is a possible ASCII diagram for unit testing in software testing:



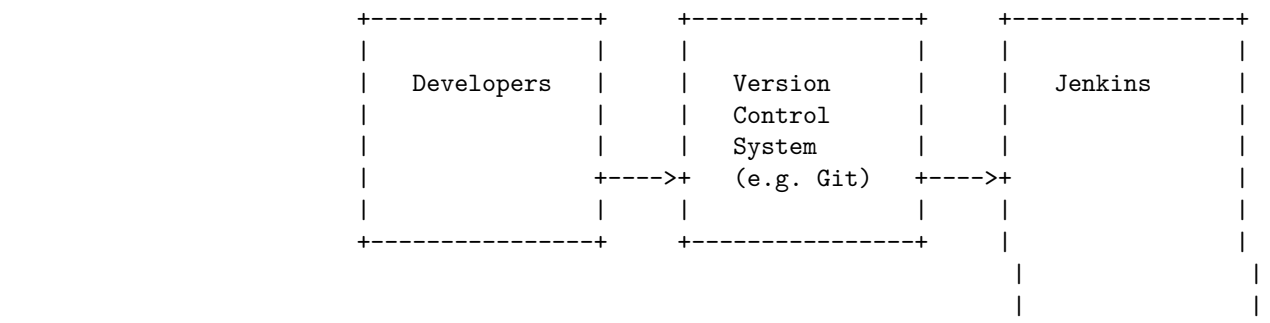
Integration testing is a level of software testing where individual units or components are combined and tested as a group to verify if they are working as intended when integrated. The purpose of this level of testing is to expose faults in the interaction between integrated units.

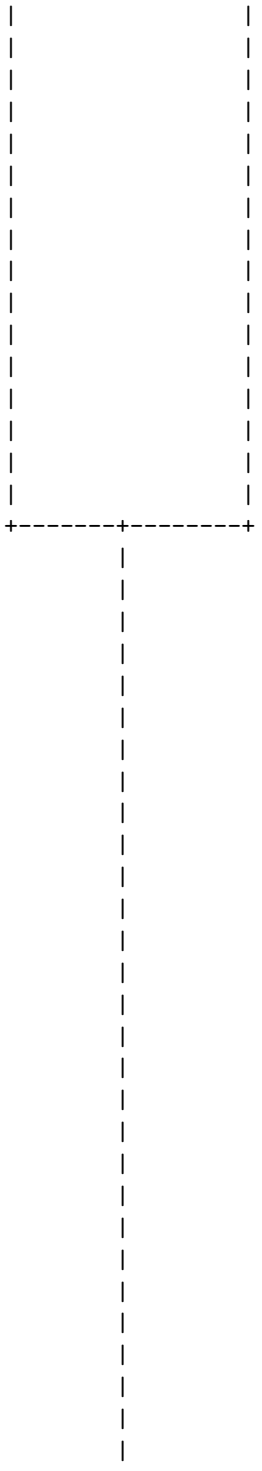
There are different types of integration testing, such as:

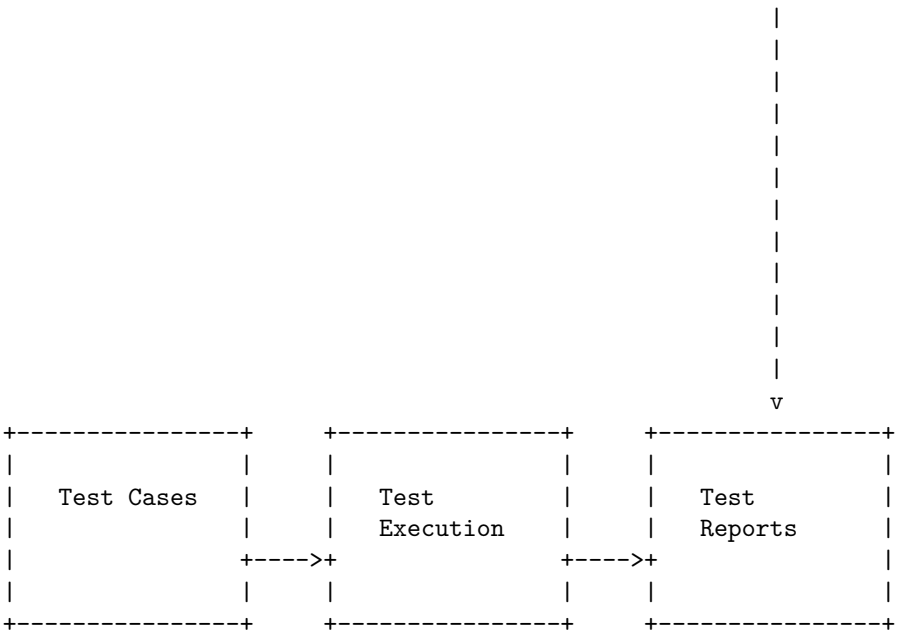
- Big bang integration testing: All the modules or components are integrated and tested together as a whole after the development is complete.
- Incremental integration testing: The modules or components are integrated and tested gradually as they are developed. This can be further divided into top-down, bottom-up, and sandwich integration testing, depending on the order of integration.
- Continuous integration testing: The modules or components are integrated and tested continuously using automated tools and frameworks.

The following diagram illustrates the basic architecture of a continuous integration testing process using a tool like Jenkins:

Integration Testing in Software Testing





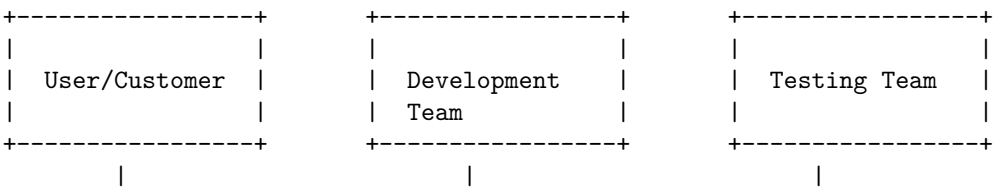


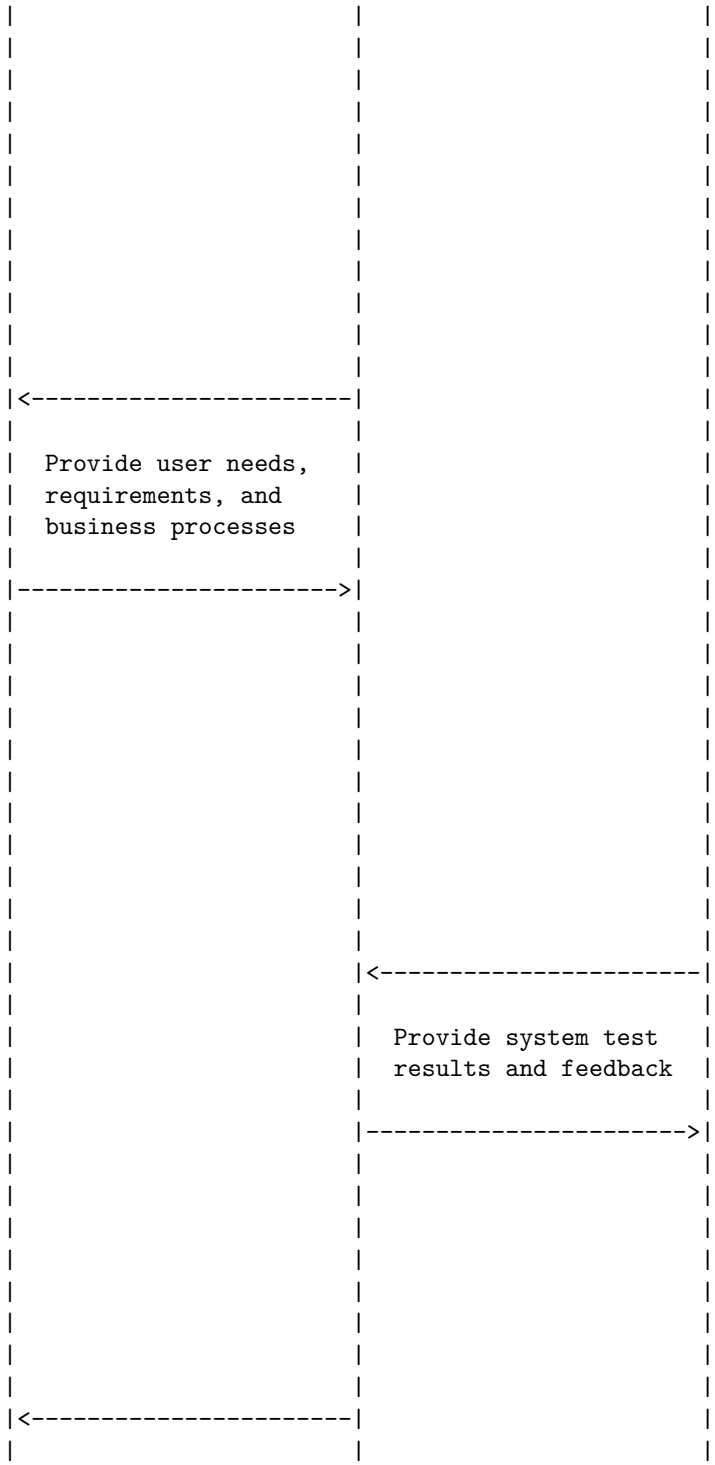
The diagram shows the following steps:

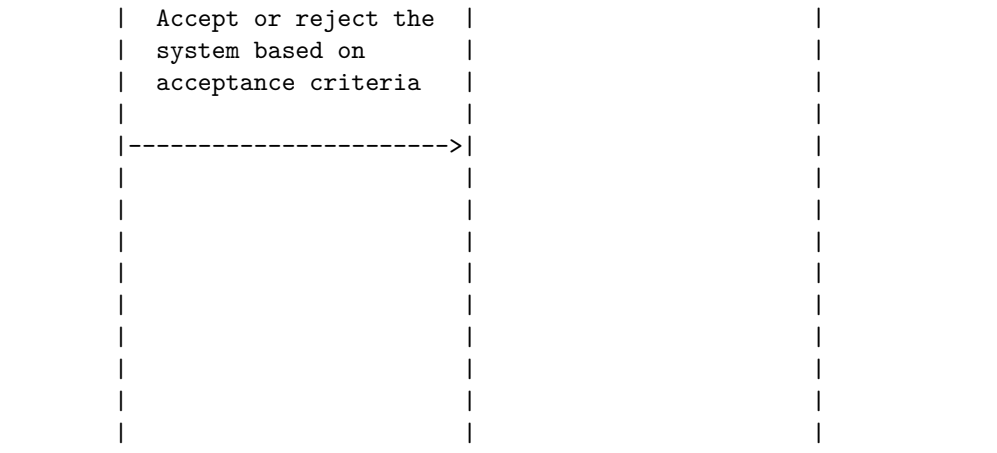
- Developers write code and push it to a version control system (e.g. Git).
- Jenkins, a continuous integration tool, monitors the version control system and triggers a build whenever there is a new commit.
- Jenkins executes the test cases that are written for the integrated modules or components.
- Jenkins generates test reports that show the results and status of the integration testing.

Acceptance testing is a level of software testing that evaluates the system's compliance with the user needs, requirements, and business processes. It is conducted to determine whether the system satisfies the acceptance criteria and whether the user, customer, or other authorized entity is willing to accept the system. Acceptance testing occurs after system testing, but before deployment. It is usually done manually, with users creating real-world situations and testing how the software reacts and performs.

The following diagram illustrates the basic architecture of acceptance testing in software testing using ASCII characters:



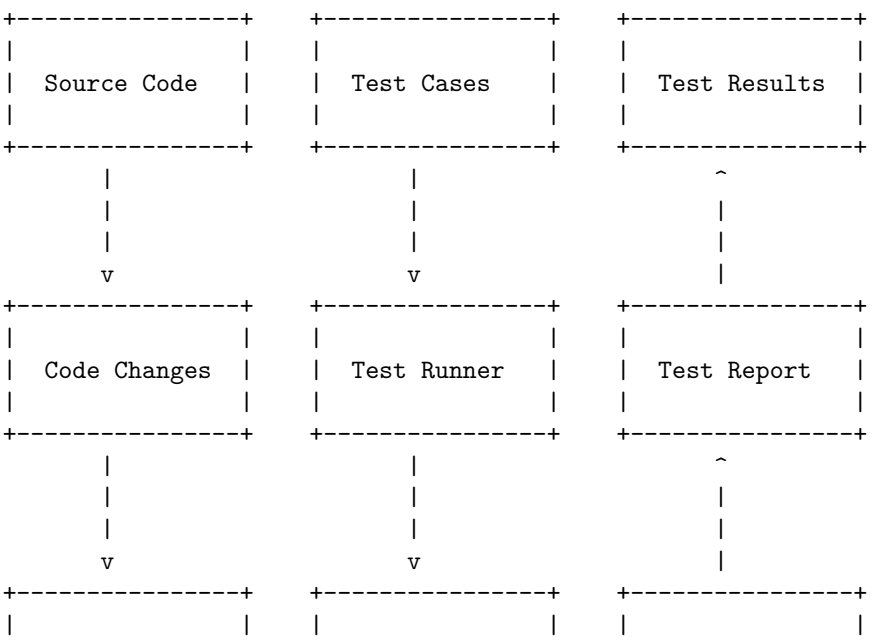

Provide user needs, requirements, and business processes
Provide system test results and feedback

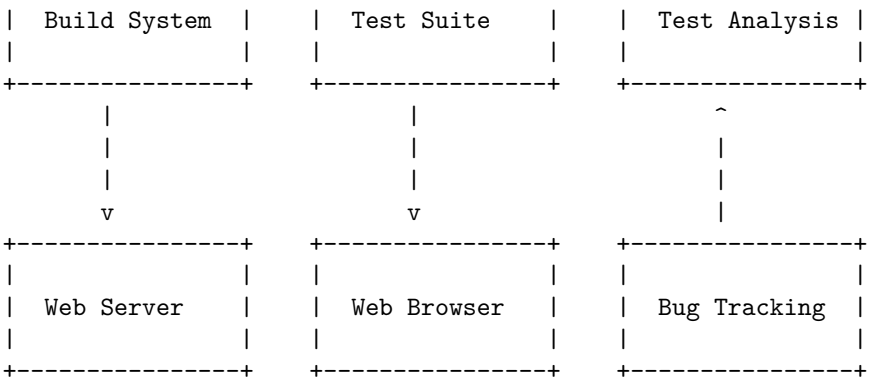


Regression testing is a software testing practice that ensures an application still functions as expected after any code changes, updates, or improvements. Regression testing is responsible for the overall stability and functionality of the existing features.

There are different types of regression testing, such as corrective, progressive, selective, complete, and partial. Each type has its own advantages and disadvantages depending on the scope, complexity, and frequency of the changes.

The following diagram illustrates the basic architecture of a regression testing process using the example of a web application:



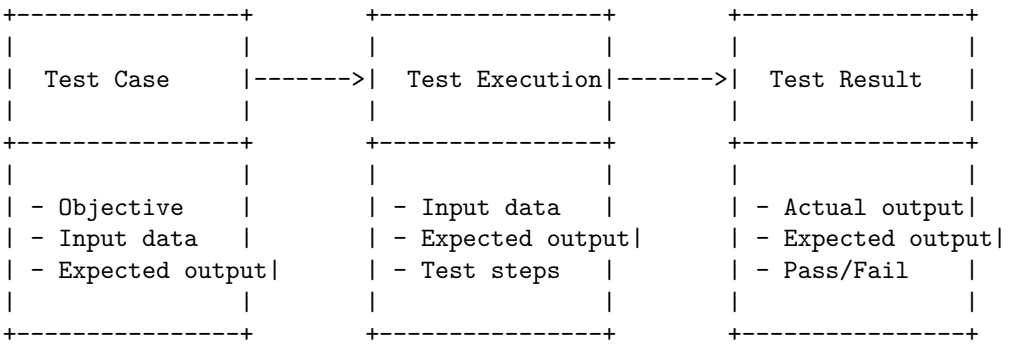


The diagram shows the following steps:

- The source code is modified by the developers to implement new features or fix bugs.
- The test cases are written or updated by the testers to cover the expected behavior of the application.
- The code changes are compiled and deployed to the web server by the build system.
- The test runner executes the test suite on the web browser, which interacts with the web server and the application.
- The test results are collected and reported by the test runner, which shows the status of each test case (pass, fail, skip, etc.).
- The test report is analyzed by the testers, who verify if the application meets the requirements and if there are any regression issues.
- The bug tracking system is used to record and track any defects found during the testing process.

Testing for functionality in software testing is a type of testing that verifies the system's behavior against the functional requirements or specifications. It ensures that the system performs the functions that the user expects and meets the quality standards.

A possible diagram for testing for functionality in software testing is:

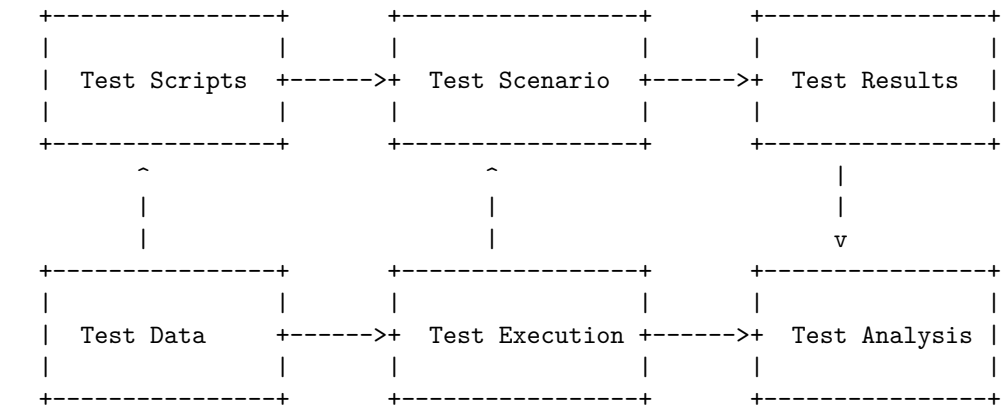


The diagram shows the basic steps of functional testing. A test case is a document that defines the objective, input data and expected output of a test. A test execution is the process of running the test case on the system and observing its behavior. A test result is the outcome of the test execution, which compares the actual and expected outputs and determines whether the test passed or failed.

Performance testing is a type of software testing that ensures software applications to perform properly under their expected workload. It is a testing technique carried out to determine system performance in terms of speed, response time, stability, reliability, scalability, and resource usage of a software application under a certain workload .

There are different types of performance testing, such as load testing, stress testing, spike testing, endurance testing, and volume testing . Each type of performance testing has a different goal and scenario. For example, load testing measures system performance as the workload increases, while stress testing measures system performance when the workload exceeds the normal limits .

The basic architecture of a performance testing process can be illustrated by the following diagram:



The test scripts are the code or commands that simulate the user actions or requests to the software application. The test scenario is the set of conditions or parameters that define the test objectives, environment, and workload. The test execution is the process of running the test scripts according to the test scenario. The test results are the data or metrics collected during the test execution. The test analysis is the process of evaluating the test results and identifying the performance bottlenecks or issues .

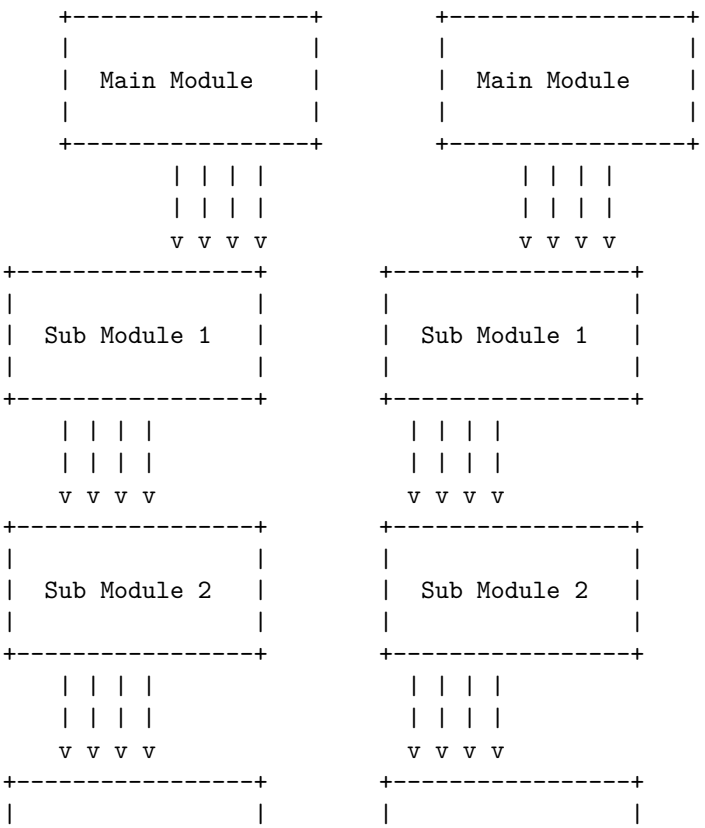
Top-down and bottom-up testing strategies are two types of integration testing techniques used to verify the functionality and interaction of different modules or components of a software system. Integration testing is the process of combining individual units of code and testing them as a group to ensure that they work together as expected.

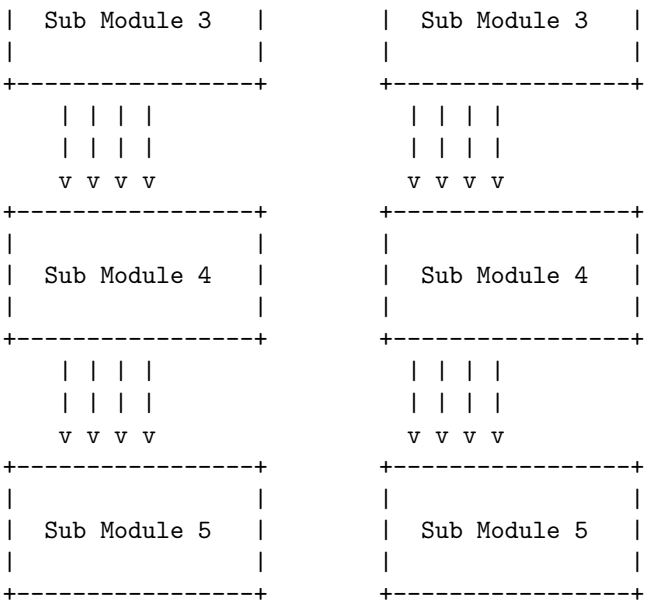
Top-down testing strategy starts with testing the higher-level modules first, and then gradually moving down to the lower-level modules. The lower-level modules are simulated by using stubs, which are dummy modules that mimic the behavior and interface of the real modules. The advantage of top-down testing is that it allows early detection of errors and inconsistencies in the main logic and functionality of the system. The disadvantage is that it requires a lot of stubs to be created and maintained, which can be time-consuming and complex.

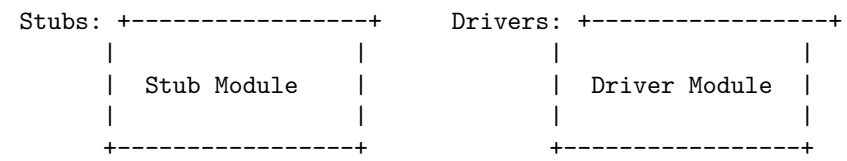
Bottom-up testing strategy starts with testing the lower-level modules first, and then gradually moving up to the higher-level modules. The higher-level modules are simulated by using drivers, which are test modules that provide input and output for the real modules. The advantage of bottom-up testing is that it allows early verification of the performance and reliability of the basic components of the system. The disadvantage is that it requires a lot of drivers to be created and maintained, which can also be time-consuming and complex.

The following diagram illustrates the basic architecture of a top-down and bottom-up testing strategy in software testing using ASCII characters:

Top-Down and Bottom-Up Testing Strategies in Software Testing

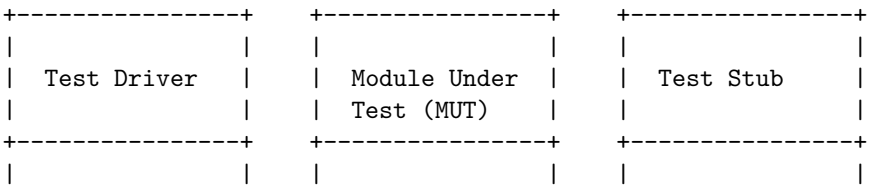


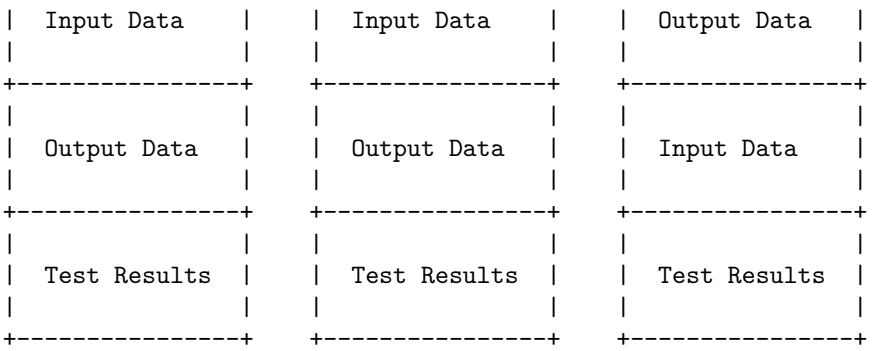




Test Drivers and Test Stubs are two types of test harnesses that are used to facilitate the integration testing of software modules. Test Drivers are programs that call the module to be tested and provide the input data, while Test Stubs are programs that are called by the module to be tested and provide the output data. Test Drivers are used in Bottom-up integration testing, where the lower level modules are tested first and then used to test the higher level modules. Test Stubs are used in Top-down integration testing, where the higher level modules are tested first and then the lower level modules are simulated by the Stubs.

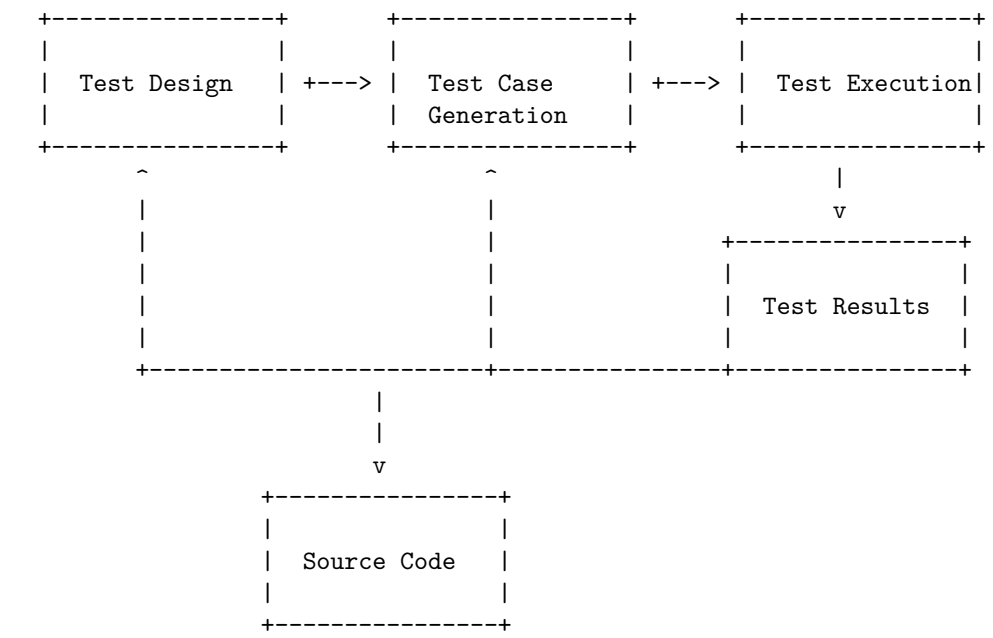
The following diagram illustrates the basic architecture of a Test Driver and a Test Stub in software testing strategy:





Structural testing, also known as white box testing, is a software testing strategy that tests the internal structure, design, and implementation of an application, using the knowledge of the source code and programming skills. Structural testing can be applied at different levels of testing, such as unit, integration, and system testing. The main objective of structural testing is to verify the code coverage, such as statements, branches, paths, and conditions.

The following diagram illustrates the basic steps of structural testing:



Structural testing involves the following techniques:

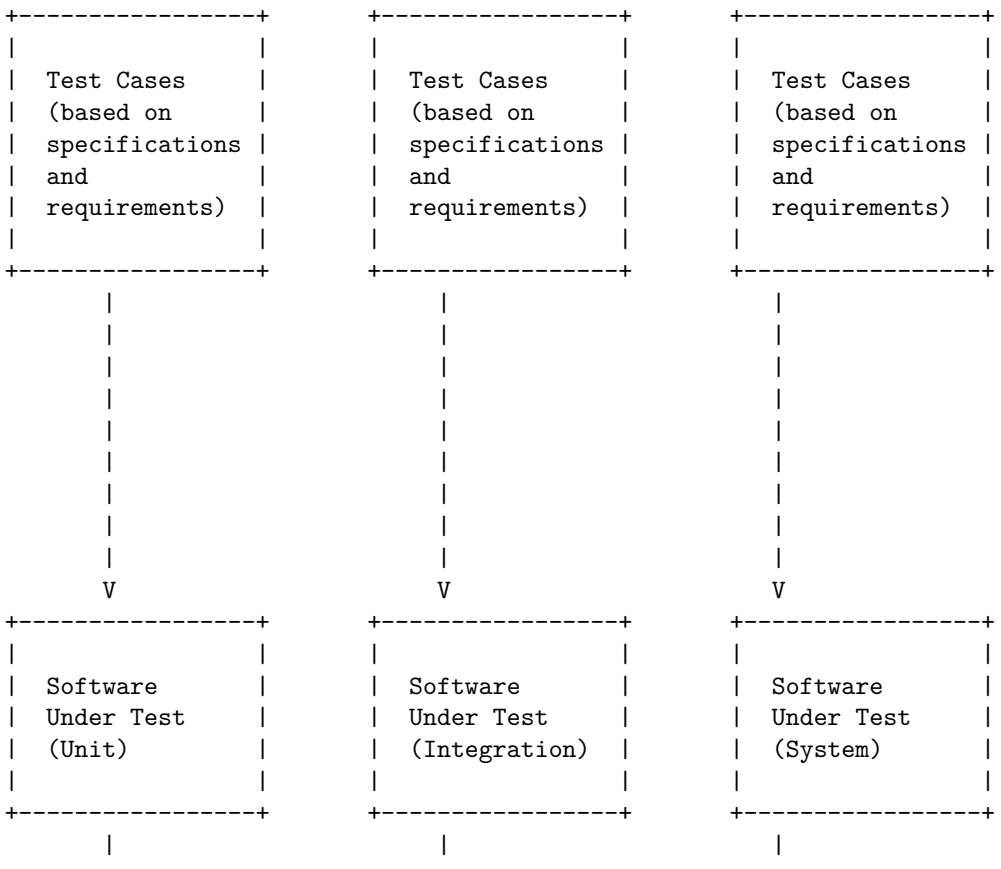
- Statement coverage: It measures the percentage of executable statements that are covered by the test cases.
- Branch coverage: It measures the percentage of decision outcomes (such as if-else, switch-case, etc.) that are covered by the test cases.

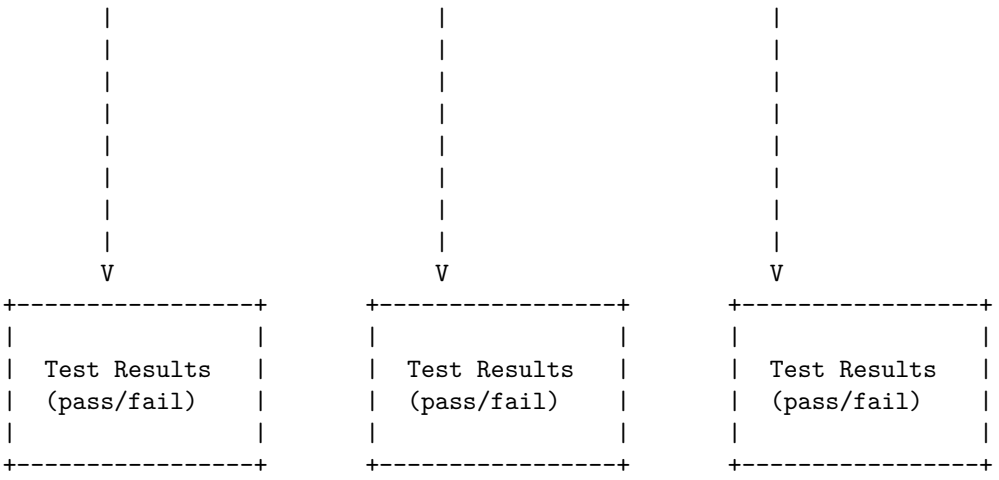
- Path coverage: It measures the percentage of possible paths (from entry to exit) that are covered by the test cases.
- Condition coverage: It measures the percentage of logical conditions (such as AND, OR, etc.) that are evaluated to both true and false by the test cases.

Functional Testing (Black Box Testing) is a software testing strategy that evaluates the functionality of the software under test without looking at the internal code structure, implementation details, or internal paths. It is based on the software specifications and requirements, and it can be applied to different levels of testing, such as unit, integration, system, and acceptance testing.

Functional Testing (Black Box Testing) can be performed using various techniques, such as equivalence partitioning, boundary value analysis, decision table testing, state transition testing, use case testing, etc. These techniques help to design test cases that cover the expected inputs, outputs, and behaviors of the software under test.

The following diagram illustrates the basic architecture of a Functional Testing (Black Box Testing) software testing strategy using ASCII characters:

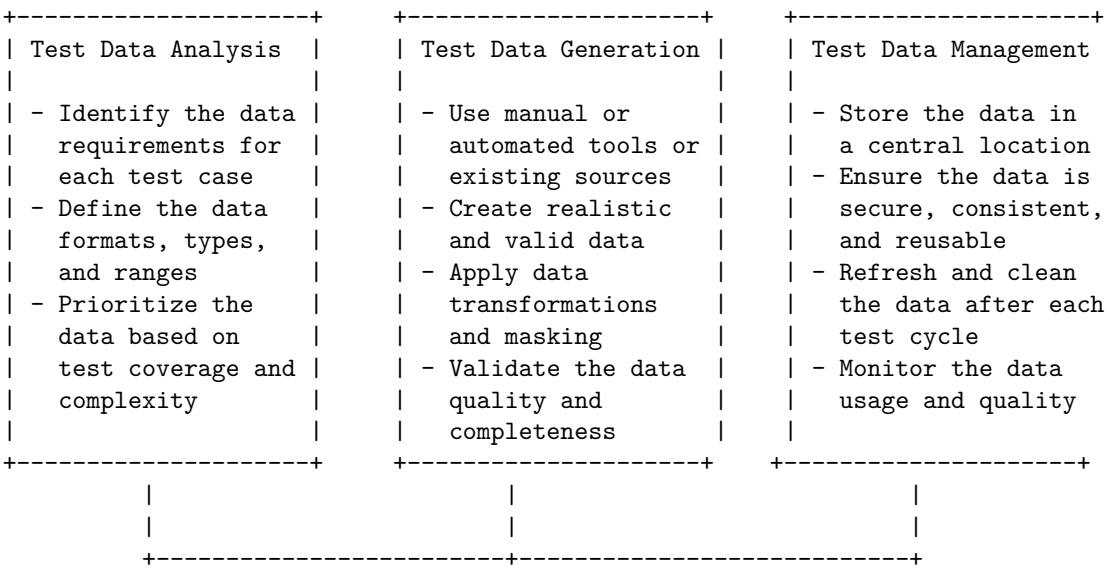


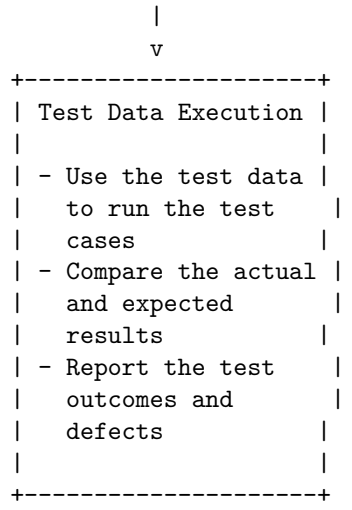


Test Data Suit Preparation is a process of creating and maintaining test data for software testing. Test data is the input that is used to execute the test cases and verify the expected output. Test data can be generated manually, automatically, or from existing sources such as production databases or files.

A software testing strategy is a high-level plan that defines the approach, scope, and objectives of software testing. It guides the testing team to select the appropriate test techniques, tools, and methods to achieve the desired quality and coverage. A software testing strategy also outlines the roles and responsibilities of the testing team, the test environment, the test schedule, and the test deliverables.

A possible diagram for Test Data Suit Preparation software testing strategy is:





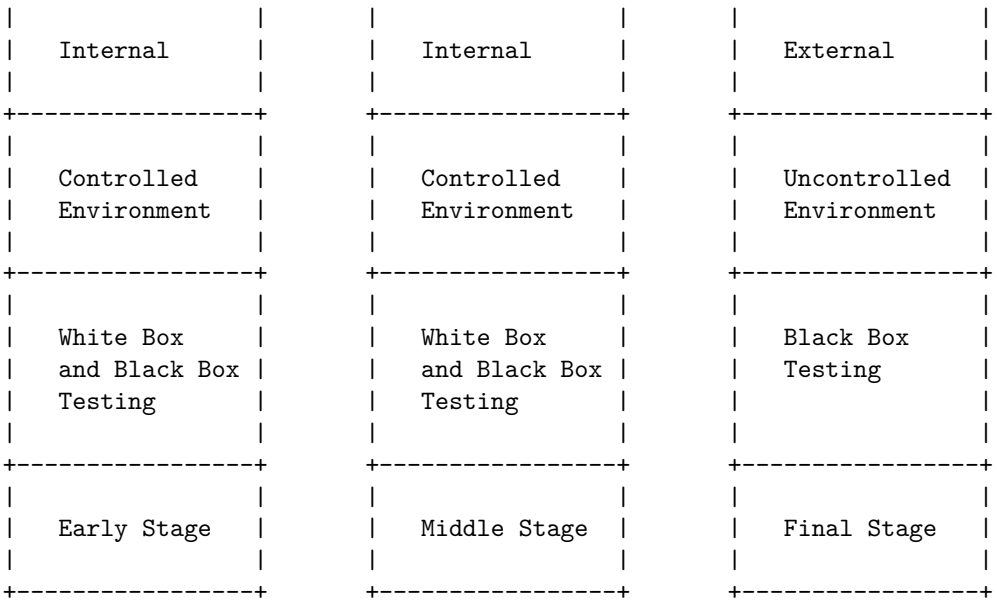
Alpha and Beta Testing of Products software testing strategy

Alpha and beta testing are two types of user acceptance testing methodologies that help to build confidence in launching a product successfully. Both testing types rely on different goals, strategies, and processes.

Alpha testing is focused on identifying bugs and validating that the software is functioning as a user would expect it to. During alpha testing, the product is tested in a testing or staging environment using white box and black box practices. Alpha testing is conducted by internal employees of the organization.

Beta testing is a type of external user acceptance testing and is the final round of testing performed before a product is finally released to the wider audience. In this testing, a nearly completed (90-95%) version of the software, known as the beta version is released to a limited number of end-users for testing. Beta testing is aimed at testing the user behavior, edge use cases, and compatibility with different platforms and also to spread product awareness. Unlike alpha testing, beta testing involves real users.

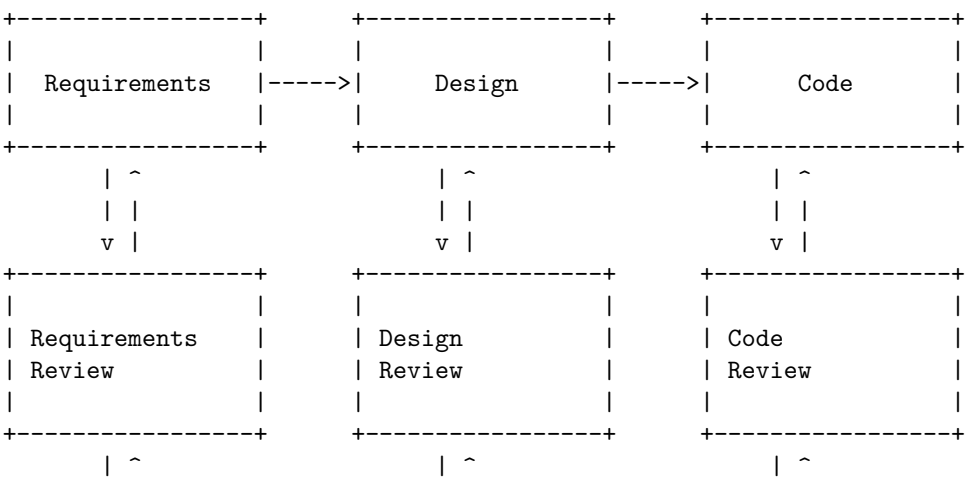
The following diagram illustrates the basic architecture of a alpha and beta testing of products software testing strategy using ASCII characters:

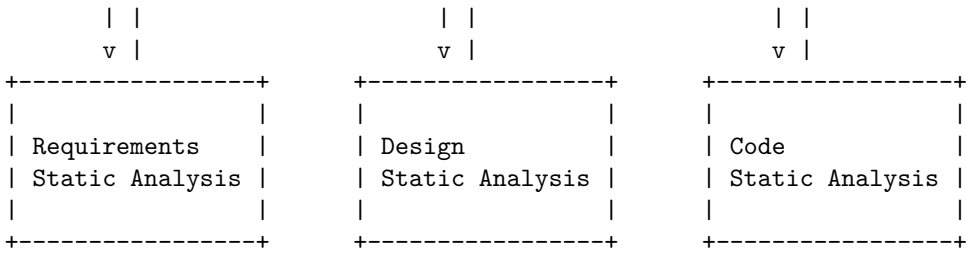


Static testing is a software testing technique that checks the defects in software without executing the code. It can be done in two ways: review and static analysis. Review is a manual process of finding and removing errors and ambiguities in the supporting documents, such as requirements, design and test cases. Static analysis is an automated process of finding and removing errors and anomalies in the code, such as syntax, logic and complexity.

Static Testing Strategies in Software Testing

The following diagram illustrates the basic architecture of a static testing process:

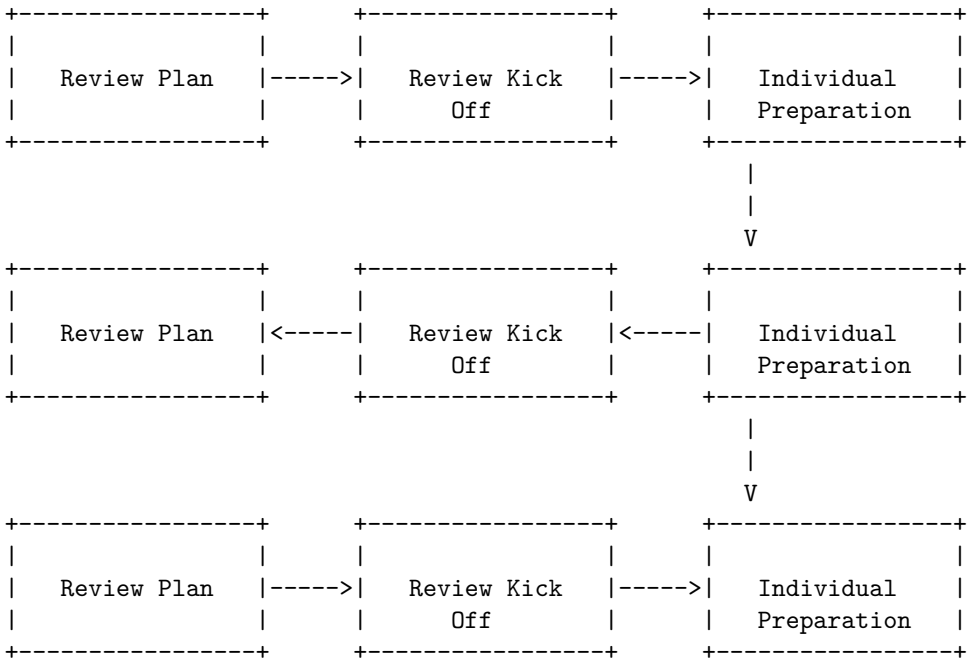


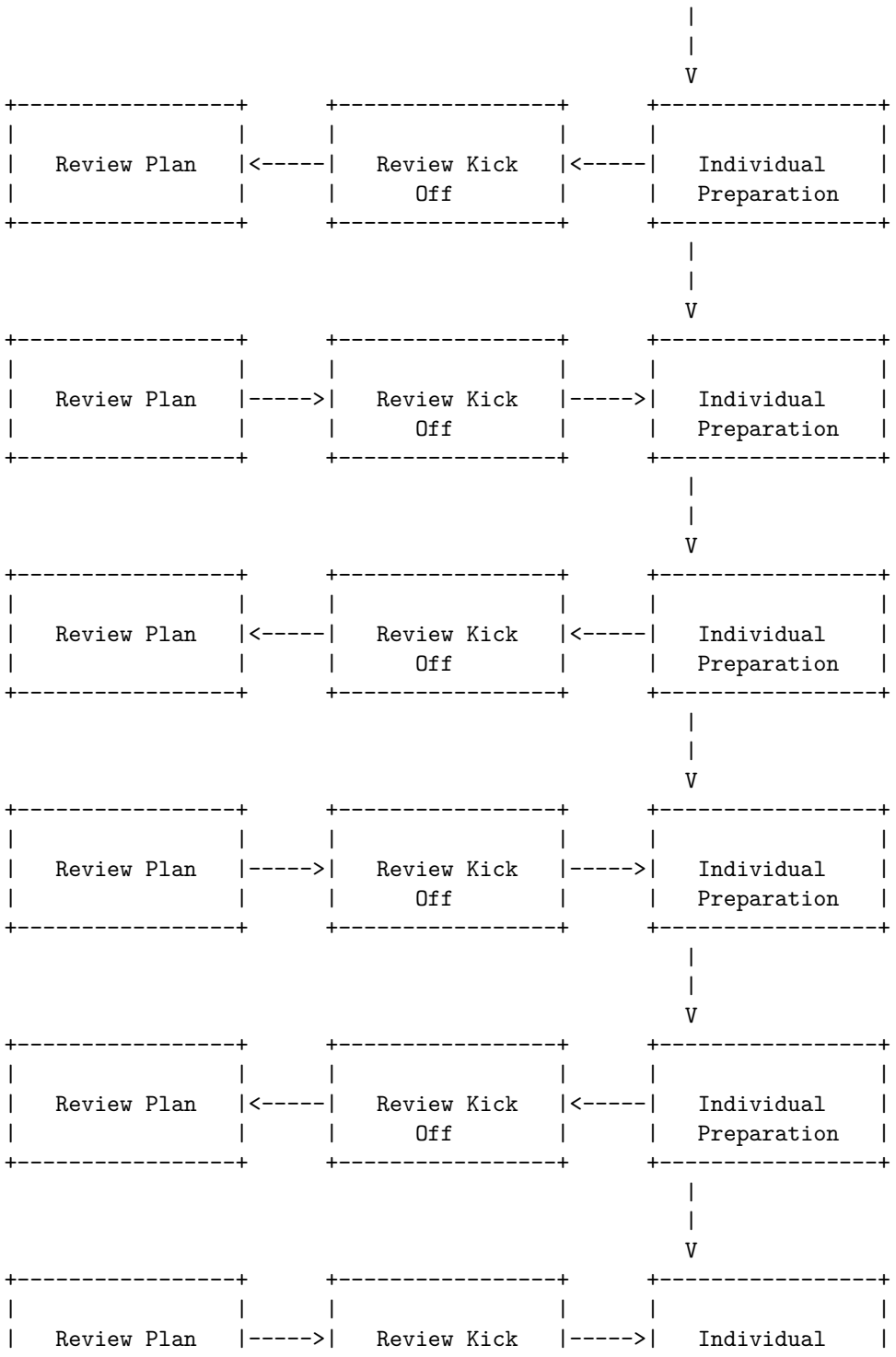


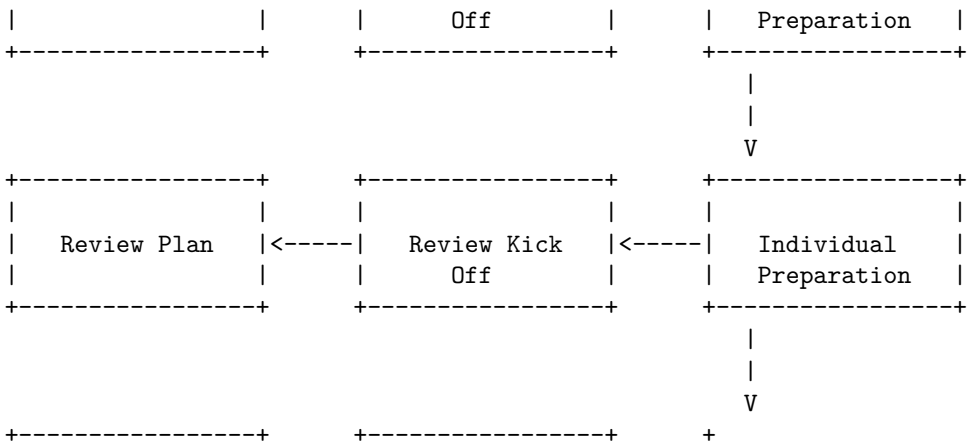
The diagram shows that static testing can be applied at different stages of the software development life cycle, from requirements to code. The output of each stage is the input of the next stage, and the feedback of each stage is the input of the previous stage. The goal of static testing is to ensure the quality and consistency of the software artifacts and to detect and correct the defects as early as possible.

Formal Technical Reviews (Peer Reviews) are a type of static testing technique that involves manual examination of software artifacts such as requirements, design, code, etc. by a team of peers and technical specialists. The main objective of FTR is to find and eliminate defects, issues, and ambiguities in the software before it is executed. FTR also helps to improve the quality of the software, verify its compliance with standards and specifications, and enhance the skills and knowledge of the reviewers.

The following diagram illustrates the basic architecture of a Formal Technical Review process using ASCII characters:

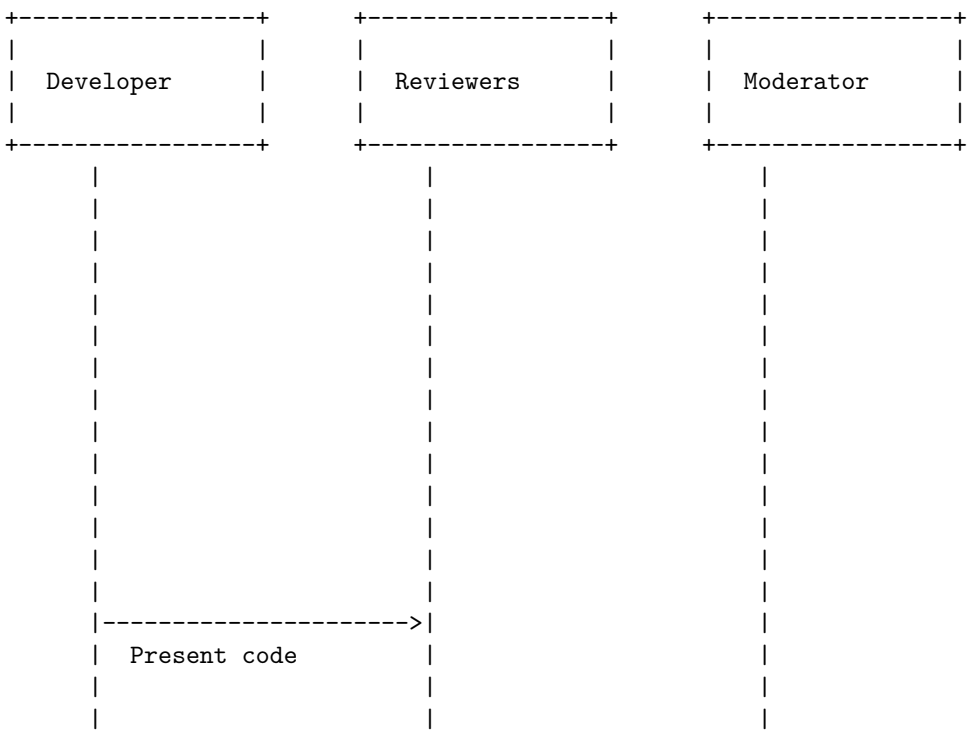


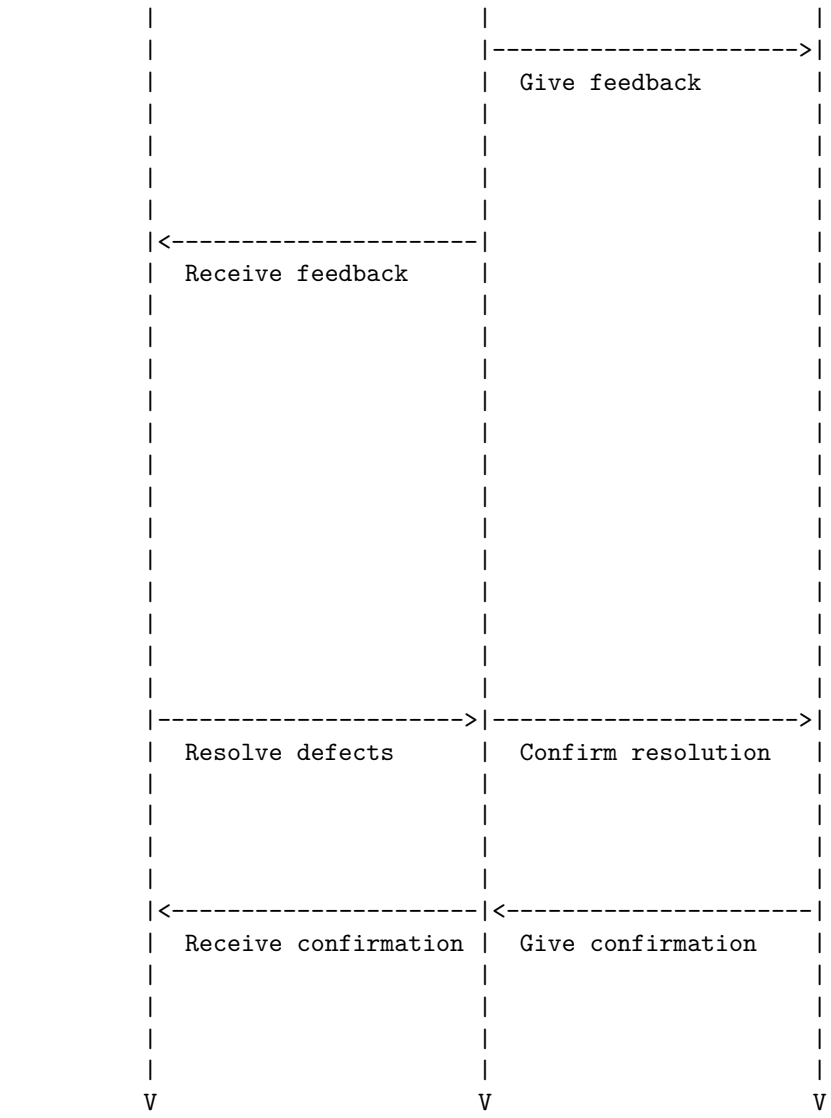

Review Plan
Review Kick Off
Individual Preparation
Review Plan
Review Kick Off
Individual Preparation
Review Plan
Review Kick Off
Individual Preparation
Review Plan
Review Kick Off
Individual Preparation
Review Plan
Review Kick
Individual



A walk through (walkthrough) is a static testing technique where the developer presents the code to others, who then give their opinions and feedback. It is a way of checking the quality and correctness of the code, as well as finding defects and errors. A walk through can also help the developer to realize problems themselves during the presentation.

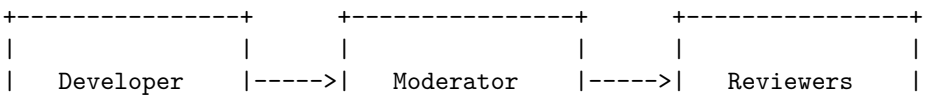
The following diagram illustrates the basic architecture of a walk through static testing strategy using ASCII characters:

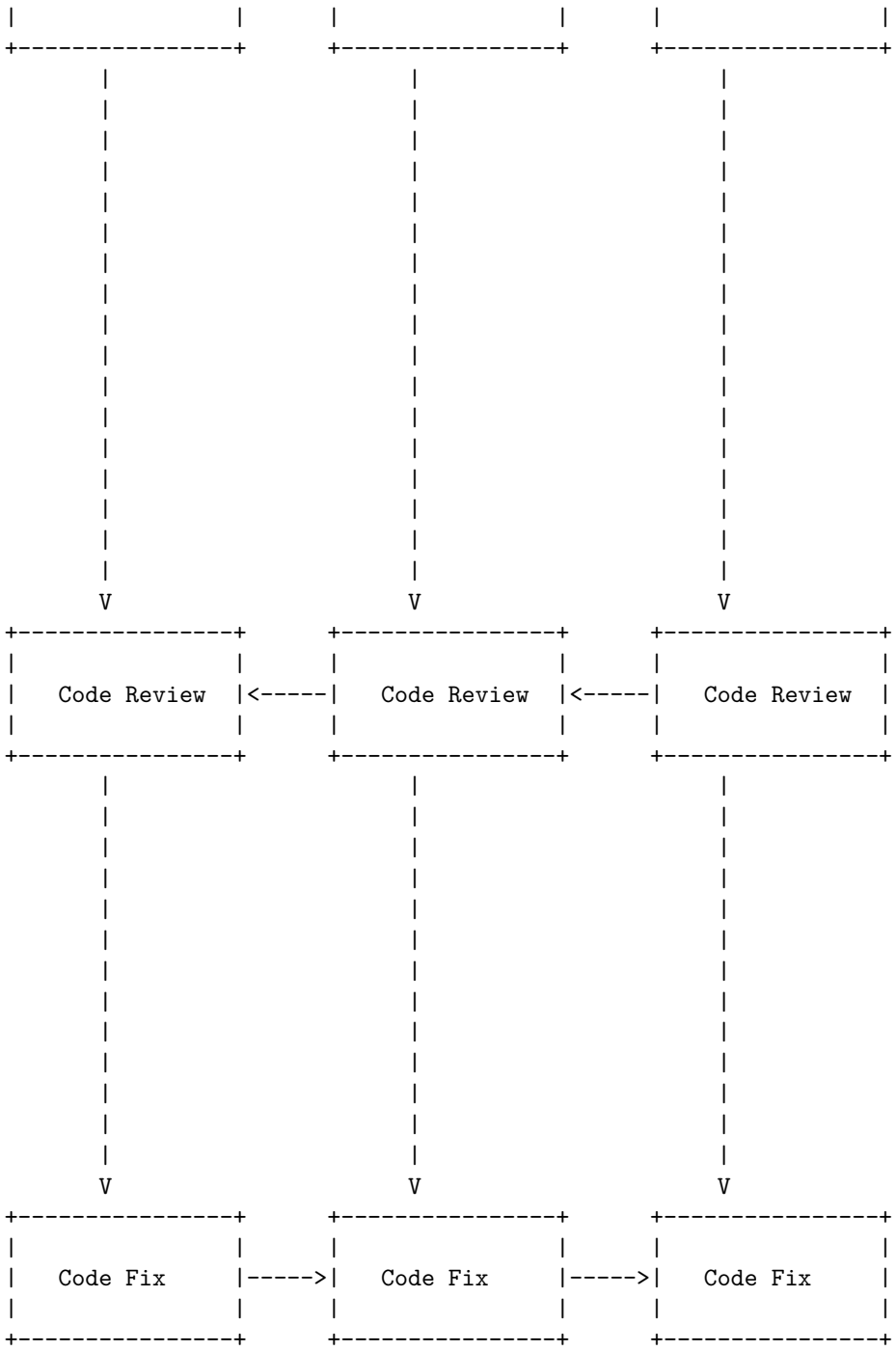




Code inspection is a type of static testing which aims in reviewing the software code and examining for any errors in that. It helps in reducing the ratio of defect multiplication and avoids later-stage error detection by simplifying all the initial error detection processes.

The following diagram illustrates the basic architecture of a code inspection process using ASCII art:



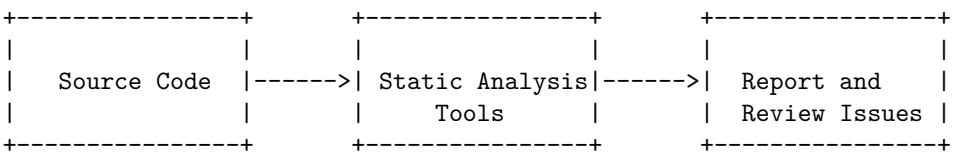


The code inspection process involves the following steps:

- The developer writes the code and submits it to the moderator for review.
- The moderator checks the code for compliance with the coding standards and guidelines, and assigns reviewers for further inspection.
- The reviewers examine the code for any logical, syntactical, or functional errors, and report their findings to the moderator.
- The moderator consolidates the feedback from the reviewers and sends it back to the developer.
- The developer fixes the code based on the feedback and resubmits it to the moderator for verification.
- The moderator verifies that the code is free of errors and approves it for deployment.

Compliance with Design and Coding Standards (Coding Standards) Static testing strategy is a process of verifying the quality, security, and compliance of the source code by using static analysis tools. Static analysis tools check the code against predefined rules and guidelines, such as MISRA, CERT, or ISO 26262, and report any violations or defects. Static analysis can be performed at any stage of the software development life cycle, but it is recommended to perform it as early as possible, preferably during the coding phase, to reduce the cost and effort of fixing the issues later.

The following diagram illustrates the basic architecture of a Compliance with Design and Coding Standards (Coding Standards) Static testing strategy:



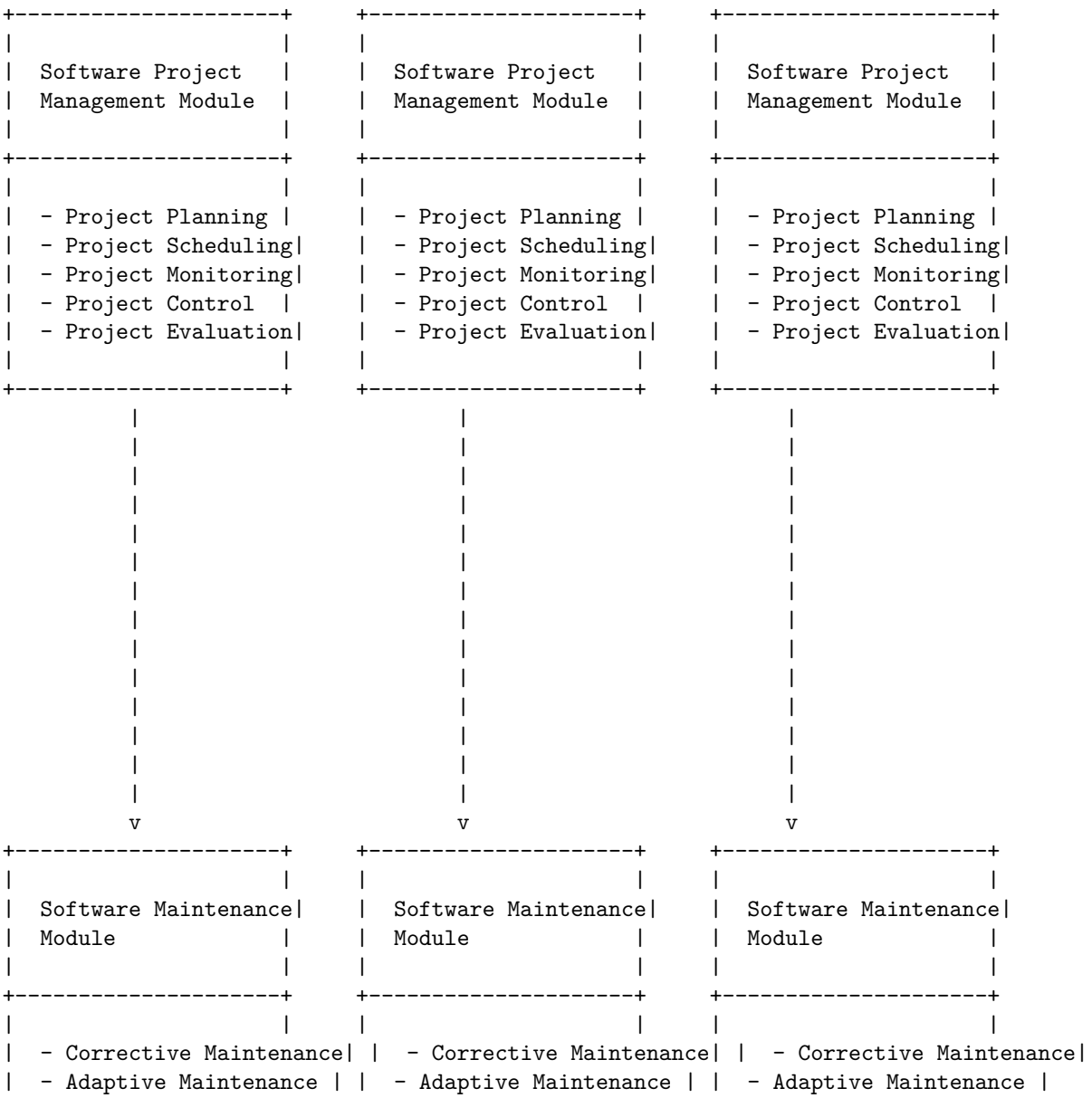
The diagram shows the following steps:

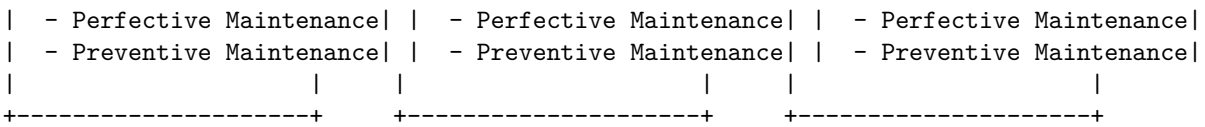
- The source code is the input for the static analysis tools. The source code can be written in any programming language, such as C, C++, Java, Python, etc.
- The static analysis tools scan the source code and check it against the predefined rules and guidelines. The tools can be configured to enforce different levels of compliance, such as mandatory, required, or advisory. The tools can also be integrated with the development environment, such as IDEs, code editors, or version control systems, to provide real-time feedback and suggestions to the developers.
- The report and review issues step is the output of the static analysis tools. The report contains the list of issues found by the tools, such as defects, vulnerabilities, or compliance violations. The report also provides the severity, priority, and location of each issue, as well as the suggested fix or mitigation. The report can be viewed in various formats, such as HTML, XML, PDF, etc. The report can also be exported to other tools, such as bug tracking systems, code review tools, or quality management systems,

to facilitate the resolution and verification of the issues.

Unit 5 - Software Maintenance and Software Project Management

The following diagram illustrates the basic architecture of a software maintenance and software project management system:





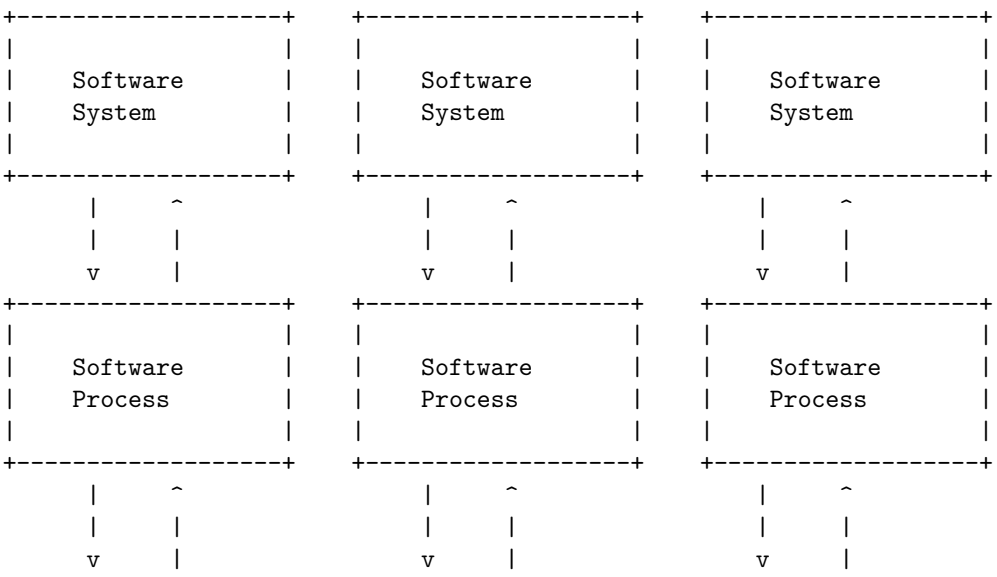
The software project management module is responsible for planning, scheduling, monitoring, controlling and evaluating the software project. It helps to define the scope, objectives, tasks, resources, budget, schedule, quality and risks of the project. It also helps to track the progress, performance, issues and changes of the project.

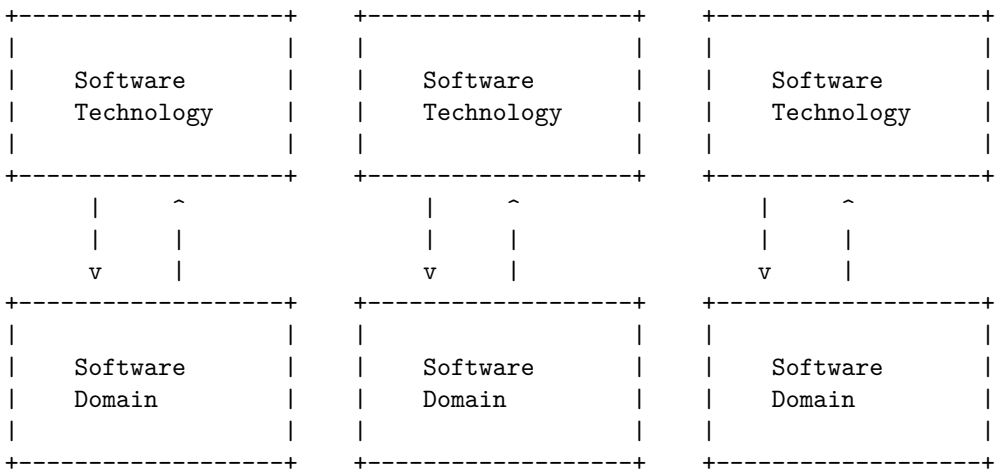
The software maintenance module is responsible for modifying the software after delivery to correct faults, improve performance, adapt to changing environments, or enhance functionality. It helps to identify, analyze, implement and test the changes in the software. It also helps to document and communicate the changes to the stakeholders.

Software as an Evolutionary Entity is a concept that describes how software systems change and adapt over time due to various factors, such as changing requirements, technologies, stakeholder knowledge, and environmental conditions. Software evolution is a continuous process that involves developing, maintaining, and updating software systems to keep them consistent, reliable, and useful. Software evolution also affects the domains that co-evolve with the software, such as the users, the developers, the processes, and the tools.

Software as an Evolutionary Entity

The following diagram illustrates the basic architecture of a software system as an evolutionary entity, using ASCII characters:





The diagram shows four layers of a software system as an evolutionary entity: the software system itself, the software process, the software technology, and the software domain. Each layer interacts with the other layers through feedback loops, which can be positive or negative, depending on the nature and direction of the change. The feedback loops can also span across different time scales, from short-term to long-term.

The software system layer represents the actual product or service that is delivered to the users or customers. It consists of the software components, the data, the interfaces, and the functionality that provide the desired value and quality. The software system layer evolves as a result of changing requirements, user feedback, bug fixes, enhancements, and maintenance activities.

The software process layer represents the activities, methods, models, standards, and tools that are used to develop, maintain, and update the software system. It consists of the software life cycle phases, such as planning, analysis, design, implementation, testing, deployment, and operation. The software process layer evolves as a result of changing technologies, best practices, regulations, and organizational goals.

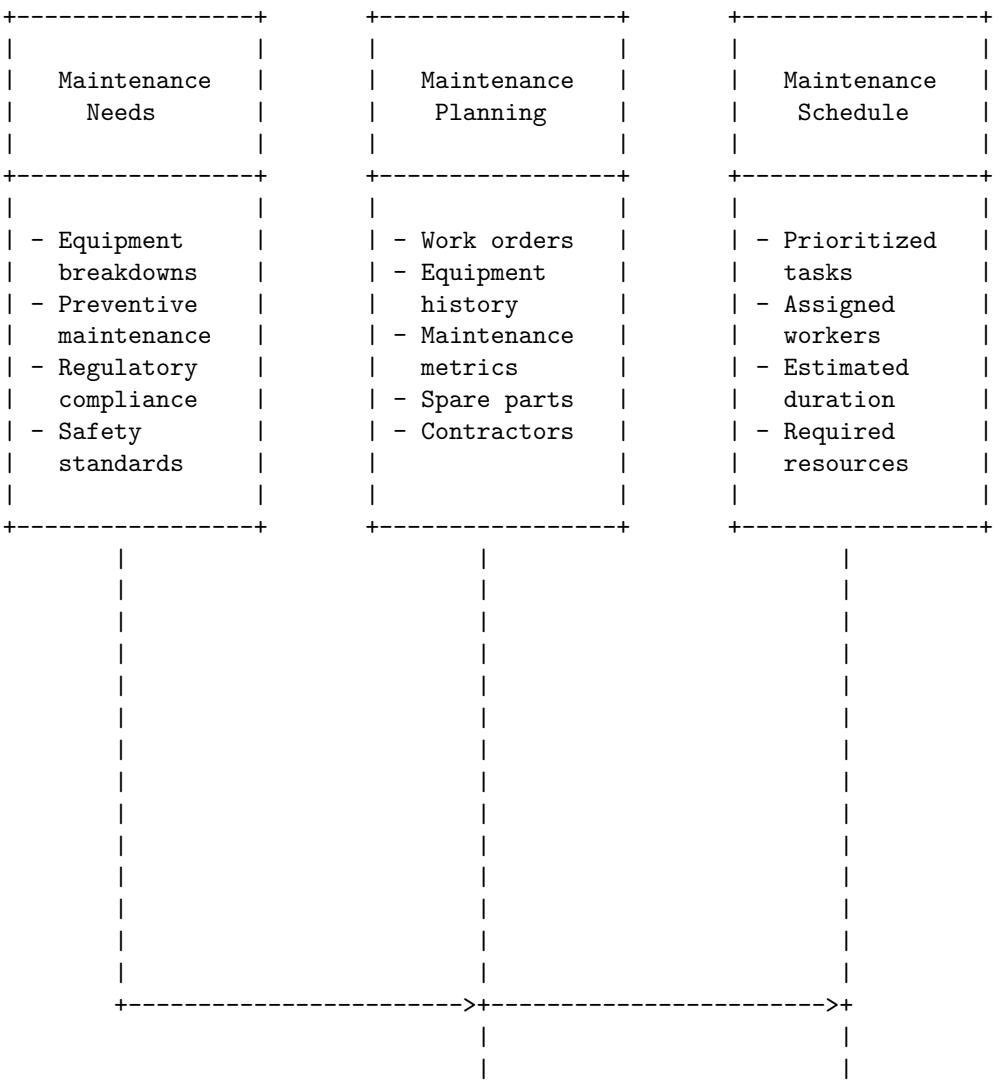
The software technology layer represents the hardware, software, and network infrastructure that support the software system and the software process. It consists of the devices, platforms, frameworks, libraries, languages, and protocols that enable the creation, execution, and communication of the software system. The software technology layer evolves as a result of innovation, obsolescence, compatibility, and performance issues.

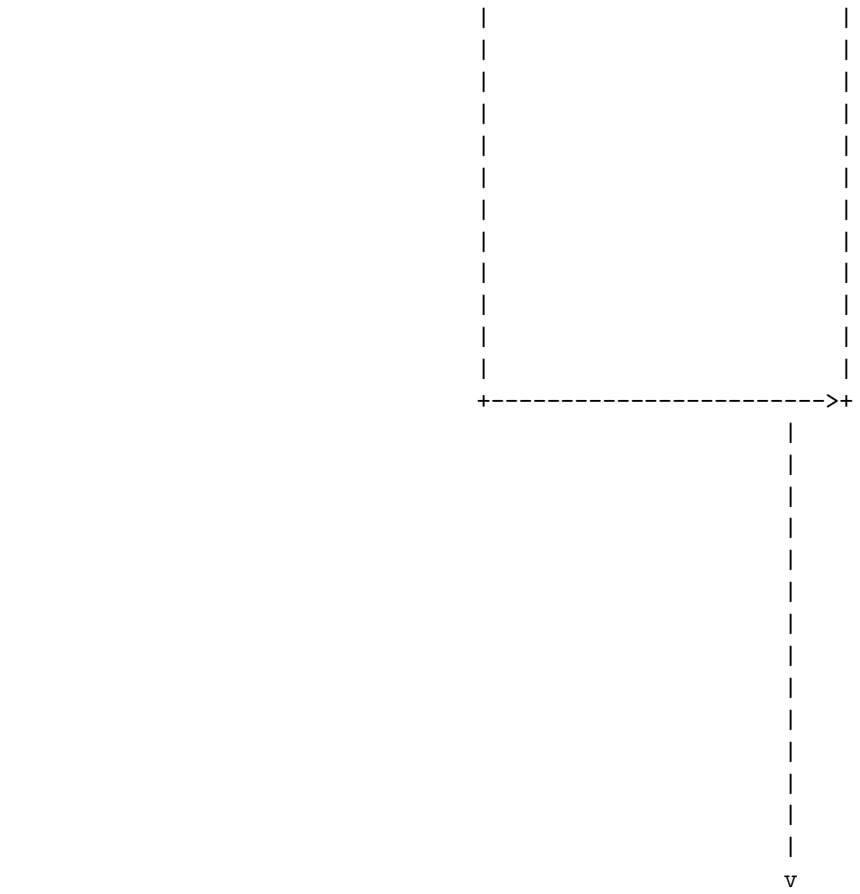
The software domain layer represents the context, environment, and stakeholders that influence and are influenced by the software system. It consists of the users, customers, competitors, regulators, suppliers, and partners that have a stake in the software system. The software domain layer evolves as a result of changing needs, preferences,

The need for maintenance and maintenance planning is to ensure the optimal performance, reliability, and safety of the equipment and assets used for operation. Maintenance planning is the process of identifying, scheduling, and allocating the necessary resources (such as labor, materials, spare parts, and contractors) for the maintenance work activities. Maintenance planning also helps to avoid time-wasting delays, reduce costs, and increase productivity.

The following is a detailed ASCII diagram for the need for maintenance and maintenance planning:

Need for Maintenance and Maintenance Planning



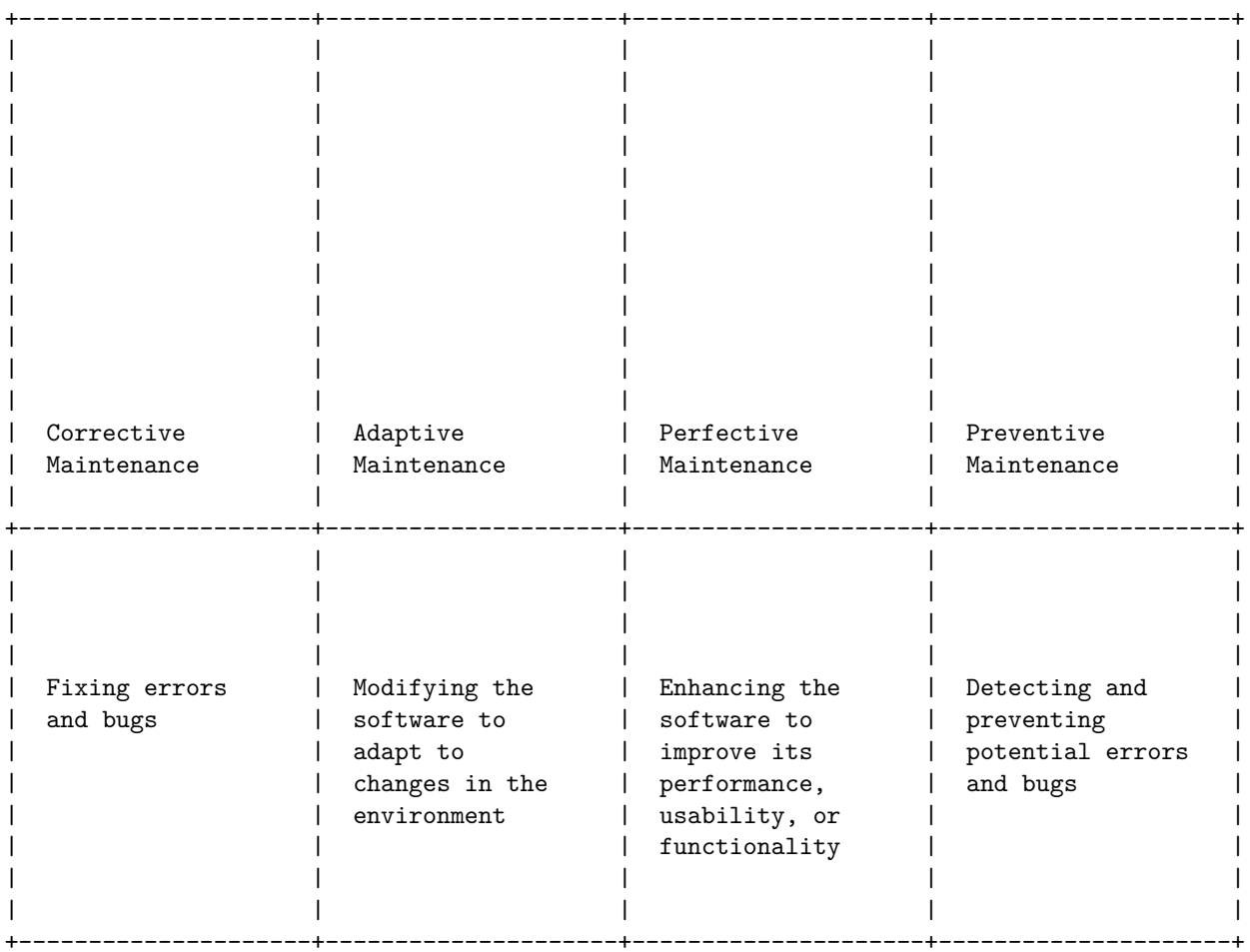


```
+-----------------+
|                 |
|   Maintenance   |
|     Work        |
|                 |
+-----------------+
|                 |
| - Execution     |
| - Inspection    |
| - Testing       |
| - Reporting     |
| - Feedback      |
|                 |
+-----------------+
```

Categories of Maintenance of Software are the types of activities that are performed to keep the software system functional, reliable, and up-to-date. According to various sources , there are four main categories of maintenance, namely:

- **Corrective maintenance**: This involves fixing errors and bugs in the software system that are observed when the software is in use or reported by the users.
- **Adaptive maintenance**: This involves modifying the software system to adapt it to changes in the environment, such as changes in hardware, software, or user requirements.
- **Perfective maintenance**: This involves enhancing the software system to improve its performance, usability, or functionality, such as adding new features, improving user interface, or optimizing code.
- **Preventive maintenance**: This involves detecting and preventing potential errors and bugs in the software system before they occur, such as refactoring code, updating documentation, or conducting code reviews.

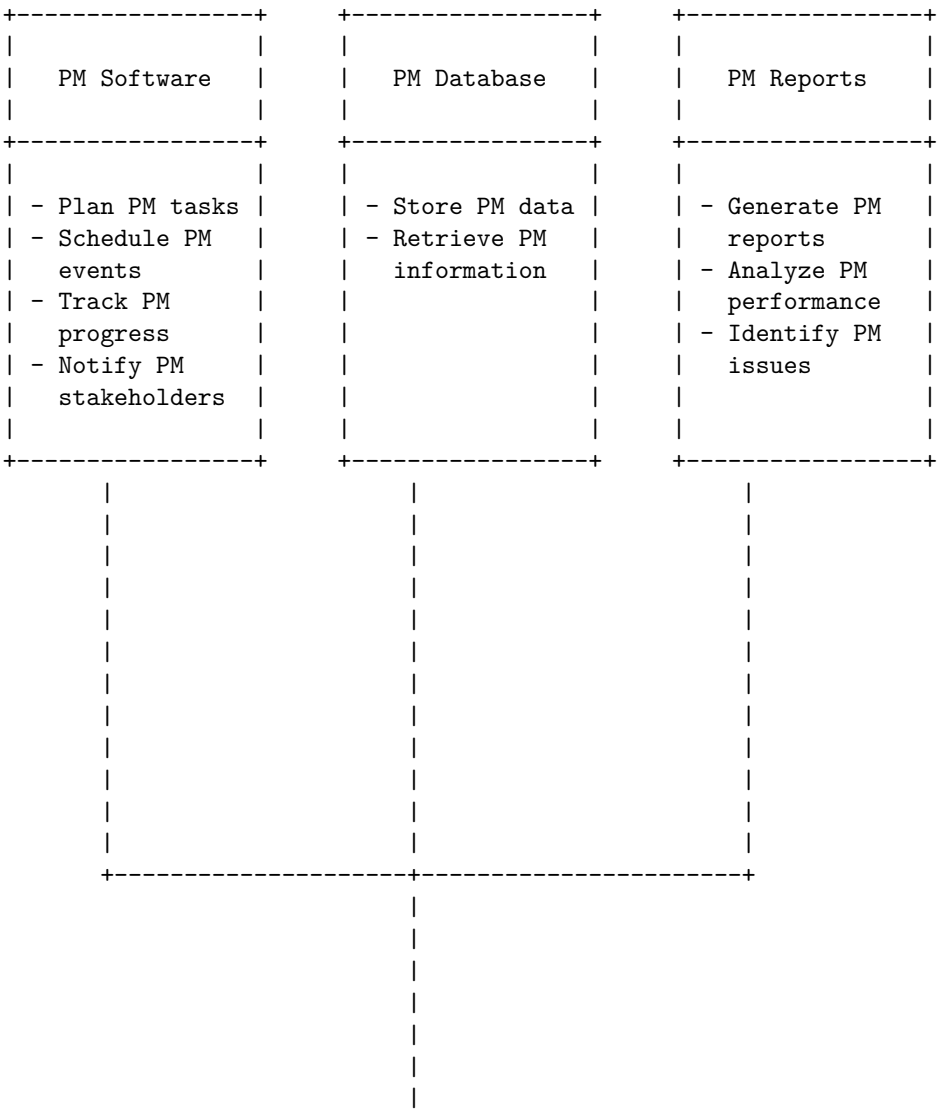
The following diagram illustrates the categories of maintenance of software using ASCII characters:



Preventive maintenance (PM) of software is the process of performing regular checks and updates on software systems to prevent failures and improve performance. PM software is a type of computerized maintenance management software (CMMS) that helps with planning, scheduling, tracking and reporting of PM activities. PM software can help lower operating costs, extend asset life spans, reduce downtime and increase efficiency.

Preventive Maintenance (PM) of Software

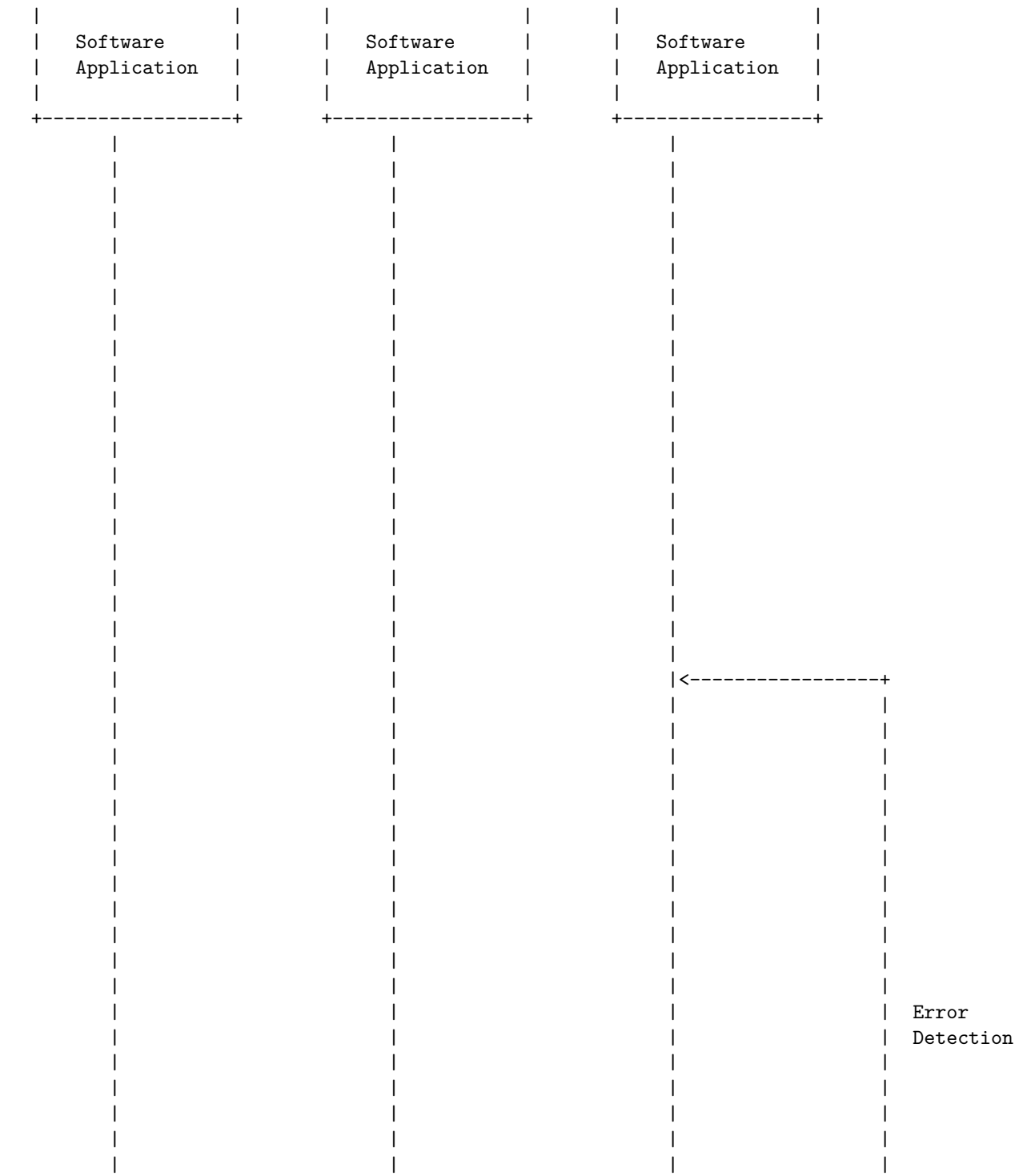
The following diagram illustrates the basic architecture of a PM software:

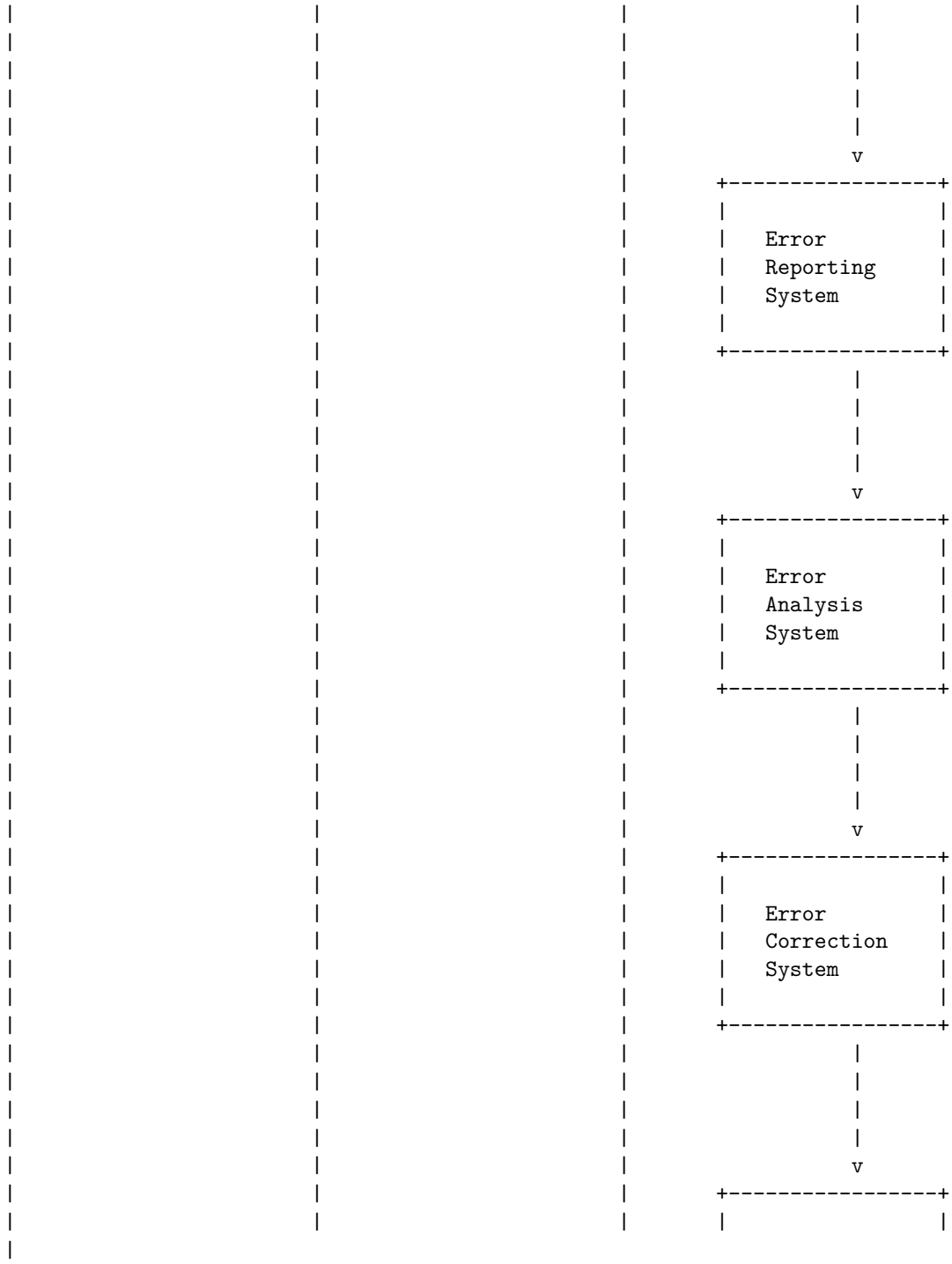


Corrective Maintenance (CM) of Software is the process of fixing errors and faults in software applications that affect its functionality, design, logic, or code . It is usually performed after the software is in use and the errors are detected by the users or the developers. Corrective Maintenance can be either reactive or proactive, depending on whether the errors are fixed before or after they cause failures. Corrective Maintenance can also involve either repair or replacement of the faulty software components.

Corrective Maintenance (CM) of Software

+-----------------+ +-----------------+ +-----------------+

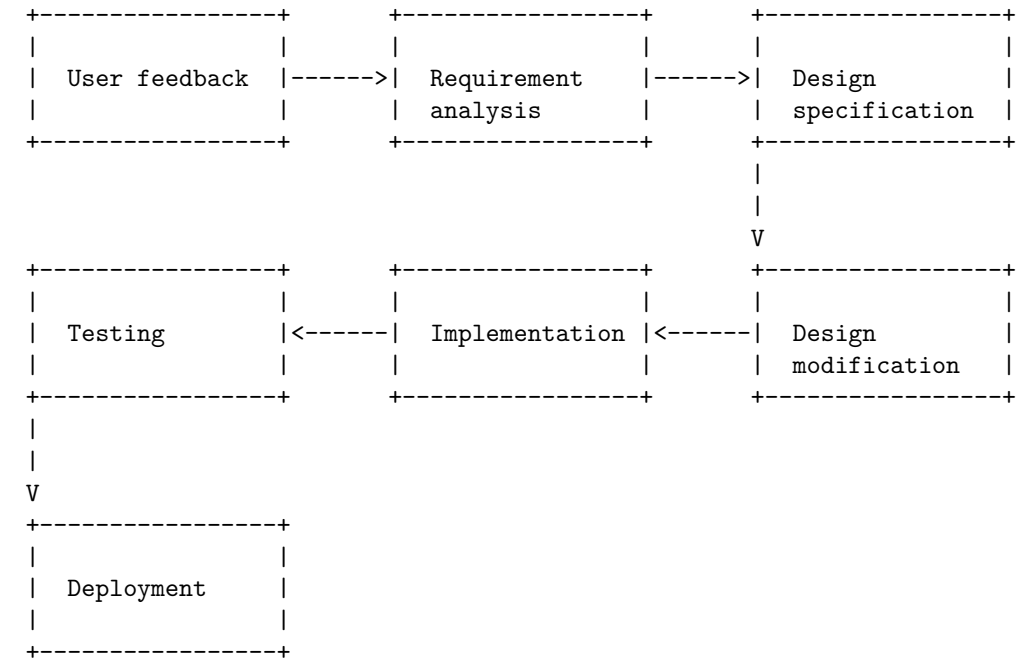

Software Application
Software Application
Software Application
Error Detection


Error
Reporting
System
Error
Analysis
System
Error
Correction
System

Perfective maintenance of software is the process of modifying software or applications to implement new or changed user requirements which concern functional enhancements. It includes adding, removing, or modifying features to keep the software usable, reliable, and relevant over a long period of time .

Perfective Maintenance (PM) of Software

The following diagram illustrates the basic steps of perfective maintenance of software using ASCII characters:



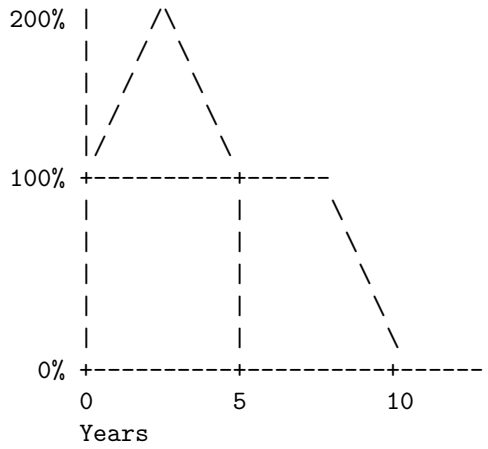
The cost of maintenance of software is the amount of money spent on keeping the software functional, secure, and up-to-date after its initial development and deployment. It includes activities such as bug fixing, performance optimization, security patching, feature enhancement, and documentation update.

The cost of maintenance of software depends on various factors, such as the type of software, the number of users, the complexity of the code, the quality of the documentation, the frequency of changes, the availability of skilled developers, and the level of support required.

According to some estimates, software maintenance and support costs are around 15–20% of the initial development costs (per year), and in total (during the entire software life cycle) they can be as high as 90% of the total cost of ownership (TCO). Monthly software maintenance costs can range from $5,000 to $50,000+, depending on the app type and the required maintenance activities .

Cost of Maintenance of Software

The following diagram illustrates the cost of maintenance of software as a percentage of the initial development cost over time, based on the assumption that the software has a life span of 10 years and the maintenance cost is 20% of the initial development cost per year.



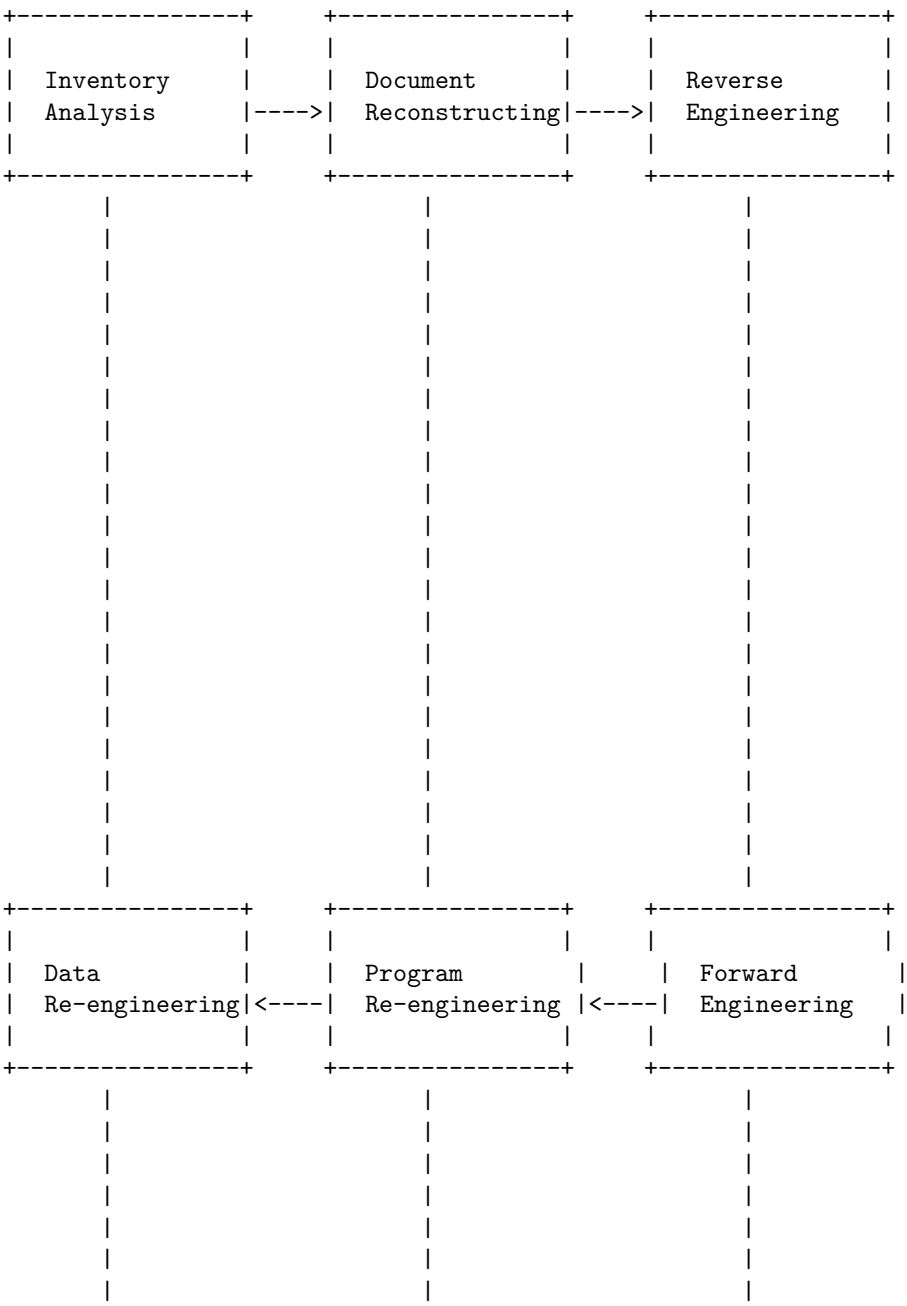
The diagram shows that the cost of maintenance of software increases over time, and eventually surpasses the initial development cost. This means that maintaining software can be more expensive than developing it in the long run. Therefore, it is important to optimize the software maintenance process and reduce the maintenance costs as much as possible. Some of the ways to do that are:

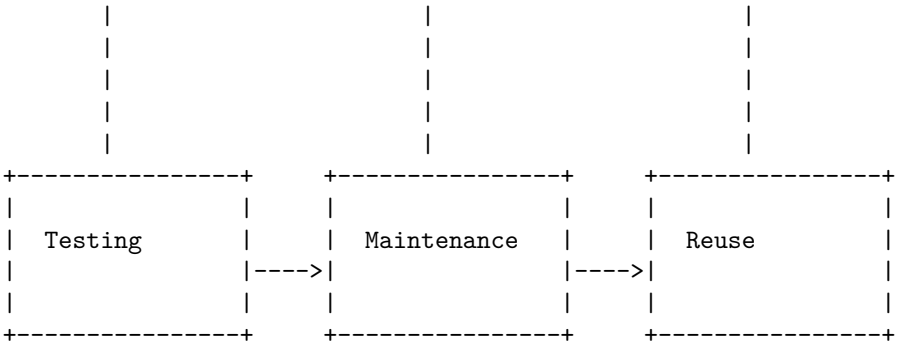
- Use high-quality code and documentation standards to minimize errors and ambiguities.
- Adopt agile methodologies and DevOps practices to deliver frequent and incremental updates.
- Implement automated testing and deployment tools to ensure quality and efficiency.
- Hire skilled and experienced developers who can handle complex and changing requirements.
- Outsource software maintenance and support to a reliable and cost-effective service provider.

Software re-engineering is a process of software development which is done to improve the maintainability of a software system. It involves examining and altering an existing system to reconstitute it in a new form. It encompasses a combination of sub-processes such as reverse engineering, forward engineering, reconstructing, etc.

Software Re- Engineering (SR) of Software

The following diagram illustrates the basic architecture of a software re-engineering process:





The main steps of the software re-engineering process are:

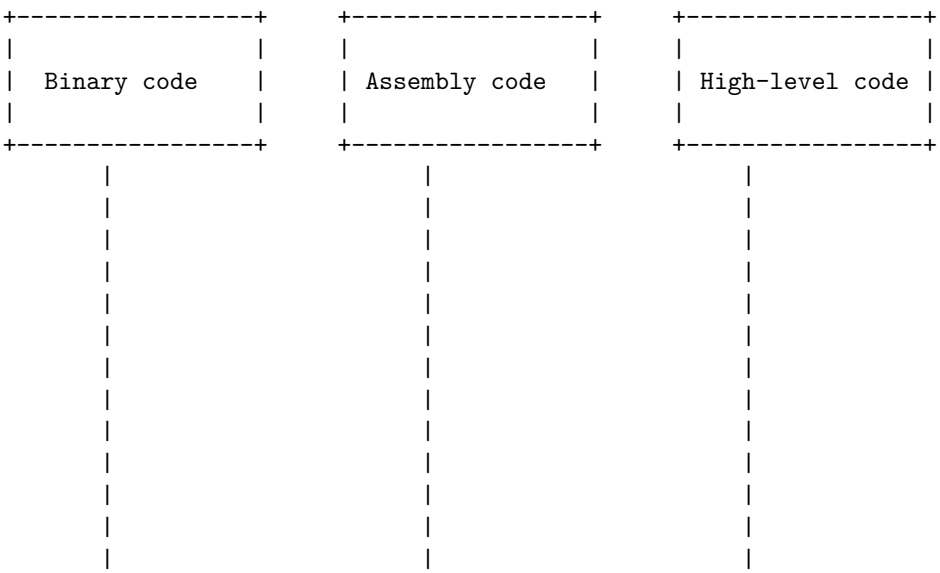
- Inventory analysis: This step involves identifying and evaluating all the applications in the software organization's inventory. The goal is to determine which applications are candidates for re-engineering, which ones are obsolete, and which ones are still useful.
- Document reconstructing: This step involves recovering and updating the documentation of the selected applications. Documentation can include user manuals, design specifications, test cases, etc. The goal is to provide accurate and consistent information about the system's functionality and operation.
- Reverse engineering: This step involves extracting the design and structure of the system from its source code and data. The goal is to understand the system's architecture, components, dependencies, interfaces, etc. Reverse engineering can use various techniques such as static analysis, dynamic analysis, data flow analysis, etc.
- Program re-engineering: This step involves modifying the source code of the system to improve its quality, performance, and maintainability. Program re-engineering can use various techniques such as restructuring, refactoring, modularization, etc. The goal is to make the code more readable, understandable, and reusable.
- Data re-engineering: This step involves modifying the data structures and schemas of the system to improve their efficiency, consistency, and integrity. Data re-engineering can use various techniques such as normalization, denormalization, migration, etc. The goal is to make the data more suitable for the system's requirements and operations.
- Forward engineering: This step involves generating a new system from the modified source code and data. The goal is to produce a system that meets the current and future needs of the users and stakeholders. Forward engineering can use various techniques such as code generation, compilation, linking, etc.
- Testing: This step involves verifying and validating the functionality and quality of the new system. Testing can use various techniques such as unit testing, integration testing, system testing, etc. The goal is to ensure that the system meets the specifications and expectations of the users and

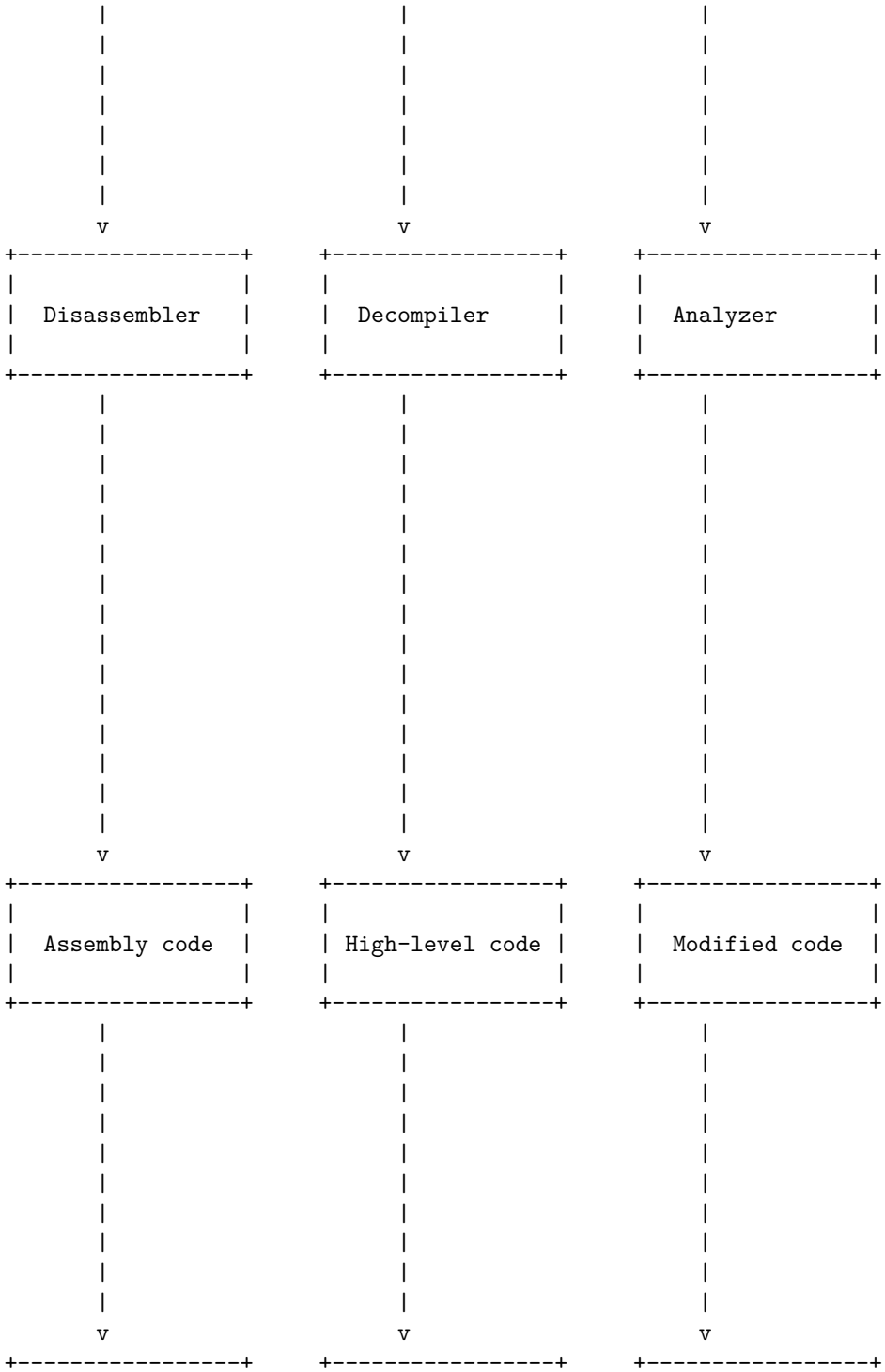
stakeholders.
- Maintenance: This step involves providing ongoing support and improvement

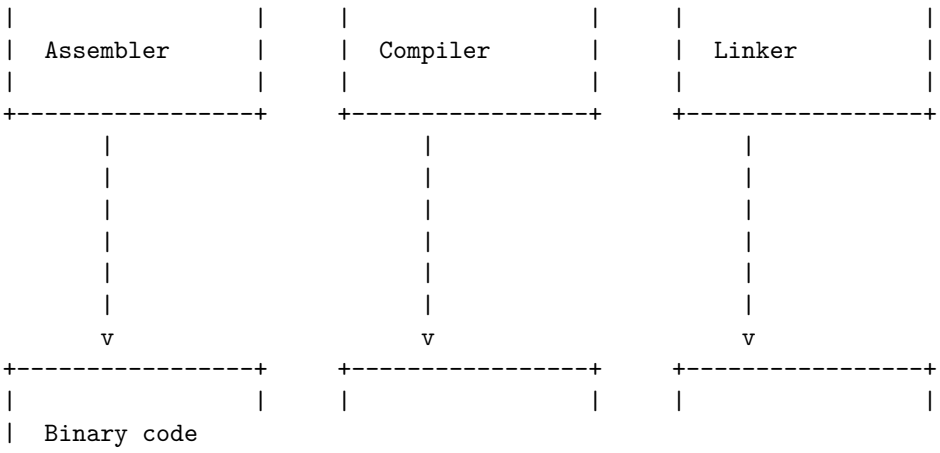
Reverse engineering of software is the process of analyzing an existing software product to understand its structure, functionality, and behavior. It can be used for various purposes, such as debugging, enhancing, modifying, or learning from the software. Reverse engineering of software typically involves the following steps:

- Disassembling: This is the process of converting the executable binary code of the software into a human-readable assembly language code. This can be done using tools such as IDA Pro, Hex Rays, or Hiew.
- Decompiling: This is the process of converting the assembly language code into a high-level programming language code, such as C, C++, or Java. This can be done using tools such as Hex Rays, Snowman, or Ghidra.
- Analyzing: This is the process of examining the decompiled code to identify the logic, algorithms, data structures, and interfaces of the software. This can be done using tools such as CFF Explorer, API Monitor, or WinHex.
- Modifying: This is the process of changing the decompiled code to add new features, fix bugs, or improve performance of the software. This can be done using tools such as Visual Studio, Eclipse, or Notepad++.
- Reassembling: This is the process of converting the modified code back into an executable binary code that can run on the target platform. This can be done using tools such as NASM, MASM, or GCC.

The following diagram illustrates the basic architecture of a reverse engineering of software process:




Disassembler
Decompiler
Analyzer
Assembly code
High-level code
Modified code

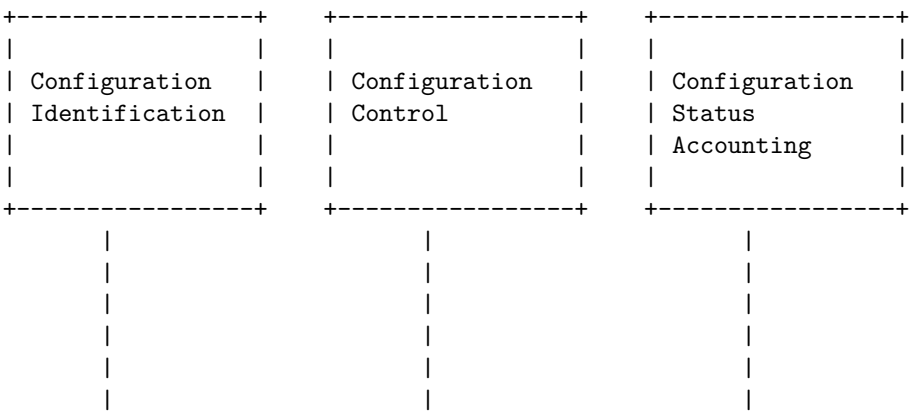


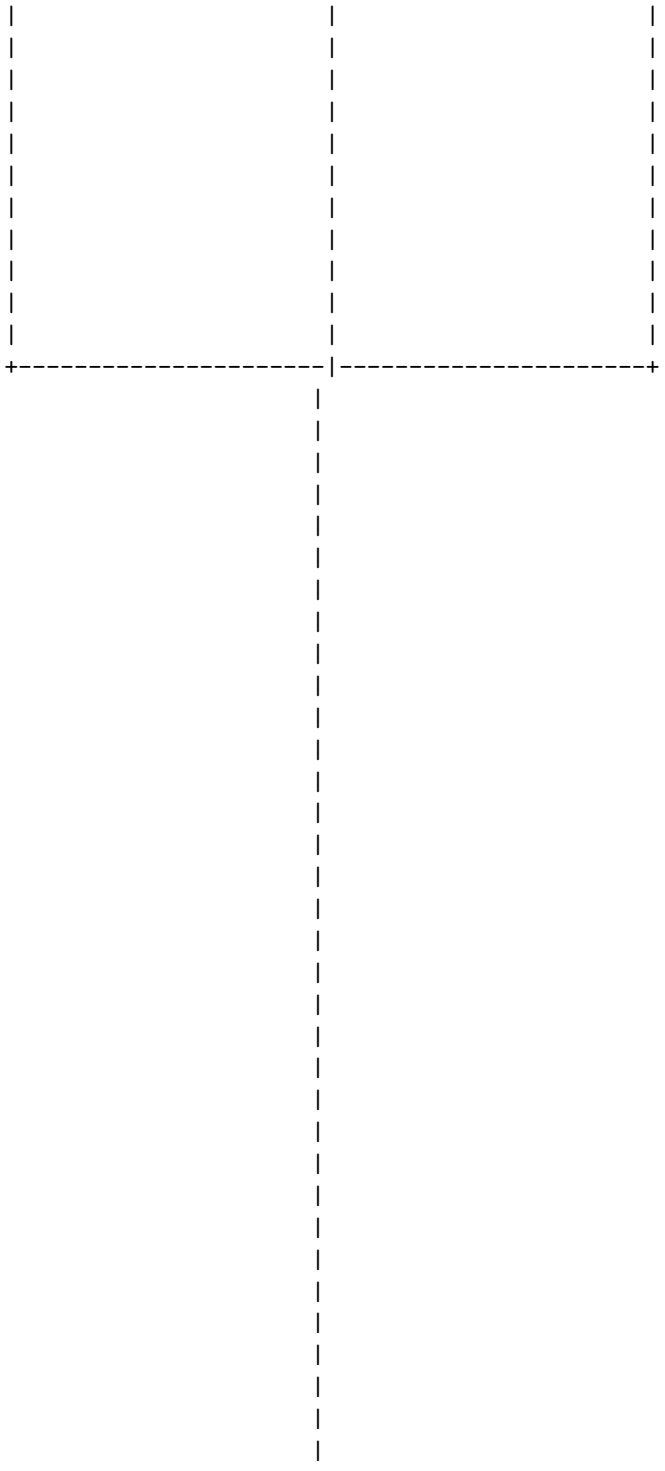
Software Configuration Management (SCM) is a process to systematically manage, organize, and control the changes in the documents, codes, and other entities during the Software Development Life Cycle. The primary goal is to increase productivity with minimal mistakes.

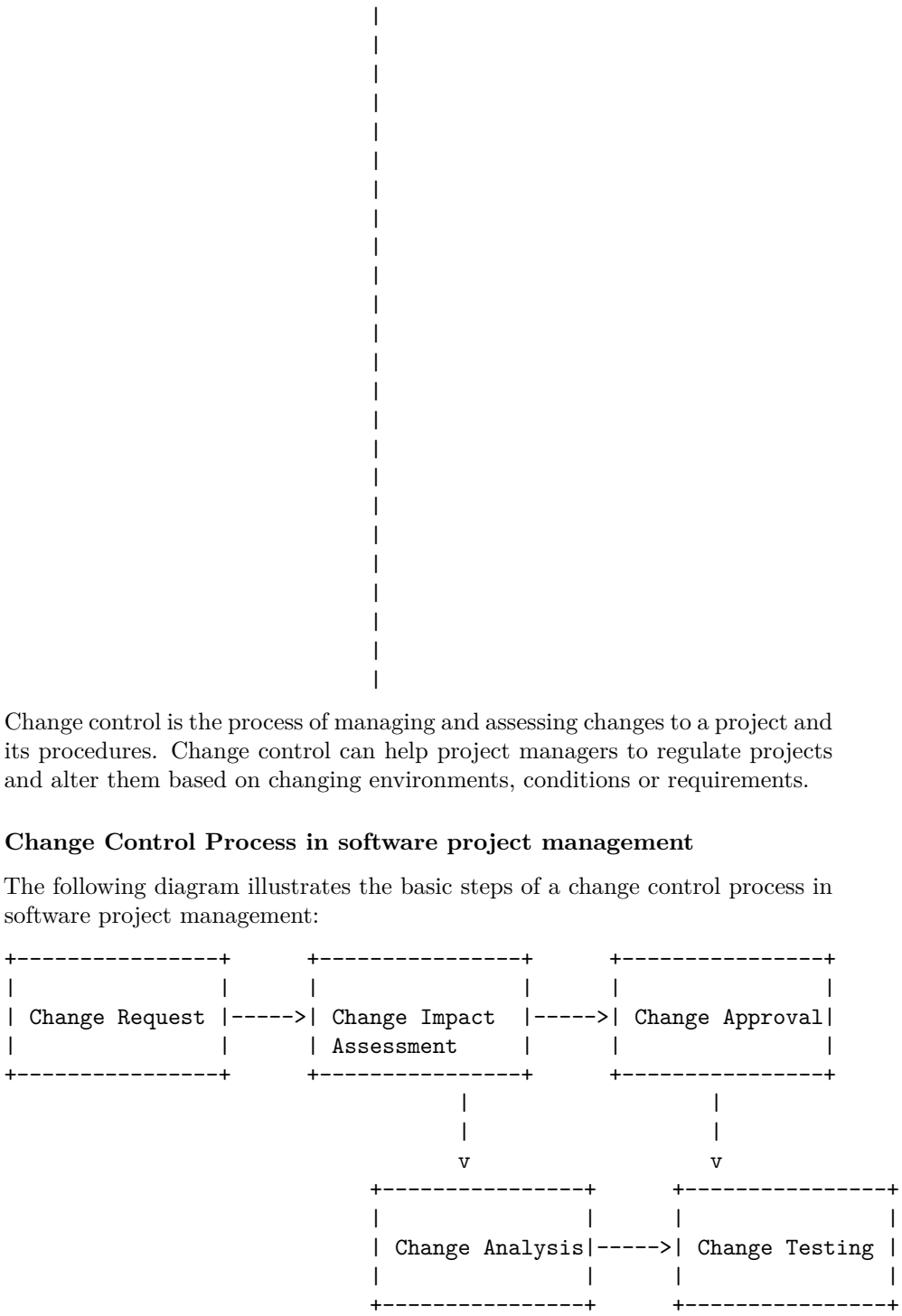
Software Configuration Management includes following activities :

- Configuration identification – Identifying configurations, configuration items and baselines
- Configuration control – Implementing a controlled change process
- Configuration status accounting – Recording and reporting all the necessary information on the status of the development process
- Configuration auditing – Ensuring that configurations contain all their intended parts and are sound with respect to their specifying documents
- Release management and delivery – Managing, storing, and installing software releases

The following diagram illustrates the basic architecture of a Software Configuration Management system using ASCII art:



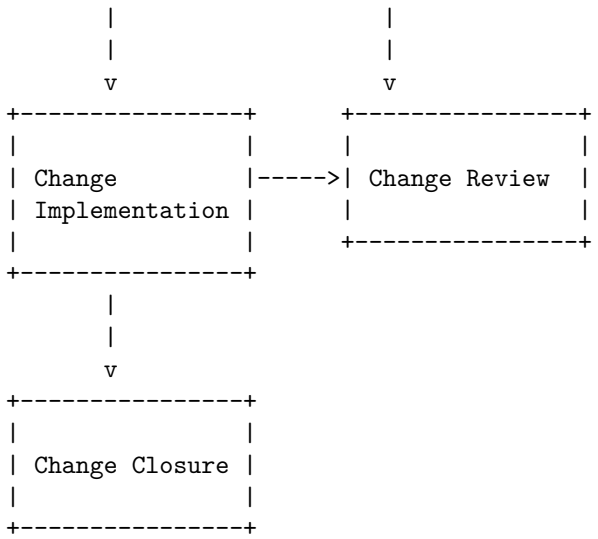




Change control is the process of managing and assessing changes to a project and its procedures. Change control can help project managers to regulate projects and alter them based on changing environments, conditions or requirements.

Change Control Process in software project management

The following diagram illustrates the basic steps of a change control process in software project management:



The steps are:

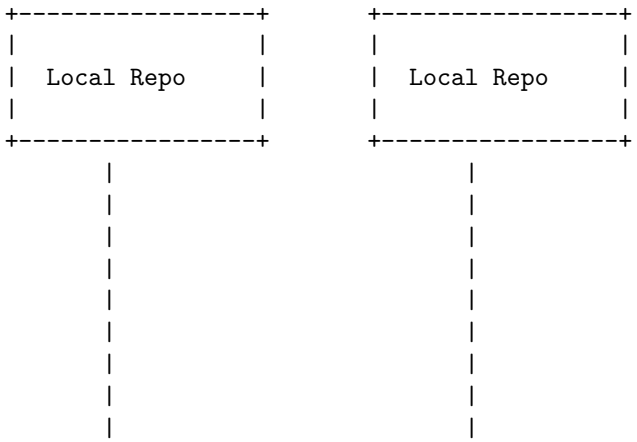
1. Change request initiation: A change is requested by anyone on the project team, a stakeholder, a client or a user . The change request is documented and categorized.
2. Change impact assessment: The project team meets and formally evaluates the change, considering its benefits, risks, costs, feasibility and alignment with the project objectives .
3. Change analysis: The project team analyzes the change and its impact on the project scope, schedule, budget, quality and resources . The team also identifies the required software changes and the dependencies among them.
4. Change approval: The change request is submitted to the change control board or the authorized stakeholders for review and approval . The change request can be approved, rejected, deferred or revised.
5. Change implementation: If the change request is approved, the project team implements the software changes according to the change plan . The team also updates the project documents and communicates the change to the relevant parties.
6. Change testing: The project team tests the software changes to ensure they meet the quality standards and the change objectives . The team also verifies that the software changes do not introduce any errors or defects.
7. Change review: The project team reviews the software changes and compares them with the original change request and the change plan . The team also evaluates the outcomes and benefits of the change.
8. Change closure: The project team closes the change request and documents the lessons learned from the change process . The team also celebrates the successful completion of the change.

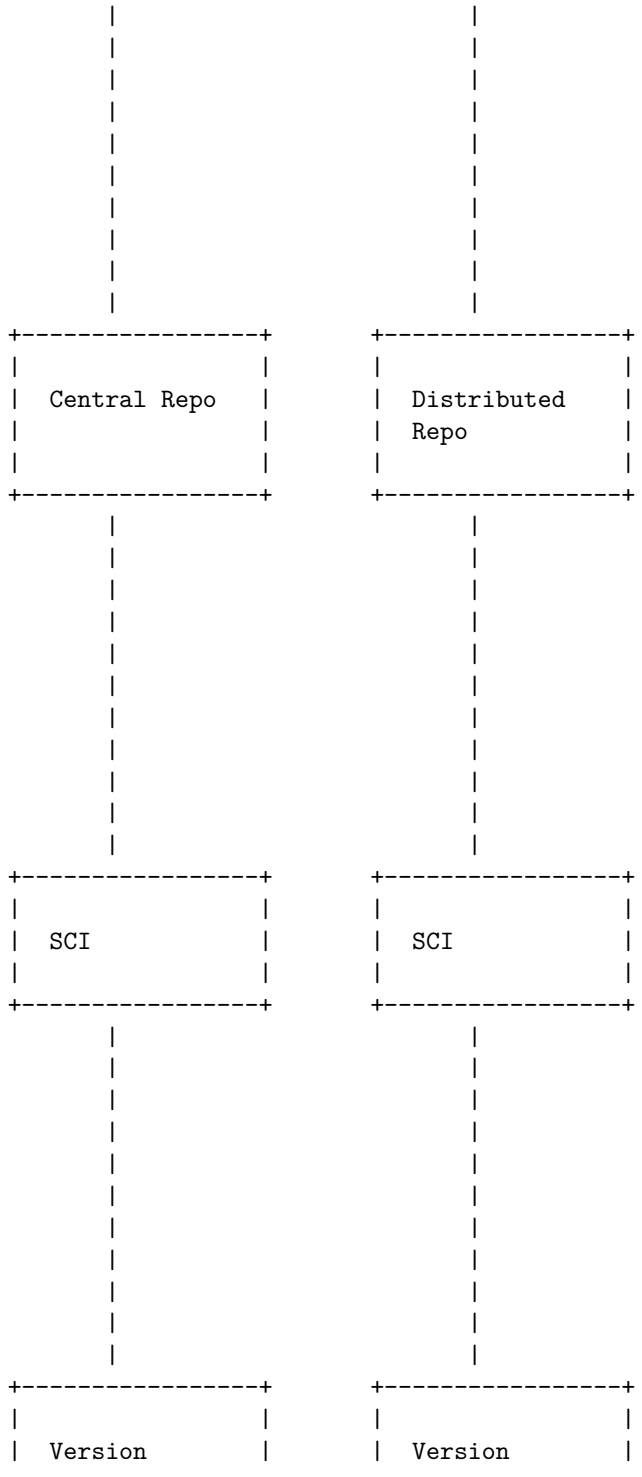
Software version control is the practice of tracking and managing changes to software code over time. Software version control systems are software tools that help software teams manage changes to source code over time. There are different types of software version control systems, such as local, centralized, and distributed.

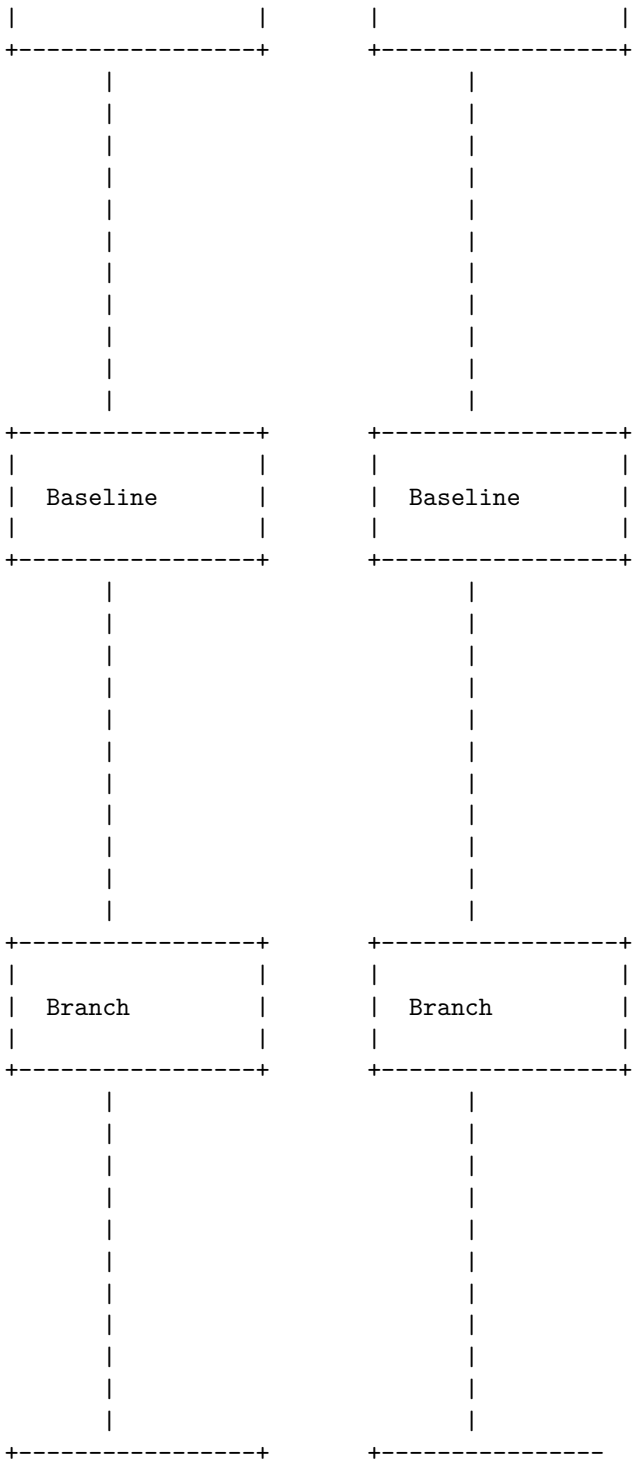
A software version control diagram is a graphical representation of the software version control system and its components. A software version control diagram can show the following elements:

- The software configuration items (SCIs) that are under version control. SCIs are the software work products that are subject to change, such as source code files, documents, images, etc.
- The version numbers that are assigned to each SCI. Version numbers are used to identify and distinguish different versions of the same SCI.
- The baselines that are established for each SCI. Baselines are the reference points that define the state of the SCIs at a given time. Baselines can be used to track the progress and quality of the software development process.
- The repositories that store the SCIs and their versions. Repositories are the databases that keep all the changes to the SCIs under version control. Repositories can be local, centralized, or distributed depending on the type of software version control system.
- The branches that are created from the mainline of development. Branches are the parallel lines of development that allow developers to work on different features or fixes without affecting the mainline. Branches can be merged back to the mainline when they are ready.
- The tags that are used to label specific versions of the SCIs. Tags are the names that are given to certain versions of the SCIs for easy reference. Tags can be used to mark important milestones, such as releases, tests, or bug fixes.

The following diagram illustrates the basic architecture of a software version control system in software project management:




Central Repo
Distributed Repo
SCI
SCI
Version
Version

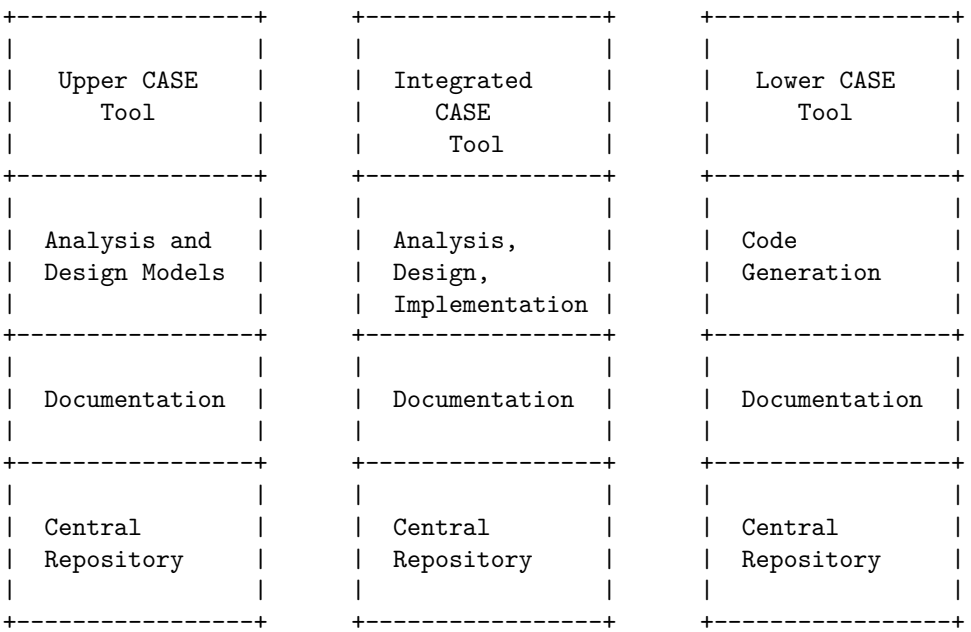

Baseline
Baseline
Branch
Branch

An Overview of CASE Tools in software project management

CASE tools are software applications that help automate various activities in the software development life cycle (SDLC). They are used by software project managers, engineers, and analysts to develop software systems of high quality and free of defects. CASE tools can be classified into three categories based on their functionality and scope:

- Upper CASE tools: These tools support the early stages of SDLC, such as analysis, design, and specification. They help in creating diagrams, models, and documents that represent the system requirements and architecture. Examples of upper CASE tools are data flow diagram tools, entity-relationship diagram tools, and structured analysis and design tools.
- Lower CASE tools: These tools support the later stages of SDLC, such as implementation, testing, and maintenance. They help in generating code, debugging, testing, and documenting the software system. Examples of lower CASE tools are code editors, compilers, debuggers, and testing tools.
- Integrated CASE tools: These tools combine the features of both upper and lower CASE tools, and provide a seamless transition from one stage to another. They also facilitate communication and collaboration among the project team members and stakeholders. Examples of integrated CASE tools are Rational Rose, Visual Studio, and Eclipse.

The following diagram illustrates the basic architecture of a CASE tool:



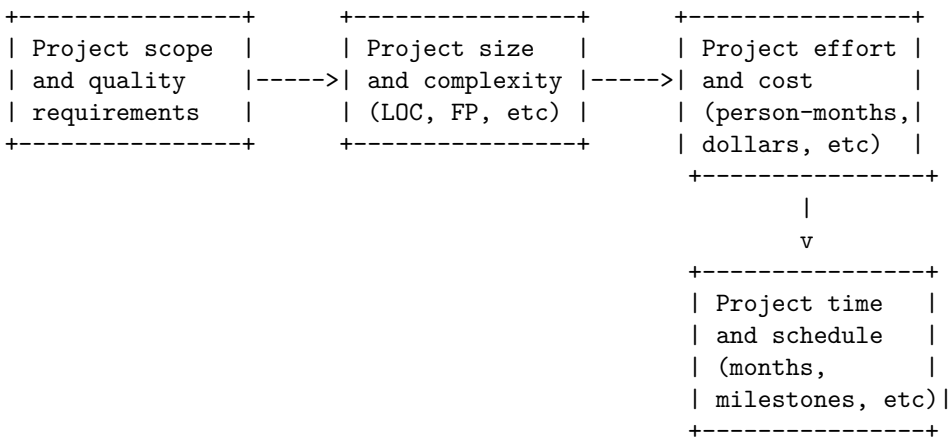
The central repository is a common database that stores all the information related to the software project, such as models, code, documents, and test cases.

It enables data sharing and consistency among the different CASE tools and project team members. It also supports version control, configuration management, and change management.

Estimation of various parameters such as cost and time in software project management is a process of predicting the resources and duration required for a software project based on its scope, complexity, and quality. There are different methods and techniques for estimating software projects, such as parametric, analogy, expert judgment, bottom-up, top-down, and so on. Each method has its own advantages and disadvantages, and the choice of the best method depends on the project characteristics, availability of data, and accuracy required.

One of the most common and widely used methods for software project estimation is the parametric method, which uses a set of mathematical formulas or models to calculate the project parameters based on some measurable attributes of the project, such as size, functionality, or lines of code. The parametric method is based on the assumption that there is a statistical relationship between the project attributes and the project parameters, and that this relationship can be derived from historical data or industry standards. The parametric method is also known as the COCOMO (Constructive Cost Model) method, which was proposed by Barry Boehm in 1981 and is based on the study of 63 projects.

The following diagram illustrates the basic steps of the parametric method for software project estimation:



The diagram shows that the parametric method starts with defining the project scope and quality requirements, which are the inputs for estimating the project size and complexity. The project size and complexity can be measured by various metrics, such as lines of code (LOC), function points (FP), use cases, features, and so on. The project size and complexity are then used as the inputs for estimating the project effort and cost, which are the main outputs of the parametric method. The project effort and cost can be expressed by vari-

ous units, such as person-months, person-hours, dollars, euros, and so on. The project effort and cost are then used as the inputs for estimating the project time and schedule, which are the secondary outputs of the parametric method. The project time and schedule can be expressed by various units, such as months, weeks, days, milestones, deliverables, and so on.

The parametric method is a quantitative and objective technique for software project estimation, which can provide consistent and reliable results if the input data and the estimation model are accurate and valid. However, the parametric method also has some limitations and challenges, such as:

- The availability and quality of historical data and industry standards may vary depending on the project domain, type, and technology.
- The estimation model may not capture all the factors and variables that affect the project parameters, such as risks, uncertainties, dependencies, and human factors.
- The estimation model may need to be calibrated and validated for different project contexts and environments, such as organizational culture, team skills, and tools.
- The estimation model may need to be updated and refined as the project progresses and more information becomes available.

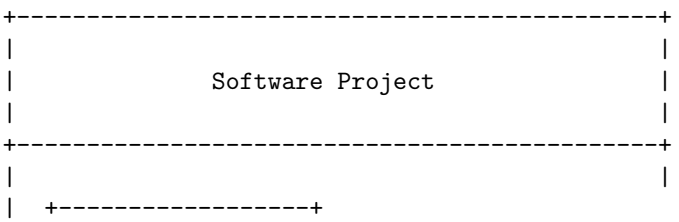
Efforts to improve software quality in software project management involve various techniques and practices that aim to deliver high-quality software products that meet the requirements and expectations of the stakeholders. Some of the common efforts to improve software quality are:

- Adoption of software development methodologies: This is the first step that must be taken to ensure project success. Different methodologies have different advantages and disadvantages, and choosing the most suitable one for the project can help to improve the software quality. Some of the popular methodologies are agile, waterfall, scrum, kanban, etc.
- Test early from the beginning: Testing is not only a final stage of software development, but also an integral part of the whole process. Testing early and often can help to detect and fix defects, errors, and bugs before they become costly and complex. Testing can also help to verify and validate the software functionality, performance, usability, security, and reliability.
- Implement automated testing: Automated testing is the use of software tools and scripts to execute test cases and check the results without human intervention. Automated testing can help to save time, effort, and resources, as well as to increase the test coverage, accuracy, and consistency. Automated testing can also help to reduce human errors and biases.
- Use continuous integration and deployment (CI/CD): CI/CD is a software development practice that involves integrating the code changes from multiple developers into a shared repository and deploying the software to the production environment frequently and automatically. CI/CD can help to improve the software quality by ensuring that the code is always tested, integrated, and deployed, as well as by enabling fast feedback and delivery.

- Conduct regular performance and security testing: Performance and security testing are two important aspects of software quality that should not be overlooked. Performance testing is the process of measuring and evaluating the speed, scalability, stability, and responsiveness of the software under different workloads and conditions. Security testing is the process of identifying and eliminating the vulnerabilities and risks that may compromise the confidentiality, integrity, and availability of the software and its data. Performance and security testing can help to improve the software quality by ensuring that the software can handle the expected and unexpected scenarios and can protect itself from malicious attacks.
- Establish clear requirements and priorities: Requirements are the specifications and expectations of the software product from the perspective of the stakeholders, such as the customers, users, developers, managers, etc. Priorities are the order of importance and urgency of the requirements. Establishing clear requirements and priorities can help to improve the software quality by providing a clear and shared vision of the software product, as well as by guiding the development and testing activities.
- Great communication: Communication is the exchange of information and ideas among the stakeholders involved in the software project. Great communication can help to improve the software quality by ensuring that everyone is on the same page, that the requirements and feedback are understood and addressed, that the issues and risks are identified and resolved, and that the collaboration and coordination are effective and efficient.
- Team training and development: Team training and development are the activities that aim to improve the skills, knowledge, and abilities of the team members involved in the software project. Team training and development can help to improve the software quality by enhancing the team's competence, confidence, and performance, as well as by fostering a culture of learning and improvement.
- Use of quality tools and standards: Quality tools and standards are the instruments and guidelines that help to measure, monitor, and improve the software quality. Quality tools and standards can help to improve the software quality by providing a systematic and consistent approach to software development and testing, as well as by ensuring compliance with the best practices and regulations. Some of the common quality tools and standards are code analysis tools, testing tools, quality metrics, quality models, quality audits, quality certifications, etc.
- Risk management: Risk management is the process of identifying, analyzing, evaluating, and mitigating the potential threats and uncertainties that may affect the software project and its outcomes. Risk management can help to improve the software quality by preventing or minimizing the negative impacts of the risks, as well as by exploiting or maximizing the positive opportunities of the risks.

The following diagram illustrates the basic architecture of the efforts to improve

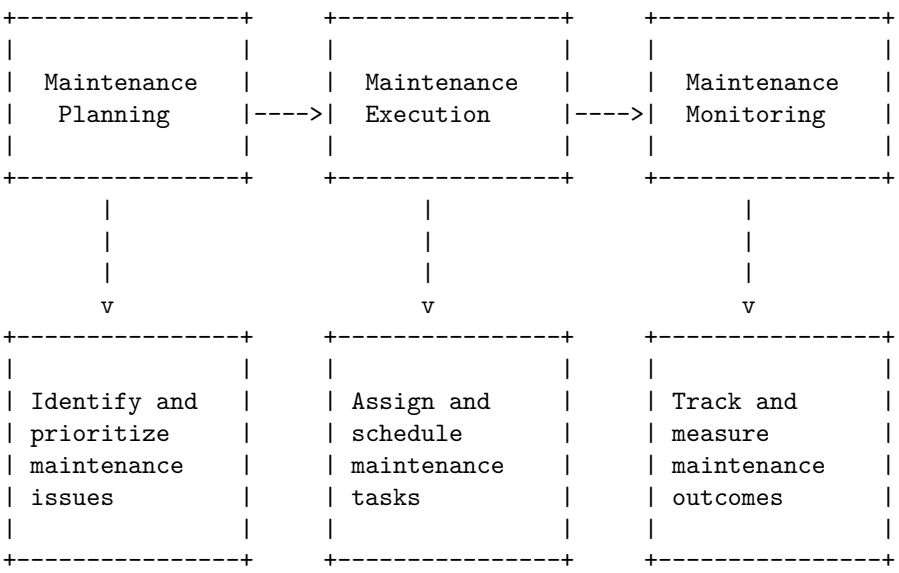
software quality in software project management using ASCII art:



Schedule/Duration of Maintenance in software project management is the process of planning, executing and monitoring the activities related to maintaining and improving the quality, performance and functionality of a software product after its release. It involves identifying the defects, bugs, errors and enhancements that need to be addressed, prioritizing them based on their impact and urgency, assigning the resources and time required to fix them, and tracking the progress and results of the maintenance work.

One possible way to draw a detailed ASCII diagram for Schedule/Duration of Maintenance in software project management is as follows:

Schedule/Duration of Maintenance in software project management



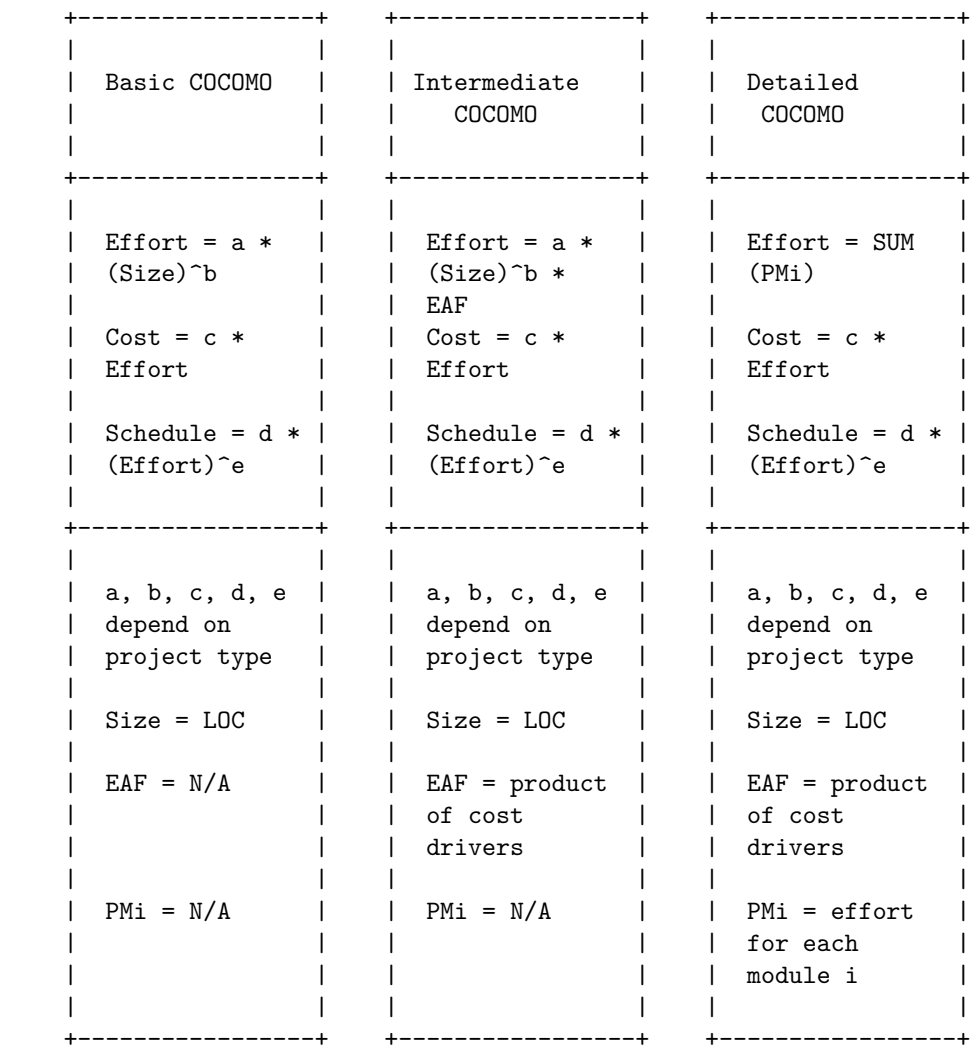
Constructive Cost Models (COCOMO) in software project management

COCOMO is a software cost estimation model that predicts the effort, cost, and schedule of a software project based on the size of the software measured in lines

of code (LOC) and other factors such as project type, development mode, and cost drivers .

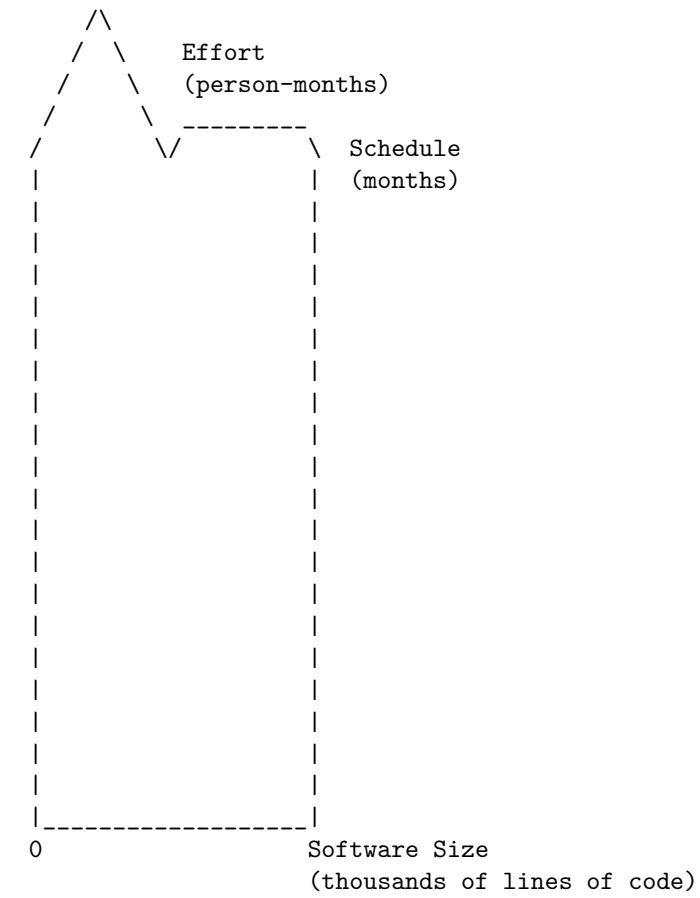
COCOMO has three levels of complexity: basic, intermediate, and detailed. Each level provides more accuracy and detail in the estimation, but also requires more information and parameters to be specified .

The following diagram illustrates the basic architecture of a COCOMO model using ASCII art:



: Software Engineering | COCOMO Model - GeeksforGeeks Constructive Cost Model (COCOMO) - Techopedia.com
COCOMO Model | Types of COCOMO Model | Pros and Cons - EDUCBA
COCOMO - Wikipedia

Resource Allocation Models (RAIM) in software project management are methods or frameworks for estimating the time, effort, and resources required to complete a software project of a given size and complexity. One of the most widely used RAIM is the Putnam model, which uses the Norden/Rayleigh curve to describe the relationship between software size, effort, schedule, and defect rate. The following diagram illustrates the basic architecture of the Putnam model using ASCII characters:



The diagram shows that the effort required to complete a software project increases exponentially with the software size, while the schedule increases logarithmically. The peak of the curve represents the optimal point of resource allocation, where the effort and schedule are balanced and the defect rate is minimized. The Putnam model uses a formula to calculate the effort and schedule based on the software size and a productivity factor that reflects the quality of the development team and the environment. The formula is:

Effort = A * Size^B

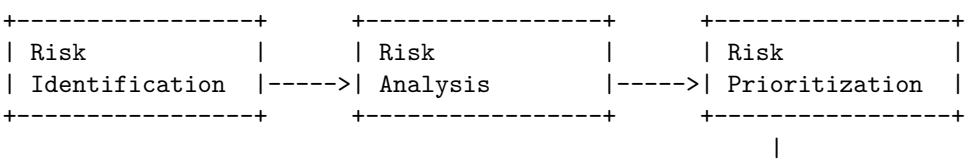
Schedule = C * Effort^(1/3) * Size^(1/9)

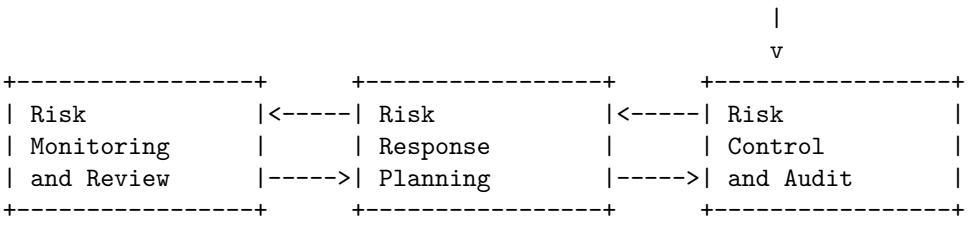
where A, B, and C are constants that depend on the productivity factor. The Putnam model can be used to estimate the resource allocation for a software project at an early stage, as well as to monitor and control the project progress and quality during the development process. However, the model also has some limitations, such as assuming a fixed software size and ignoring the effects of changing requirements, technology, and team dynamics. Therefore, the Putnam model should be used with caution and adjusted according to the specific characteristics and context of each software project.

Software Risk Analysis and Management is a process of identifying, assessing, and mitigating the risks that may affect the quality, cost, or schedule of a software project. It involves the following steps:

- Risk identification: This is the process of finding out the possible sources and causes of risk in a software project. Some of the common risk sources are requirements, design, technology, testing, resources, stakeholders, and environment. Risk identification can be done using various techniques, such as checklists, brainstorming, interviews, surveys, and historical data.
- Risk analysis: This is the process of estimating the probability and impact of each identified risk on the project objectives. Probability is the likelihood of the risk occurring, and impact is the severity of the consequence if the risk occurs. Risk analysis can be done using qualitative or quantitative methods, such as risk matrices, risk scoring, risk simulation, and sensitivity analysis.
- Risk prioritization: This is the process of ranking the risks according to their importance and urgency. The most critical risks are those that have high probability and high impact, and they should be addressed first. Risk prioritization can be done using various criteria, such as risk exposure, risk value, risk index, and risk ranking.
- Risk response planning: This is the process of developing strategies and actions to reduce, avoid, transfer, or accept the risks. Risk response planning can be done using various techniques, such as risk mitigation, risk contingency, risk avoidance, risk transfer, and risk acceptance.
- Risk monitoring and control: This is the process of tracking and reviewing the status and effectiveness of the risk responses, and taking corrective actions if needed. Risk monitoring and control can be done using various tools, such as risk registers, risk reports, risk audits, and risk reviews.

The following diagram illustrates the basic architecture of a software risk analysis and management process in a software project management:



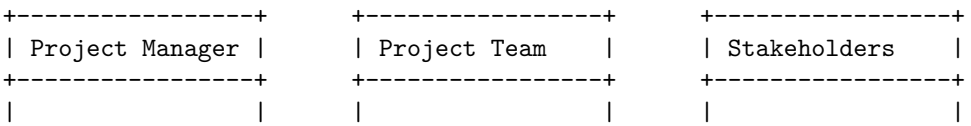


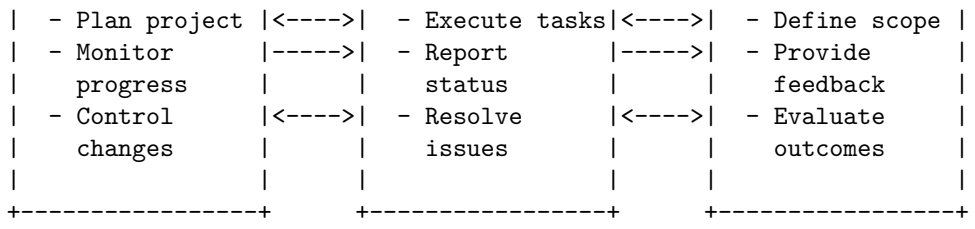
Software project management is the process of planning, organizing, executing, monitoring and controlling software projects. There are different types of diagrams that can be used to represent various aspects of software project management, such as:

- Project planning diagram: This diagram shows the scope, objectives, deliverables, milestones, tasks and dependencies of a software project. It can be used to define the project scope, schedule, budget and resources.
- Gantt chart: This diagram shows the start and end dates, duration and progress of each task in a software project. It can be used to track the project schedule, identify critical paths, manage dependencies and allocate resources.
- Work breakdown structure (WBS) diagram: This diagram shows the hierarchical decomposition of a software project into smaller and manageable components. It can be used to clarify the project scope, assign responsibilities, estimate costs and monitor progress.
- Flow chart: This diagram shows the sequence of steps, decisions and actions involved in a software process or activity. It can be used to describe the project workflow, document the project procedures, identify bottlenecks and improve the project efficiency.
- Resource allocation chart: This diagram shows the distribution of resources (such as human, material, equipment, etc.) across different tasks or phases of a software project. It can be used to plan the project resources, optimize the resource utilization, avoid resource conflicts and balance the resource workload.
- Project timeline: This diagram shows the chronological order of events, milestones, deliverables and deadlines in a software project. It can be used to communicate the project expectations, highlight the project achievements, manage the project risks and report the project status.

The following diagram illustrates the basic architecture of a software project management system using ASCII characters:

Software Project Management




- Plan project
- Monitor progress
- Control changes
- Execute tasks
- Report status
- Resolve issues
- Define scope
- Provide feedback
- Evaluate outcomes